The Rust Programming Language

- Note: I did the Brown University version of *The Rust Programming Language* at https://rust-book.cs.brown.edu. Passages are quoted or adapted from the book, which is licensed under MIT/Apache-2.0.

Chapter 1 - Getting Started

1.2 - Hello, World

- If you use more than one word in a filename, the convention is to separate the words with underscores.
- The `main` function is the first code that runs in every executable Rust program. Declare functions with `fn`. Rust requires curly brackets around function bodies, and it's good style to place the opening curly bracket on the same line as the function declaration with one space in between.
- `println!` is a macro. Macros end in `!`.
- Most lines of Rust code end with a semicolon.

1.3 - Hello, Cargo

- Cargo handles building your code and downloading and building your dependencies. `cargo new hello_cargo` will create a `hello_cargo` project. It also makes this a Git repository and generates a .gitignore if this isn't done within an existing Git repository. It creates *Cargo.toml* and a *src* directory with a *main.rs* file inside. The default *Cargo.toml*:

```
[package]
name = "hello_cargo"
version = "0.1.0"
edition = "2024"

[dependencies]
```

- In Rust, packages of code are referred to as *crates*. We will add crates to dependencies in Chapter 2.

- Cargo expects your source files to live inside the *src* directory. The top level project directory is for README files, license information, configuration files, and anything else not related to your code.
- If you started your project without Cargo, you can convert it by moving project code into the *src* directory and creating a *Cargo.toml* file. One easy way to create a *Cargo.toml* file is to run `cargo init`.
- In the `hello_cargo` project, `cargo build` will generate a *target/debug/hello_cargo* executable. The default build is a debug build. The first time you run `cargo build`, it will create *Cargo.lock* which keeps track of exact versions of dependencies in your project. You never need to manually edit *Cargo.lock*. `cargo run` will build the project and run the resultant executable in one command. `cargo check` makes sure your code compiles without generating an executable, and is often much faster. Many Rustaceans run `cargo check` periodically as they write their program to make sure it compiles then run `cargo build` when they're ready to use the executable.
- `cargo build --release` compiles your project with optimization and puts it in the *target/release* directory. Be sure to build a release if you are benchmarking. Releases are compiled with optimizations to make them run faster, but they take longer to compile.

Chapter 2 - Programming a Guessing Game

- Our first version of the game will allow a player to input a guess and echo it back:

```rust
// ch02_guessing_game/src/main.rs
use std::io;

fn main() {
    println!("Guess the number!");

    println!("Please input your guess.");

    let mut guess = String::new();

    io::stdin()
        .read_line(&mut guess)
        .expect("Failed to read line");

    println!("You guessed: {guess}");
}
```

- To obtain user input we need `io` from the `std` (standard library). By default, Rust has a set of items defined in the standard library that it brings into scope for every program called the *prelude*. Any type not in the prelude needs to be brought into scope with the `use` statement.

- We use the `let` command to create a variable. In Rust, variables are immutable by default. We add `mut` before the variable name to make it mutable. `String::new` returns a new instance of a `String`. `String` is a string type provided by the standard library that is a growable, UTF-8 encoded bit of text. `::` means `new` is an associated function of the `String` type. The `new` function creates a new, empty instance of a `String`.

- Because we did `use std::io`, we can call the `stdin` function from the `io` module. If we hadn't imported it with `use`, we could still call it using `std::io::stdin`. The `stdin` function returns an instance of `std::io::Stdin` which represents a handle to the standard input for your terminal. The `.read_line(&mut guess)` line calls the `read_line` method on the standard input handle to get input from the user. We pass `&mut guess` to it as an argument to tell it what string to store the user input in. `read_line` takes whatever the user types into standard input and appends it into a string, so it always takes a string as an argument. The string argument needs to be mutable so that the method can change the string's content. The `&` indicates that this argument is a *reference*, which gives you a way to let multiple parts of your code access one piece of data without needing to copy that data into memory multiple times. Like variables, references are immutable by default. Therefore we write `&mut guess` instead of `&guess` to make it mutable.

- It is often useful to break up long lines when you call a method with the `.method_name()` syntax. Now `.expect("Failed to read line")` is called on the `Result` value returned by `.read_line`. `Result` is an *enumeration*, often called an *enum*, which is a type that can be in one of multiple possible states. We call each possible state a *variant*. `Result`'s variants are `Ok` and `Err` and it is meant to encode error-handling information. The `Ok` variant indicates the operation was successful, and it contains the successfully generated value. The `Err` variant means the operation failed, and it contains information about how or why the operation failed. Values of the `Result` type, like values of any type, have methods defined on them. An instance of `Result` has an `expect` method that you can call. If the `Result` is an `Err` value, `expect` will cause the program to crash and display the message that you passed as an argument to `expect`. So if `read_line` returns an `Err`, it would likely be the result of an error coming from the underlying operating system. If this instance of `Result` is an `Ok` value, `expect` will take the return value that `Ok` is holding and return just that value to you so that you can use it. In this case, that value is the number of bytes in the user's input. If you don't call `expect`, the program will compile but you will get a warning: `warning: unused `Result` that must be used`. The right way to suppress the warning is to actually write error handling code, but in our case we just want to crash if there is a problem so we use `expect`. We'll learn how to recover from errors in Chapter 9.

- `println!("You guessed: {guess}");` prints the string that now contains the user's input. The `{}` set of curly brackets is a placeholder. When printing the value of a variable, the variable name can go inside the curly brackets. When printing the result of evaluating an expression, place empty curly brackets in the format string, then follow the format string with a comma separated list of expressions to print in each empty curly bracket placeholder.

```
// printing a variable and the result of an expression in one call
let x = 5;
let y = 10;

println!("x = {x} and y + 2 = {}", y + 2); // x = 5 and y + 2 = 12
```

- Next we'll include a secret number for the user to guess. We are going to generate one between 1 and 100. There is no random number functionality in the standard library, but the Rust team provides a `rand` crate. The code we have been working on is a binary crate, which is an executable. `rand` is a library crate, which contains code that is intended to be used in other programs and cannot be executed on its own. Add `rand` to your Cargo.toml dependencies:

```
[dependencies]
rand = "0.8.5"
```

- "0.8.5" means any version at least 0.8.5 and below 0.9.0 in SemVer. When we include an external dependency, Cargo fetches the latest version of everything that dependency needs from the *registry*, which is a copy of data from https://crates.io/. You can watch the dependencies build by running `cargo build` now that you've updated *Cargo.toml*. Now that we have run `cargo build`, we have a *Cargo.lock* file that specifies which versions to use to build our project. *Cargo.lock* is often checked into source control and is important for reproducible builds. `cargo update` will figure out the latest versions from your specifications in *Cargo.toml* and update *Cargo.lock*.

```
// ch02_guessing_game/src/main.rs
use std::io;
use rand::Rng;

fn main() {
    println!("Guess the number!");
    let secret_number = rand::thread_rng().gen_range(1..=100);
    println!("The secret number is: {secret_number}");
    // --snip--
}
```

- We start with `use rand::Rng`. The `Rng` trait defines methods that random number generators implement and needs to be in scope to use those methods. The `rand::thread_rng` function gives us a generator local to the current thread of execution and seeded by the operating system. We call the `.gen_range` method and give it a range expression as an argument for it to generate a number in the range. `gen_range` is defined by the `Rng` trait that we brought into scope. The kind of range expression we are using takes the form of `start..=end` and is inclusive on the lower and upper bounds.

- You don't need to know everything in every crate. Since we have declared dependencies, `cargo doc --open` will generate documentation for our code and dependencies and open it in the browser.

- Note the following code will not compile:

```rust
// ch02_guessing_game/src/main.rs
// DOES NOT COMPILE! error[E0308]: mismatched types
use std::cmp::Ordering;
use std::io;

use rand::Rng;

fn main() {
    // --snip--
    println!("You guessed: {guess}");

    match guess.cmp(&secret_number) {
        Ordering::Less => println!("Too small!"),
        Ordering::Greater => println!("Too big!"),
        Ordering::Equal => println!("You win!"),
    }
}
```

- Above we `use std::cmp::Ordering` to bring it into scope. The `Ordering` type is another enum and has the variants `Less`, `Greater`, and `Equal`, the three outcomes possible when you compare two values. The `cmp` method compares two values and can be called on anything that can be compared. It takes a reference to whatever you want to compare with. Here we compare `guess` to `secret_number`. It returns a variant of `Ordering` and we use a `match` expression to decide what to do next based on which variant of `Ordering` was returned from the call. A `match` expression is made of *arms*. An arm consists of a *pattern* to match against, and the code that should be run if the value given to `match` fits that arm's pattern. Rust takes the value given to `match` and looks through each arm's pattern in turn. Patterns and `match` let you express a variety of situations your code might encounter and how to handle them all. The `match` expression ends after the first successful match.

- Rust is statically typed but you don't always need to provide types because it can infer them. Here we are using an `i32` where it expects a `String`. `guess` is a `String`, which Rust inferred. The `secret_number` is a random number, so an integer, more specifically an `i32` because Rust will default to that unless it is otherwise specified. Rust cannot compare a string and a number type. So we should convert the `String` to a number type.

```rust
// ch02_guessing_game/src/main.rs
use std::cmp::Ordering;
use std::io;

use rand::Rng;

fn main() {
    // --snip--
    let mut guess = String::new();

    io::stdin()
        .read_line(&mut guess)
        .expect("Failed to read line");

    let guess: u32 = guess.trim().parse().expect("Please type a number!");
    println!("You guessed: {guess}");

    match guess.cmp(&secret_number) {
        Ordering::Less => println!("Too small!"),
        Ordering::Greater => println!("Too big!"),
        Ordering::Equal => println!("You win!"),
    }
}
```

- Rust allows us to declare a new `guess` variable which *shadows* the existing `guess` variable. We often shadow the old variable when we want to convert a value from one type to another type. We bind the new variable to the expression `guess.trim().parse()`. The `guess` in the expression is the original string. `trim` removes any whitespace at the beginning and end, which we must do before converting it to a `u32`, which can only contain numerical data. The user must press `enter` to satisfy `read_line`, so `guess` contains a trailing newline character. The `parse` method on strings converts a string to another type. It can tell from the fact we are assigning it to a `u32` what to convert to. Additionally, since Rust can tell this is a `u32` it will make the `secret_number` a `u32` as well so we can compare them. The `parse` method only works on characters that can logically be converted into numbers so can easily cause errors. Because it might fail, it returns a `Result` type. If `parse` can successfully convert the string to a number, it will return the `Ok` variant of `Result`, and `expect` will return the number that we want from the `Ok` value.
- Now we want to allow multiple guesses with `loop`:

```
use std::cmp::Ordering;
use std::io;

use rand::Rng;

fn main() {
    println!("Guess the number!");

    let secret_number = rand::thread_rng().gen_range(1..=100);
    println!("The secret number is: {secret_number}");

    loop {
        println!("Please input your guess.");
        // --snip--
        match guess.cmp(&secret_number) {
            Ordering::Less => println!("Too small!"),
            Ordering::Greater => println!("Too big!"),
            Ordering::Equal => println!("You win!"),
        }
    }
}
```

- As you can see, we moved everything from the guess input prompt onward into a loop. We didn't even have to indent by hand thanks to `rustfmt`. There is only one problem, the `loop` asks for another guess forever and there's no way to quit other than ctrl-C or entering a non-number answer to crash. Adding a `break` after they win will fix that:

```
// ch02_guessing_game/src/main.rs
// --snip--
        match guess.cmp(&secret_number) {
            Ordering::Less => println!("Too small!"),
            Ordering::Greater => println!("Too big!"),
            Ordering::Equal => {
                println!("You win!");
                break;
            }
        }
    }
}
```

- We also want to handle invalid input better than a crash. `continue` conveniently lets you start over the loop, so we'll ignore invalid input and do that to take another guess.

```
// ch02_guessing_game/src/main.rs
// --snip--
        io::stdin()
            .read_line(&mut guess)
            .expect("Failed to read line");
        let guess: u32 = match guess.trim().parse() {
            Ok(num) => num,
            Err(_) => continue,
        };
        println!("You guessed: {guess}");
// --snip--
```

- We switch from an `expect` which crashes the program to a `match` on the `Result`. `parse` returns the `Result` and `Result` is an enum that has variants of `Ok` and `Err`. The `Ok` contains a `num` that is the parse result. Otherwise, it returns an `Err` and we ignore the error message and `continue`. `_` is a catch-all value; in this example we are saying match all `Err` values, no matter what information they have inside them.
- We'll also delete the `println!` that displays the secret number, since that was just for testing. And here is our final code:

```rust
// ch02_guessing_game/src/main.rs
use std::cmp::Ordering;
use std::io;

use rand::Rng;

fn main() {
    println!("Guess the number!");
    let secret_number = rand::thread_rng().gen_range(1..=100);

    loop {
        println!("Please input your guess.");
        let mut guess = String::new();
        io::stdin()
            .read_line(&mut guess)
            .expect("Failed to read line");
        let guess: u32 = match guess.trim().parse() {
            Ok(num) => num,
            Err(_) => continue,
        };
        println!("You guessed: {guess}");

        match guess.cmp(&secret_number) {
            Ordering::Less => println!("Too small!"),
            Ordering::Greater => println!("Too big!"),
            Ordering::Equal => {
                println!("You win!");
                break;
            }
        }
    }
}
```

Chapter 3 - Common Programming Concepts

3.1 - Variables and Mutability

- Variables are immutable by default. When a variable is immutable, once a value is bound to a name you cannot change that value. If you try to assign twice to an immutable variable `x`, you will get an `error[E0384]: cannot assign twice to immutable variable `x`` . You can declare them mutable by adding `mut` in front of the variable name when defining

them with `let`. Note that you can reassign a mutable variable, but its type needs to remain the same or you will get an `error[E0308]: mismatched types` error.

- You can declare a constant with `const` instead of `let`. You cannot use `mut` with a `const`; they are always immutable. The type of the value must be annotated. Constants can be declared in any scope, including global scope. Also, they must be set to a constant expression and not the result of a value that could only be computed at runtime. Rust's naming convention for constants is to use all capitals with underscores separating words. `const THREE_HOURS_IN_SECONDS: u32 = 60 * 60 * 3;` You see that Rust can evaluate a limited set of operations at compile time, so we can easily compute the number of seconds in 3 hours. They are valid for the entire time the program runs so can be useful for values multiple parts of the program might need to know about.
- When you declare a new variable with the same name as a previous variable, we say the original variable is shadowed by the new one. The original variable is inaccessible until the new variable's scope ends. Since we're creating a new variable, we can re-use the name with a different type when we shadow a variable. You can also shadow an immutable variable.

3.2 - Data Types

- Every value in Rust is of a certain *data type*. We'll look at scalar and compound types. Rust is *statically typed*, which means that it must know the types of all variables at compile time. The compiler can usually infer what type we want to use based on the value and how we use it. In cases when many types are possible, we must add a type annotation like: `let guess: u32 = "42".parse().expect("Not a number!");`. Without the `u32` that would cause an `error[E0284]: type annotations needed`.

- A *scalar* type represents a single value. Rust has four primary scalar types: integers, floating-point numbers, Booleans, and characters. `u32` indicates an unsigned integer (signed integer types start with `i` instead of `u`) that takes up 32 bits of space. The `32` could be `8`, `16`, `32`, `64`, or `128`. There is also an architecture dependent `isize` and `usize`. These are 64 bits on 64-bit architectures and 32 bits on 32-bit architectures. Signed integers can store numbers from -(2^(n-1)) to 2^(n-1) - 1 inclusive where *n* is the number of bits. Unsigned integers can store numbers from 0 to 2^n - 1. So an `i8` holds from -128 to 127, and a `u8` holds 0 to 255.

- Numeric literals can use a `_` as a visual separator. You can also declare them a particular type with a type suffix, such as `57u8`. `98_222` is a decimal value. `0xff` is hex. `0o77` is octal. `0b1111_0000` is binary. And `b'A'` is a byte (`u8` only). Integer types default to `i32`.

The primary time you would use `isize` or `usize` is as an index to some sort of collection. Rust panics in debug mode when you overflow an integer and it performs *two's complement wrapping* in release mode. There are four families of methods to specifically handle overflow:

  - Wrap in all modes with the `wrapping_*` methods, such as `wrapping_add`.
  - Return the `None` value if there is overflow with the `checked_*` methods.
  - Return the value and a Boolean indicating overflow with the `overflowing_*` methods.
  - Saturate at the value's min or max values with the `saturating_*` methods.
- Rust has two primitive types for *floating-point numbers*, `f32` and `f64`. The default is `f64` since it is roughly the same speed on modern hardware while capable of more precision. All floating-point types are signed.

- Rust supports the basic mathematical operations you'd expect for all the number types: addition, subtraction, multiplication, division, and remainder. Integer division truncates toward zero to the nearest integer.

- Booleans are one byte and can be `true` or `false`. Their type is written `bool`. The main way to use them is in conditionals.

- Rust's `char` type is the language's most primitive alphabetic type. A literal in single quotes is a `char` and one in double quotes is a string slice (`&str`), not a `String`. Rust's `char` type is 4 bytes and represents a Unicode scalar value ranging from `U+0000` to `U+D7FF` and `U+E000` to `U+10FFFF` inclusive.

- *Compound types* can group multiple values into one type. Rust has two primitive compound types: tuples and arrays.

- A *tuple* is a general way of grouping together a number of values with a variety of types into one compound type. Tuples have a fixed length and cannot grow or shrink in size once declared. You declare one with a comma-separated list of values inside parentheses. Each position has a type, and types of different values don't need to be the same.

```
fn main() {
    let tup: (i32, f64, u8) = (500, 6.4, 1);  // the types are unnecessary
    let (x, y, z) = tup;
    println!("The value of y is: {y}");
}
```

- Above we are *destructuring* the tuple `tup` into three separate variables. You can also access a value by using a period followed by the index of the value we want to access, so `tup.1` for `6.4` above. The first index is 0. A tuple with no values is called *unit* and written `()`, representing an empty value or an empty return type. Expressions implicitly return the unit value if they don't return any other value. You can modify elements of a mutable tuple like `tup.1 = 6.5`.

- The *array* is a fixed length collection of multiple values that must have the same type. We write the values in an array as a comma-separated list inside square brackets:

```
fn main() {
    let a = [1, 2, 3, 4, 5];
}
```

- Arrays are useful when you want your data allocated on the stack instead of the heap or when you want to ensure a fixed number of elements, but an array isn't as flexible as a vector type which can grow or shrink in size because its contents live on the heap. If you're unsure whether to use an array or vector, you probably need a vector. If you know you always need twelve months, an array is a good candidate. You write an array's type using square brackets with the type of each element, a semicolon, and then the number of elements in the array like `let a: [i32; 5] = [1, 2, 3, 4, 5];`. You can also initialize an array to contain the same value by specifying the initial value, a semicolon, then the length of the array like `let a = [3; 5];` instead of `let a = [3, 3, 3, 3, 3];`. You access array values with indexing like `a[1]`. Array indices start at 0. Attempting to access past the end of the array will cause a runtime error (a panic).

3.3 - Functions

- The `fn` keyword allows you to declare functions. Rust code uses *snake case* for function and variable names. Rust doesn't care where you define your functions, only that they are defined somewhere in a scope that can be seen by the caller.

- We can define functions to have *parameters*. The values you provide it are called *arguments*. In function signatures you must declare the type of each parameter. This is a deliberate decision for the language, and in practice the compiler almost never needs type annotations elsewhere because they are in the function definitions. It can also give more helpful error messages. You separate multiple parameters by commas:

```
fn main() {
    print_labeled_measurement(5, 'h');
}

fn print_labeled_measurement(value: i32, unit_label: char) {
    println!("The measurement is: {value}{unit_label}");
}
```

- Function bodies are made up of a series of statements optionally ending in an expression. *Statements* are instructions that perform some action and do not return a value. *Expressions* evaluate to a resultant value. A `let` or function declaration with `fn` is a statement. Therefore, you can't assign either to another variable. `let x = (let y = 6);` will cause an `error: expected expression, found `let` statement`. This is different from C and Ruby where assignment returns the value of the assignment. Expressions evaluate to a value and make up most of the rest of the code that you'll write in Rust. `5 + 6` is an expression that evaluates to `11`. Expressions can be part of statements, like the `6` in `let x = 6`. Function calls are also expressions, as are macro calls. A new scope block created with curly brackets is an expression.

```
fn main() {
    let y = {
        let x = 3;
        x + 1
    };
    println!("The value of y is: {y}");  // The value of y is: 4
}
```

- The `x + 1` at the end doesn't have a semicolon because expressions do not include ending semicolons. If you add one, it turns it into a statement, and it will then not return a value.

- Functions can return values. We do not name return values, but we must declare their type after an arrow (`->`). In Rust, the return value of a function is synonymous with the value of the final expression in the block of the body of a function. You can return early by using the `return` keyword and specifying a value, but most functions return the last expression implicitly.

```rust
fn main() {
    let x = plus_one(5);
    println!("The value of x is: {x}");
}

fn plus_one(x: i32) -> i32 {
    x + 1
}
```

- If you had put a semicolon after `x + 1`, you would have gotten an `error[E0308]: mismatched types` error, because `()` isn't an `i32`, and statements evaluate to `()`.

3.4 - Comments

- Comments start with `//` and continue to the end of the line, or you can use the multiline comment syntax with `/*` and `*/`. Comments can be placed at the end of lines of code, but you'll most commonly see them on the line above code they are annotating. Rust also has documentation comments which we will discuss in Chapter 14.

3.5 - Control Flow

- An `if` expression allows you to branch your code depending on conditions. Blocks of code associated with the conditions in `if` expressions are sometimes called *arms*, just like the arms in `match` expressions. The condition must be a `bool` or you will get an `error[E0308]: mismatched types`. The `else` clause is optional. You can use multiple conditions by combining `if` and `else` in an `else if` expression. Rust only executes the block for the first `true` condition it finds and doesn't check the rest. Using too many `else if` expressions can clutter your code, so prefer `match`.

```rust
fn main() {
    let number = 7;

    if number > 5 {
        println!("condition was true");
    } else if number == 0 {
        println!("number is zero");
    } else {
        println!("condition was false");
    } // condition was true
}
```

- Because `if` is an expression, you can use it in a `let` statement like:

```
fn main() {
    let condition = true;
    let number = if condition { 5 } else { 6 };

    println!("The value of number is: {number}"); // The value of number is:
5
}
```

- The values that have the potential to be results from each arm of the `if` must be the same type or you will get an `error[E0308]: `if` and `else` have incompatible types` error.

- Rust has three kinds of loops: `loop`, `while`, and `for`. The `loop` keyword tells Rust to execute a block of code over and over either forever or until you explicitly tell it to stop. A `break` keyword says to stop executing the loop, and a `continue` keyword skips over the remaining code in this iteration and starts the next iteration. You can provide a value to break to use as the return value of the loop.

```
fn main() {
    let mut counter = 0;
    let result = loop {
        counter += 1;
        if counter == 10 {
            break counter * 2;
        }
    };
    println!("The result is {result}"); // The result is 20
}
```

- Notice above how you end the `let` that assigns the value returned from the loop to `result` with a semicolon, because it's a statement. You can also `return` from a loop, but `break` exits the current loop while `return` always exits the current function.

- If you have loops within loops, `break` and `continue` apply to the innermost loop at that point. You can optionally specify a *loop label*. Loop labels begin with a single quote:

```rust
fn main() {
    let mut count = 0;
    'counting_up: loop {
        println!("count = {count}");
        let mut remaining = 10;
        loop {
            println!("remaining = {remaining}");
            if remaining == 9 {
                break;
            }
            if count == 2 {
                break 'counting_up;
            }
            remaining -= 1;
        }
        count += 1;
    }
    println!("End count = {count}"); // End count = 2
}
```

- The `while` loop simply loops while a condition is `true`. It is more convenient than combining `loop`, `if`, `else`, and `break` to do the same thing.

```rust
fn main() {
    let mut number = 3;
    while number != 0 {
        println!("{number}!");
        number -= 1;
    }
    println!("LIFTOFF!!!");
}
```

- A `for` loop executes some code for each item in a collection. The safety and conciseness of `for` loops makes them the most commonly used loop construct in Rust. Even if you want to count 3, 2, 1 like above, a Rustacean would often use `for` with a Range.

```rust
fn main() {
    let a = [10, 20, 30, 40, 50];
    for element in a {
        println!("the value is: {element}");
    }
}

fn main() {
    for number in (1..4).rev() {
        println!("{number}!");
    }
    println!("LIFTOFF!!!");
}
```

Chapter 4 - Understanding Ownership

- Here is a link to this chapter if you want to re-read it with diagrams: https://rust-book.cs.brown.edu/ch04-00-understanding-ownership.html.

4.1 - What is Ownership?

- Ownership is what enables Rust to make memory safety guarantees without needing a garbage collector. Ownership is a discipline for ensuring the **safety** of Rust programs. Safety is the absence of undefined behavior. In a dynamic language, the interpreter must check every time you use a variable to make sure it is defined and otherwise throw a `NameError` or `ReferenceError`. Rust's goal is to compile programs into efficient binaries that require as few runtime checks as possible. Therefore Rust does not check at runtime whether a variable is defined before being used. Instead, Rust checks at compile-time. If you try to compile a program that uses a variable before it is defined, you will receive an `error[E0425]: cannot find value `x` in this scope`. Here is a program with undefined behavior that gives that error on compilation:

```rust
// DOES NOT COMPILE! error[E0425]: cannot find value `x` in this scope
fn read(y: bool) {
    if y {
        println!("y is true!");
    }
}

fn main() {
    read(x); // oh no! x isn't defined!
    let x = true;
}

// here it is in assembly
// you don't know assembly, but read operates on the value on edi, and we
// do not know what is there initially. so read's behavior is undefined!
main:
    ; ...
    call    read
    mov     edi, 1      ; mov is after call
    ; ...
```

- A foundational goal of Rust is to ensure that your programs never have undefined behavior. That is the meaning of "safety." About 70% of reported security vulnerabilities in low-level systems are caused by memory corruption, which is one form of undefined behavior. A secondary goal of Rust is to prevent undefined behavior at compile-time instead of run-time. This goal has two motivations:

    1. Catching bugs at compile-time means avoiding those bugs in production, improving the reliability of your software.
    2. Catching bugs at compile-time means fewer runtime checks for those bugs, improving the performance of your software.
- Rust provides a particular way to think about memory. Ownership is a discipline for safely using memory within that way of thinking.

- Variables live in the stack.

```
fn main() {
    let n = 5; // L1
    let y = plus_one(n); // L3
    println!("The value of y is: {y}");
}

fn plus_one(x: i32) -> i32 {
    // L2
    x + 1
}

L1              L2              L3
Stack           Stack           Stack
main            main            main
  n | 5           n | 5           n | 5
                plus_one          y | 6
                  x | 5
```

- Variables live in **frames**. A frame is a mapping from variables to values within a single scope, such as a function. For example:
  - The frame for `main` at location `L1` holds `n = 5`.
  - The frame for `plus_one` at location `L2` holds `x = 5`.
  - The frame for `main` at location `L3` holds `n = 5; y = 6`.
- Frames are organized into a **stack** of currently-called-functions. For example, at `L2` the frame for `main` sits above the frame for the called function `plus_one`. After a function returns, Rust deallocates the function's frame. (Deallocation is called **freeing** or **dropping**). The sequence of frames is called a stack because the most recent frame added is always the next frame freed. Note that this doesn't fully capture what happens. As we saw in assembly, Rust might put `n` or `x` into a register rather than a stack frame. But this is an implementation detail that doesn't matter for our discussion, so we can focus on frame-only variables.
- When an expression reads a variable, the variable's value is copied from its slot in the stack frame:

```
let a = 5; // L1
let mut b = a; // L2
b += 1; // L3

L1              L2              L3
Stack           Stack           Stack
main            main            main
  a | 5           a | 5           a | 5
                  b | 5           b | 6
```

- Boxes live in the heap. If we copied a one million element array from `a` to `b`, we would end up with two million elements in memory. To transfer data without copying it, Rust uses **pointers**. A pointer is a value that describes a location in memory. The value that a pointer points to is called its **pointee**. One common way to make a pointer is to allocate memory in the **heap**. The heap is a separate region of memory where data can live indefinitely. Heap data is not tied to a specific stack frame. Rust provides a construction called `Box` for putting data on the heap.

```
let a = Box::new([0; 1_000_000]); // L1
let b = a; // L2

// moving a to b transfers ownership; a is grayed out and can't be used
L1                              L2
Stack       Heap                Stack       Heap
main                            main
  a | *---> [0, 0, ..., 0]        a | x moved
                                  b | *---> [0, 0, ..., 0]
```

- Observe that now, there is only ever a single array at a time. At `L1`, the value of `a` is a pointer to the array inside the heap. The statement `let b = a` copies the pointer from `a` to `b`, but the pointed-to data is not copied. Note that `a` is grayed out because it has been *moved*.

- Rust does not permit manual memory management. Rust automatically allocates and deallocates stack frames. Heap data is allocated when calling `Box::new`. There is no `free` command in Rust. Rust automatically frees a box's heap memory. It is almost correct to say "If a variable is bound to a box, when Rust deallocates the variable's frame, then Rust deallocates the box's heap memory."

```
fn main() {
    let a_num = 4; // L1
    make_and_drop(); // L3
}

fn make_and_drop() {
    let a_box = Box::new(5); // L2
}

L1              L2                          L3
Stack           Stack          Heap         Stack
main            main                          main
  a_num | 4       a_num | 4                     a_num | 4
                  make_and_drop
                    a_box | *---> 5
```

- Now we get back to the idea of *moving*. If we bound an array of a million elements to both `a` and `b`, Rust would try to free it when each went out of scope. But freeing freed memory is undefined behavior! Enter *ownership*! When `a` is bound to `Box::new([0; 1_000_000])`, we say that `a` **owns** the box. The statement `let b = a` **moves** ownership of the box from `a` to `b`.
- **Box deallocation principle** (fully correct): If a variable owns a box, when Rust deallocates the variable's frame, then Rust deallocates the box's heap memory.
- Boxes are used by Rust data structures like `Vec`, `String`, and `HashMap` to hold a variable number of elements.

```
fn main() {
    let first = String::from("Ferris"); // L1
    let full = add_suffix(first); // L4
    println!("{full}");
}

fn add_suffix(mut name: String) -> String {
    // L2
    name.push_str(" Jr."); // L3
    name
}

L1                                  L2
Stack           Heap                Stack           Heap
main                                main
  first | *---> "Ferris"              first | x moved
                                    add_suffix
                                      name  | *---> "Ferris"

L3                                  L4
Stack           Heap                Stack           Heap
main                                main
  first | x moved                     first | x moved
add_suffix                          full  | *---> "Ferris Jr."
  name  | *---> "Ferris Jr."
```

- There are several points to make about this example.
  - At `L1`, the string "Ferris" has been allocated on the heap. It is owned by `first`.
  - Ownership only moves forwards, from `first` to `name` then to `full`.
  - `L2`: calling `add_suffix(first)` moves the `String` into `name`. Only the pointer is copied. The heap data stays where it is.
  - `L3`: the method call `name.push_str(" Jr.")` resizes the string's heap allocation. This does three things. First, it creates a new larger allocation. Second, it writes "Ferris Jr." into the new allocation. Third, it frees the original heap memory. `first` now points to deallocated memory. Since `name` owns the string, it can change it with `push_str`. The function signature has to say `mut name` for that to be allowed.
  - `L4`: the frame for `add_suffix` is gone. This function returned `name`, transferring ownership of the string to `full`. The heap string isn't freed, because it now has a new owner.

- A variable cannot be used after being moved. There is no problem with `first` pointing at deallocated memory, as long as we don't try to use `first` after it has become invalid.

```
// DOES NOT COMPILE! error[E0382]: borrow of moved value: `first`
fn main() {
    let first = String::from("Ferris");
    let full = add_suffix(first);
    println!("{full}, originally {first}"); // L1
}

fn add_suffix(mut name: String) -> String {
    name.push_str(" Jr.");
    name
}

L1
Stack           Heap
main
  first | *--> [freed]
  full  | *--> "Ferris Jr."
```

- **Moved heap data principle**: if a variable `x` moves ownership of heap data to another variable `y`, then `x` cannot be used after the move.
- Moving ownership of heap data avoids undefined behavior from reading deallocated memory.
- You can use `.clone()` to make a deep copy of the heap data, then move the clone instead of the original, so the original stays usable. The obvious drawback is that you have two copies in memory.
- Ownership is primarily a discipline of heap management.
    - All heap data must be owned by exactly one variable.
    - Rust deallocates heap data once its owner goes out of scope.
    - Ownership can be transferred by moves, which happen on assignments and function calls.
    - Heap data can only be accessed through its current owner, not a previous owner.

4.2 - References and Borrowing

- Ownership, boxes, and moves provide a foundation for safely programming with the heap. However, move-only APIs can be inconvenient to use.

```
// DOES NOT COMPILE! error[E0382]: borrow of moved value: `m1` (and `m2`)
fn main() {
    let m1 = String::from("Hello");
    let m2 = String::from("world");
    greet(m1, m2); // L2
    let s = format!("{} {}", m1, m2); // L3
}

fn greet(g1: String, g2: String) {
    println!("{} {}!", g1, g2); // L1
}

L1                  L2                    L3
Stack        Heap   Stack        Heap     Stack        Heap
main                main                  main
  m1 | x moved        m1 | *--> [freed]     m1 | *--> [freed] <-- UB
  m2 | x moved        m2 | *--> [freed]     m2 | *--> [freed] <-- UB
greet
  g1 | *--> Hello
  g2 | *--> world

UB means Undefined Behavior, which would happen here because a pointer would
be used after its pointee is freed.
```

- A **reference** is a kind of pointer.

```
fn main() {
    let m1 = String::from("Hello");
    let m2 = String::from("world"); // L1
    greet(&m1, &m2); // L3 - note the ampersands
    let s = format!("{} {}", m1, m2);
}

fn greet(g1: &String, g2: &String) { // note the ampersands
    // L2
    println!("{} {}!", g1, g2);
}
```

```
L1                          L2
Stack        Heap           Stack        Heap
main                        main
  m1 | *---> "Hello"          m1 | *---> "Hello"
  m2 | *---> "world"          m2 | *---> "world"
                            greet
                              g1 | *-> m1
                              g2 | *-> m2

L3
Stack        Heap
main
  m1 | *---> "Hello"
  m2 | *---> "world"
```

- The expression `&m1` uses the ampersand operator to create a reference to (or "borrow") `m1`. The type of the `greet` parameter `g1` is changed to `&String`, meaning "a reference to a `String`". Observe at `L2` that there are two steps from `g1` to "Hello". `g1` is a reference that points to `m1` on the stack, and `m1` is a String containing a box that points to "Hello" on the heap. `m1` continues to own the heap data "Hello". `g1` does not own either `m1` or "Hello". Therefore after `greet` ends and the program reaches `L3`, no heap data has been deallocated. Only the stack frame for `greet` disappears. References are **non-owning pointers**, because they do not own the data they point to.

- There is a dereference operator in Rust (`*`), but you probably won't see it very often. Rust implicitly inserts dereferences and references in certain cases, such as calling a method with the dot operator or calling `println!`. Arithmetic like `r + 1` also works on references to numbers. You do need `*` to assign through a reference (`*n += 1`), or to use the value itself in a comparison (`if *r > 3`) or a typed assignment (`let y: i32 = *r;`), but the compiler will suggest it if you forget.

- The macro `vec!` creates a vector with the elements between the brackets when you write `let mut v: Vec<i32> = vec![1, 2, 3];`.
- Pointer Safety Principle: data should never be aliased and mutated at the same time.
- Rust enforces the principle for boxes by disallowing aliasing. Assigning a box from one variable to another will move ownership, invalidating the previous variable. Owned data can only be accessed through the owner - no aliases. By design, references are meant to temporarily create aliases. We will explain the basics of how Rust ensures the safety of references through the **borrow checker**.
- The core idea behind the borrow checker is that variables have three kinds of **permissions** on their data:
    - Read (R): data can be copied to another location.
    - Write (W): data can be mutated.
    - Own (O): data can be moved or dropped.
- These permissions don't exist at runtime, only within the compiler. They describe how the compiler "thinks" about your program before the program is executed. By default, a variable has read/own permissions (RO) on its data. If a variable is annotated with `mut`, then it also has write permission (W). The key idea is that references can temporarily remove these permissions.
- Permission diagram legend (Brown's symbol = my notation):
    - `↑` = `[init]`: gained permissions on initialization.
    - `→` = `[borrow]`: lost permissions because it was borrowed.
    - `↺` = `[regain]`: permissions returned when a borrow ends.
    - `↓` = `[end]`: lost permissions because it isn't used again.
    - Inline dots on an expression = `[needs]`: permissions the line requires.
    - `+R` gained, `R` kept, `–R` lost (Brown strikes through), `.` never had (Brown's dash).
    - `<– ERROR` marks a `[needs]` the place doesn't have.
    - `F` (flow, Section 4.2) is a fourth column after O. It doesn't change within a function, so it only appears in `[needs]` rows.

```
let mut v: Vec<i32> = vec![1, 2, 3];    // v    [init]    +R +W +O
let num: &i32 = &v[2];                  // v    [borrow]   R -W -O
                                        // num  [init]    +R  . +O
                                        // *num [init]    +R  .  .
println!("Third element is {}", *num);  // v    [regain]   R +W +O
                                        // num  [end]     -R  . -O
                                        // *num [end]     -R  .  .
v.push(4);                              // v    [end]     -R -W -O
```

- In plain English:

1. After `let mut v = (...)`, the variable `v` has been initialized (indicated by [init]). It gains +R +W +O permissions (the plus sign indicates gain).
2. After `let num = &v[2]`, the data in `v` has been **borrowed** by `num` (indicated by [borrow]). Three things happen:
   - The borrow removes W O permissions from `v` (shown as -W -O for loss). `v` cannot be written, moved or dropped, but it can still be read.
   - The variable `num` has gained +R +O permissions. `num` is not writable (the missing W permission is shown as a .) because it was not marked `let mut`.
   - The **place** `*num` has gained the +R permission.
3. After `println!(...)`, then `num` is no longer in use, so `v` is no longer borrowed. Therefore:
   - `v` regains its +W +O permissions (indicated by [regain]).
   - `num` and `*num` have lost all of their permissions (indicated by [end]).
4. After `v.push(4)`, then `v` is no longer in use, and it loses all of its permissions.

- Other details: borrowing part of `v` (one element) borrows all of `v`. If `v` were allowed to push while the `num` reference was live, it might have to reallocate a new array on the heap and `num` would be left dangling. The borrow ends when the reference is no longer being used; it doesn't have to go out of scope. Borrowing `v` temporarily takes away `v`'s ability to change, move or drop its data, so that `num` can't end up pointing at freed memory.
- Also, why do we see both `num` and `*num`? Because accessing data through a reference is not the same as manipulating the reference itself. For example, if `num` is declared with `let mut num: &i32 = &x;`, then `num` can be reassigned to a different reference (`num` has W), but the data it points to cannot be mutated (`*num` does not have W). In general, permissions are defined on **places** not just variables. A place is anything you can put on the left-hand side of an assignment including:

        - Variables like `a`.
        - Dereferences of places like `*a`.
        - Array accesses of places like `a[0]`.
        - Fields of places like `a.0` for tuples or `a.field` for structs.
        - Any combination of the above like `*((*a)[0].1)`.
    - Places lose their permissions when they go unused because some permissions are mutually exclusive. If you write `num = &v[2]`, then `v` cannot be mutated or dropped while `num` is in use. But we could still use `num` later in the above code; it just means it doesn't lose its permissions until later. It has to stop being used before we `push` though, because we don't have W permission until its permissions are ended. Since 2018, Rust has ended a borrow at the reference's last use. Before that, a borrow lasted until the end of scope. This is called "non-lexical lifetimes" (NLL).
- Put simply, creating a reference to data ("borrowing" it) causes that data to be temporarily read-only until the reference is no longer in use. Rust uses these permissions in its **borrow checker**. The borrow checker looks for potentially unsafe operations involving references.

```
// DOES NOT COMPILE! error[E0502]: cannot borrow `v` as mutable because it
//                                 is also borrowed as immutable
let mut v: Vec<i32> = vec![1, 2, 3];    // v    [init]   +R +W +O
let num: &i32 = &v[2];                  // v    [needs]   R
                                        // v    [borrow]  R -W -O
                                        // num  [init]   +R  . +O
                                        // *num [init]   +R  .  .
v.push(4);                              // v    [needs]   R  W  <- ERROR
println!("Third element is {}", *num);
```

- The error message explains that `v` cannot be mutated while the reference `num` is in use. That's the surface-level reason - the underlying issue is that `num` could be invalidated by `push` and Rust catches that potential violation of memory safety.

- Mutable references provide unique and non-owning access to data. So far we have seen read-only **immutable references** (also called **shared references**). Immutable references permit aliasing but disallow mutation. However, it is also useful to temporarily provide mutable access to data without moving it. The mechanism for this is **mutable references** (also called **unique references**). Here is a simple example:

```
let mut v: Vec<i32> = vec![1, 2, 3];    // v    [init]   +R +W +O
let num: &mut i32 = &mut v[2];          // v    [needs]   R  W
                                        // v    [borrow] -R -W -O
                                        // num  [init]   +R  . +O
                                        // *num [init]   +R +W  .
*num += 1;                              // *num [needs]   R  W
println!("Third element is {}", *num);  // *num [needs]   R
                                        // v    [regain] +R +W +O
                                        // num  [end]    -R  . -O
                                        // *num [end]    -R -W  .
println!("Vector is now {:?}", v);      // v    [needs]   R
                                        // v    [end]    -R -W -O
```

- A mutable reference is created with the `&mut` operator. The type of `num` is written as `&mut i32`. There are two differences from immutable references:
    1. When `num` was an immutable reference, `v` still had the R permission. Now that `num` is a mutable reference, `v` has lost *all* permissions while `num` is in use.
    2. When `num` was an immutable reference, the place `*num` only had the R permission. Now that `num` is a mutable reference, `*num` has also gained the W permission.
- The first observation is what makes mutable references *safe*. Mutable references allow mutation but prevent aliasing. The borrowed place `v` becomes temporarily unusable, so effectively not an alias.
- The second observation is what makes mutable references *useful*. `v[2]` can be mutated through `*num`. For example `*num += 1` mutates `v[2]`.
- Mutable references can also be temporarily downgraded to read-only references like: `let num2: &i32 = &*num;` where `*num` loses W and `*num2` gains R.
- Don't confuse a mutable variable holding a reference with a mutable reference. `let mut` on the variable controls whether `num` can be reassigned to point somewhere else (W on `num`). `&mut` in the type controls whether the data it points to can be changed (W on `*num`). They're independent:
    - `let mut num: &i32` can point elsewhere, but can't change the data.
    - `let num: &mut i32` can change the data, but can't point elsewhere.
- The rule behind that: `let mut` lets you change what the variable *directly holds*. The two-`mut` split only exists for references, because a reference variable has two places (`r` and

`*r`). A variable that owns its value (a `Vec`, `String`, `i32`) has just one place, so a single `let mut` covers both modifying the value and reassigning the variable:

```
let mut v = vec![1, 2, 3];  // v owns the Vec: one place, one `mut`
v.push(4);                  // modify the Vec
v = vec![9];                // or replace it; same `mut`

let r = &mut v;             // r holds a reference: two places
r.push(5);                  // fine without `let mut r`; changes *r, not r
```

- A reference changes permissions while it is "in use". "in use" here refers to a reference's **lifetime**, or the range of code spanning from its birth (where the reference is created) to its death (the last time the reference is used).

```
fn ascii_capitalize(v: &mut Vec<char>) {          // v  [init]   +R  . +O
                                                  // *v [init]   +R +W  .
    let c = &v[0];                                // v  [needs]   R
                                                  // v  [borrow]  R  . -O
                                                  // *v [borrow]  R -W  .
                                                  // c  [init]   +R  . +O
    if c.is_ascii_lowercase() {                   // c  [needs]   R
        let up = c.to_ascii_uppercase();          // c  [needs]   R
                                                  // c  [end]    -R  . -O
                                                  // v  [regain]  R  . +O
                                                  // *v [regain]  R +W  .
                                                  // up [init]   +R  . +O
        v[0] = up;                                // up [needs]   R
                                                  // *v [needs]   R  W
    } else {                                      // c  [end]    -R  . -O
                                                  // v  [regain]  R  . +O
                                                  // *v [regain]  R +W  .
        println!("Already capitalized: {:?}", v);  // v  [needs]   R
    }
}
```

- `v[0] = up` automatically dereferences `v`; it means `(*v)[0] = up`.
- Data must outlive all of its references. Rust enforces this property in two ways. The first way deals with references that are created and dropped within the scope of a single function. `drop` requires O permission, and making a reference to a variable removes its O permission. When references are either input to a function or output from a function, Rust

doesn't know how long they live. For that we will have a new permission F, the flow permission. The following is a safe function that returns a reference to the first element of a vector:

```
fn first(strings: &Vec<String>) -> &String { // strings  [init]   +R  . +O
                                             // *strings [init]   +R  .  .
    let s_ref = &strings[0];                 // strings  [needs]   R        F
                                             // *strings [needs]   R
                                             // strings  [borrow]  R  . -O
                                             // s_ref    [init]   +R  . +O
                                             // *s_ref   [init]   +R  .  .
    s_ref                                    // s_ref    [needs]   R        F
}
```

- The flow permission is expected whenever an expression uses an input reference (like `&strings[0]`), or returns an output reference (like `return s_ref`). Unlike RWO permissions, F does not change throughout the body of a function. A reference has the F permission if it's allowed to be used (that is to *flow*) in a particular expression. The flow permission is decided by the function signature. `first` only has one input reference, so Rust assumes the output comes from it (called *lifetime elision*). That parameter has F, and so does `s_ref`, because it points into the data the parameter borrowed. If we change `first` to `first_or` which has a `default` parameter, the following won't compile:

```
// DOES NOT COMPILE! error[E0106]: missing lifetime specifier
fn first_or(strings: &Vec<String>, default: &String) -> &String {
    if strings.len() > 0 {
        &strings[0]     // strings  [needs]   R        F <- ERROR
    } else {
        default         // default  [needs]   R        F <- ERROR
    }
}
```

- The help message from the compiler helps you understand the above error: `this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from `strings` or `default``. To understand why, imagine the following:

```
// DOES NOT COMPILE! error[E0106]: missing lifetime specifier
fn main() {
    let strings = vec![];
    let default = String::from("default");
    let s = first_or(&strings, &default);
    drop(default);
    println!("{}", s);
}
```

- The program is unsafe if `first_or` allows `default` to *flow* into the return value. Rust would only allow this program to compile if it was certain that `default` cannot flow into the return value. Rust provides a mechanism called *lifetime parameters* which will be explained in Chapter 10, but for now it is enough to know that: input/output references are treated differently than references within a function body and Rust uses a different mechanism, the F permission, to check the safety of those references.
- To see F in another context, consider this:

```
// DOES NOT COMPILE! error[E0106]: missing lifetime specifier
fn return_a_string() -> &String {
    let s = String::from("Hello world");
    let s_ref = &s;
    s_ref
}
```

- Clearly `s_ref` was invalidated when `return_a_string` returns because `s` gets popped from the stack and its memory deallocated. There are no input references, so nothing in the function has F, and a reference into a local like `s_ref` can never flow out.

4.3 - Fixing Ownership Errors

- Rust will always reject an unsafe program. But Rust will also sometimes reject a safe program. We need to learn how to respond to both.

Fixing an Unsafe Program: Returning a Reference to the Stack

```
// DOES NOT COMPILE! error[E0106]: missing lifetime specifier
fn return_a_string() -> &String {
    let s = String::from("Hello world");
    &s
}
```

- This is clearly unsafe. We make a reference to a variable that is popped off the stack when we return. The simplest fix is to move ownership out of the function, changing `&String` to `String`:

```
fn return_a_string() -> String {
    let s = String::from("Hello world");
    s
}
```

- Or we could return a string literal, which lives forever (indicated by `'static`). This works if we never intend to change the string and avoids heap allocation:

```
fn return_a_string() -> &'static str {
    "Hello world"
}
```

- We also have the option to defer borrow checking to run time by using garbage collection. Here is a reference-counted pointer, which will be explained more in Chapter 15:

```
use std::rc::Rc;
fn return_a_string() -> Rc<String> {
    let s = Rc::new(String::from("Hello world"));
    Rc::clone(&s)
}
```

- Put simply, `Rc::clone` only clones a pointer to `s` and not the data itself. At runtime, the `Rc` checks when the last `Rc` pointing to the data has been dropped and then deallocates the data. We could also have the caller provide a "slot" to put the string using a mutable reference:

```
fn return_a_string(output: &mut String) {
    output.replace_range(.., "Hello world");
}
```

- This style can be verbose, but it can also be more memory-efficient if the caller needs to carefully control when allocations occur.

Fixing an Unsafe Program: Not Enough Permissions

- Another common issue is trying to mutate read-only data, or trying to drop data behind a reference.

```
// DOES NOT COMPILE! error[E0596]: cannot borrow `*name` as mutable, as it
//                                 is behind a `&` reference
fn stringify_name_with_title(name: &Vec<String>) -> String {
                                    // name  [init]  +R  . +O
                                    // *name [init]  +R  .  .
    name.push(String::from("Esq."));  // *name [needs]  R  W  <- ERROR
    let full = name.join(" ");
    full
}
// ideally: ["Ferris", "Jr."] => "Ferris Jr. Esq."
```

- This program is unsafe because `push` could invalidate other references to `name` outside of `stringify_name_with_title`. Consider:

```
fn main() {
    let name = vec![String::from("Ferris")];
    let first = &name[0]; // L1
    stringify_name_with_title(&name); // L2
    println!("{}", first); // L3
}

L1
Stack           Heap
main
  name  | *---> buf1 [*] ---> "Ferris"
  first | *---> buf1[0]

L2 (push moved the slots to buf2 and freed buf1)
Stack           Heap
main
  name  | *---> buf2 [*, *] -> "Ferris", "Esq."
  first | *---> [freed]

L3
  println!("{}", first) reads through first  <-- UB
```

- The easiest solution, but also a bad solution, is to simply pass a `&mut Vec<String>` to `stringify_name_with_title`. However, functions should not mutate their inputs if the caller would not expect it. Another option is to take ownership of the name by changing `&Vec<String>` to `Vec<String>`:

```
fn stringify_name_with_title(mut name: Vec<String>) -> String {
    name.push(String::from("Esq."));
    let full = name.join(" ");
    full
}
```

  This is also a bad solution. It is very rare for Rust functions to take ownership of heap-owning data structures like `Vec` or `String`. This version would make the input `name` unusable, which is annoying to a caller.

- Instead we could continue passing `&Vec` but clone the vector:

```
fn stringify_name_with_title(name: &Vec<String>) -> String {
    let mut name_clone = name.clone();
    name_clone.push(String::from("Esq."));
    let full = name_clone.join(" ");
    full
}
```

- We could avoid unnecessary copies by adding the suffix later, which works because `slice::join` already copies the data from `name` into `full`:

```
fn stringify_name_with_title(name: &Vec<String>) -> String {
    let mut full = name.join(" ");
    full.push_str(" Esq.");
    full
}
```

Fixing an Unsafe Program: Aliasing and Mutating a Data Structure

- Another unsafe operation is using a reference to heap data that gets deallocated by another alias. This function gets a reference to the largest string in a vector and then uses it while mutating the vector:

```
// DOES NOT COMPILE! error[E0502]: cannot borrow `*dst` as mutable because it
//                                 is also borrowed as immutable
fn add_big_strings(dst: &mut Vec<String>, src: &[String]) {
                                                    // *dst [init]    +R +W
,
    let largest: &String =
        dst.iter().max_by_key(|s| s.len()).unwrap();// *dst [needs]    R
                                                    // *dst [borrow]   R -W
,
    for s in src {
        if s.len() > largest.len() {
            dst.push(s.clone());                    // *dst [needs]    R  W
                                                    //      <- ERROR: no W
        }
    }
}
```

- This program is unsafe because `dst.push(...)` could deallocate the contents of `dst` and invalidate the reference `largest`. The most direct fix would be to shorten the lifetime of `largest` to not overlap with `dst.push(...)`. One possibility is to clone `largest`:

```
fn add_big_strings(dst: &mut Vec<String>, src: &[String]) {
    let largest: String = dst.iter().max_by_key(|s|
s.len()).unwrap().clone();
    for s in src {
        if s.len() > largest.len() {
            dst.push(s.clone());
        }
    }
}
```

- This may involve a performance hit for allocating and copying the string data. Another possibility is to perform all the length comparisons first and mutate `dst` afterwards:

```
fn add_big_strings(dst: &mut Vec<String>, src: &[String]) {
    let largest: &String = dst.iter().max_by_key(|s| s.len()).unwrap();
    let to_add: Vec<String> =
        src.iter().filter(|s| s.len() > largest.len()).cloned().collect();
    dst.extend(to_add);
}
```

- However this also causes a performance hit for allocating the vector `to_add`. A final possibility is to copy out the length of `largest`, since we actually only need its length and not its contents. It's arguably the most idiomatic and performant solution:

```
fn add_big_strings(dst: &mut Vec<String>, src: &[String]) {
    let largest_len: usize =
        dst.iter().max_by_key(|s| s.len()).unwrap().len();
    for s in src {
        if s.len() > largest_len {
            dst.push(s.clone());
        }
    }
}
```

- These solutions share the same key idea: shortening the lifetime of borrows on `dst` to not overlap with a mutation to `dst`.

Fixing an Unsafe Program: Copying vs. Moving Out of a Collection

- First we will look at two small programs. The first is safe and the second will not compile.

```
let v: Vec<i32> = vec![0, 1, 2];  // v      [init]   +R  . +O
let n_ref: &i32 = &v[0];          // v      [needs]   R
                                  // v      [borrow]  R  . -O
                                  // n_ref  [init]   +R  . +O
                                  // *n_ref [init]   +R  .  .
let n: i32 = *n_ref;              // *n_ref [needs]   R
                                  // v      [end]    -R  .  .
                                  // n_ref  [end]    -R  . -O
                                  // *n_ref [end]    -R  .  .
                                  // n      [init]   +R  . +O
```

```
// DOES NOT COMPILE! error[E0507]: cannot move out of `*s_ref` which is
behind
//                                a shared reference
let v: Vec<String> =
    vec![String::from("Hello world")];  // v      [init]   +R  . +O
let s_ref: &String = &v[0];             // v      [needs]   R
                                        // v      [borrow]  R  . -O
                                        // s_ref  [init]   +R  . +O
                                        // *s_ref [init]   +R  .  .
let s: String = *s_ref;                 // *s_ref [needs]   R     0  <- ERROR
```

- The difference is that a `String` owns heap data and an `i32` does not. Copying the `String` out of `v` would copy its pointer to the heap data, so Rust would free that memory when `s` goes out of scope, then free the same memory again when `v` goes out of scope (a double free). If a value does not own heap data, then it can be copied without a move: `i32` and `&String` can be copied, but `String` cannot. One exception is mutable references: an `&mut i32` does not own heap data, but it cannot be copied. After `let b = a;` where `a` is an `&mut i32`, `a` can no longer be used, because we can't have two usable mutable references to the same data.

- You can fix this by avoiding taking ownership of the string and just using an immutable reference:

```rust
let v: Vec<String> = vec![String::from("Hello world")];
let s_ref: &String = &v[0];
println!("{s_ref}!");
```

- You can also clone the data you want to get ownership of and leave the vector alone:

```rust
let v: Vec<String> = vec![String::from("Hello world")];
let mut s: String = v[0].clone();
s.push('!');
println!("{s}");
```

- Finally, you can use a method like `Vec::remove` to move the string out of the vector:

```rust
let mut v: Vec<String> = vec![String::from("Hello world")];
let mut s: String = v.remove(0);
s.push('!');
println!("{s}");
assert!(v.len() == 0);
```

Fixing a Safe Program: Mutating Different Tuple Fields

- First let's look at an example of fine-grained permission tracking that works:

```rust
let mut name = (              // name   [init]   +R +W +O
    String::from("Ferris"),   // name.0 [init]   +R +W +O
    String::from("Rustacean") // name.1 [init]   +R +W +O
);
let first = &name.0;          // name   [borrow]  R -W -O
                              // name.0 [borrow]  R -W -O
name.1.push_str(", Esq.");    // name.1 [needs]   R  W
println!("{first} {}", name.1);
```

- If however we define a `get_first` function, Rust might lose track of the fact that `name.1` still has write permission:

```rust
// DOES NOT COMPILE! error[E0502]: cannot borrow `name.1` as mutable because
//                                 it is also borrowed as immutable
fn get_first(name: &(String, String)) -> &String {
    &name.0
}

fn main() {
    let mut name = (
        String::from("Ferris"),
        String::from("Rustacean")
    );
    let first = get_first(&name);   // name   [borrow]  R -W -O
                                    // name.0 [borrow]  R -W -O
                                    // name.1 [borrow]  R -W -O
    name.1.push_str(", Esq.");      // name.1 [needs]   R  W    <- ERROR
    println!("{first} {}", name.1);
}
```

- To fix this, we can either inline `&name.0` or defer borrow checking to runtime with cells, which we will discuss later.

Fixing a Safe Program: Mutating Different Array Elements

- A similar problem occurs when we borrow elements of an array. Borrowing one element works fine:

```rust
let mut a = [0, 1, 2, 3];
let x = &mut a[1];          // a[_] [borrow] -R -W  .
*x += 1;                    // a[_] [regain] +R +W  .
println!("{a:?}");
```

- But borrowing a second element while the first borrow is live fails, even though `a[1]` and `a[2]` don't overlap:

```rust
// DOES NOT COMPILE! error[E0502]: cannot borrow `a[_]` as immutable because
//                                 it is also borrowed as mutable
let mut a = [0, 1, 2, 3];
let x = &mut a[1];          // a[_] [borrow] -R -W  .
let y = &a[2];              // a[_] [needs]   R       <- ERROR
*x += *y;
```

- Rust's borrow checker doesn't keep track of which elements in an array are borrowed. It uses a single place, `a[_]`, to represent all indexes of `a`. It does this because an index could be calculated at runtime and not known at compile time. We can work around the borrow checker with `slice::split_at_mut`:

```
let mut a = [0, 1, 2, 3];
let (a_l, a_r) = a.split_at_mut(2);
let x = &mut a_l[1];
let y = &a_r[0];
*x += *y;
```

- We could also directly use `unsafe`:

```
let mut a = [0, 1, 2, 3];
let x = &mut a[1] as *mut i32;
let y = &a[2] as *const i32;
unsafe { *x += *y; } // DO NOT DO THIS unless you know what you're doing!
```

4.4 - The Slice Type

- A slice is a kind of reference (a non-owning pointer) that refers to a contiguous sequence of elements in a collection rather than a whole collection. To motivate why slices are useful, consider a function that takes a string of words separated by spaces and returns the first word it finds in that string. A function to do this might have the signature: `fn first_word(s: &String) -> ?`. We don't want ownership of the string, and what should we return? We don't have a way to say "part of a string". This function returns the index of the end of the first word:

```
fn first_word(s: &String) -> usize {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return i;
        }
    }

    s.len()
}
```

- However, the number we return isn't connected to the original `String` at all. Someone could `clear` the `String` and we would have a number that means nothing. A *string slice* is a reference to part of a `String` and looks like:

```
let s = String::from("hello world");

let hello: &str = &s[0..5];
let world: &str = &s[6..11];
let s2: &String = &s; // L1

L1
Stack                 Heap
main
  s     | *---------> "hello world"
  hello | *, len 5 --> "hello world"[0..5]
  world | *, len 5 --> "hello world"[6..11]
  s2    | *-> s
```

- We create slices using a range within brackets by specifying `[starting_index..ending_index]` where `starting_index` is the first position in the slice and `ending_index` is one more than the last position in the slice. Slices are "fat" pointers, or pointers with metadata. The metadata is the length of the slice. `hello` points to "h" and has a `len` of 5. As references, slices change the permissions on referenced data.

```
fn main() {
    let mut s = String::from("hello");  // s      [init]   +R +W +O
    let hello: &str = &s[0..5];         // s      [needs]   R
                                        // s      [borrow]  R -W -O
                                        // hello  [init]   +R  . +O
                                        // *hello [init]   +R  .  .
    println!("{hello}");                // hello  [needs]   R
                                        // s      [regain]  R +W +O
                                        // hello  [end]    -R  . -O
                                        // *hello [end]    -R  .  .
    s.push_str(" world");               // s      [needs]   R  W
                                        // s      [end]    -R -W -O
}
```

- In range syntax, you can drop `starting_index` if you want to start at `0`, and drop `ending_index` if you want to end at `len`. `[0..2]` becomes `[..2]` and `[3..len]` becomes `[3..]`. You can drop both values to take a slice of the entire string.

- So now let's rewrite `first_word` with string slices:

```
fn first_word(s: &String) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}
```

- Now when we call `first_word`, we get back a single value that is tied to the underlying data. The compiler ensures the reference into the `String` remains valid. This does not compile:

```
// DOES NOT COMPILE! error[E0502]: cannot borrow `s` as mutable because it is
//                                 also borrowed as immutable
fn main() {
    let mut s = String::from("hello world");
    let word = first_word(&s);                  // s [borrow]  R -W -O
    s.clear();                                  // s [needs]   R  W    <- ERROR
    println!("the first word is: {}", word);
}
```

- Recall from the borrowing rules that if we have an immutable reference to something, we cannot also take a mutable reference. Because `clear` needs to truncate the `String`, it needs to get a mutable reference. The `println!` after the call to `clear` uses the reference in `word`, so the immutable reference must still be active at that point. But Rust disallows the mutable reference in `clear` and the immutable reference in `word` from existing at the same time, and compilation fails.

- String literals are slices. When we type `let s = "Hello, world!";` the type of `s` is `&str`. It's a slice pointing to that specific point of the binary. `&str` is an immutable reference. As another improvement, we can change the signature of `first_word` to `fn first_word(s: &str) -> &str {` and it will be able to work both on a slice of the `String` or a reference to the `String`.

```
fn main() {
    let my_string = String::from("hello world");

    // `first_word` works on slices of `String`s, whether partial or whole.
    let word = first_word(&my_string[0..6]);
    let word = first_word(&my_string[..]);
    // `first_word` also works on references to `String`s, which are
    // equivalent to whole slices of `String`s.
    let word = first_word(&my_string);

    let my_string_literal = "hello world";

    // `first_word` works on slices of string literals, partial or whole
    let word = first_word(&my_string_literal[0..6]);
    let word = first_word(&my_string_literal[..]);

    // Because string literals *are* string slices already,
    // this works too, without the slice syntax!
    let word = first_word(my_string_literal);
}
```

- Slice syntax also works to refer to part of an array:

```
let a = [1, 2, 3, 4, 5];
let slice = &a[1..3];
assert_eq!(slice, &[2, 3]);
```

- Slice syntax also works with other collections. We'll talk about them when we discuss vectors in Chapter 8.
- Remember, a slice is both a pointer and a length, so it takes up slightly more memory than just a pointer (16 bytes vs 8 bytes on 64-bit architecture).

4.5 - Ownership Recap

- Ownership at Runtime
    - Rust allocates local variables in stack frames, which are allocated when a function is called and deallocated when the call ends.
    - Local variables can hold either data (like numbers, booleans, tuples, etc.) or pointers.

- Pointers can be created either through boxes (pointers owning data on the heap) or references (non-owning pointers).

```
fn main() {
    let mut a_num = 0;
    inner(&mut a_num); // L2
}

fn inner(x: &mut i32) {
    let another_num = 1;
    let a_stack_ref = &another_num;

    let a_box = Box::new(2);
    let a_box_stack_ref = &a_box;
    let a_box_heap_ref = &*a_box; // L1

    *x += 5;
}
```

```
L1                                          L2
Stack                          Heap         Stack
main                                        main
  a_num           | 0                          a_num | 5
inner
  x               | *-> a_num
  another_num     | 1
  a_stack_ref     | *-> another_num
  a_box           | *----------> 2
  a_box_stack_ref | *-> a_box
  a_box_heap_ref  | *----------> 2 (the same 2)
```

- Slices are a special kind of reference that refer to a contiguous sequence of data in memory.

```
fn main() {
    let s = String::from("abcdefg");
    let s_slice = &s[2..5]; // L1
}

L1
Stack                Heap
main
  s       | *---------> "abcdefg"
  s_slice | *, len 3 --> "abcdefg"[2..5] ("cde", not a separate copy)
```

- Rust tracks R (read), W (write), and O (own) permissions on each variable. Rust requires that a variable has appropriate permissions to perform a given operation. As a basic example, if a variable isn't declared `let mut`, then it is missing the W permission and cannot be mutated. A variable's permissions can be changed if it is **moved** or **borrowed**. A move of a variable with a non-copyable type like `Box<T>` or `String` requires RO permissions and the move eliminates all permissions on the original variable. Borrowing a variable (creating a reference to it) temporarily removes some of the variable's permissions. An immutable borrow creates an immutable reference and also disables the borrowed data from being mutated or moved. You cannot mutate through the original or the immutable reference and moving data out of the reference (with `let s2 = *s_ref`) is also not ok. A mutable borrow creates a mutable reference which disables the borrowed data from being read, written, or moved. Mutating through the reference is fine, but you cannot even read through the original variable until you are done with the mutable reference.

4.6 - Chapter Summary: Ownership & Borrowing

- **The Three Rules of Ownership**:
    1. Each value in Rust has an owner.
    2. There can only be one owner at a time.
    3. When the owner goes out of scope, the value is dropped.
- **Borrowing Rules (Aliases vs. Mutation)**:
    - You can have any number of immutable references (`&T`) to a resource, OR exactly one mutable reference (`&mut T`), but never both simultaneously.
    - References must always be valid (the borrow checker ensures references never outlive the data they point to).
- **Moves vs. Copies**:

    - `Copy` types (e.g., `i32`, `bool`, `char`, arrays/tuples of `Copy` types) are cheap bitwise stack copies; original remains valid.
    - Non-`Copy` types (e.g., `String`, `Vec<T>`, `Box<T>`) move ownership on assignment or function call, invalidating the original variable.
- **Permissions Quick Reference (Brown Model)**:

Operation	Required	Effect on Source Variable
Read / copy (`y = x`)	`R`	None (source retains `R`, `W`, `O`)
Move (`y = x`)	`R`, `O`	Loses all permissions (`-R -W -O`)
Borrow immutably (`&x`)	`R`	Loses `W` and `O` until borrow ends
Borrow mutably (`&mut x`)	`R`, `W`	Loses `R`, `W`, `O` until borrow ends

Chapter 5 - Using Structs to Structure Related Data

5.1 - Defining and Instantiating Structs

- Structs are like tuples in that both hold multiple related values, but in a struct you name each piece of data so it's clear what the values mean. You define a struct with the `struct` keyword:

```
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}
```

- To use a struct after we've defined it, we create an instance of that struct by specifying concrete values for each of the fields.

```rust
fn main() {
    let user1 = User {
        email: String::from("someone@example.com"),
        username: String::from("someusername123"),
        active: true,
        sign_in_count: 1,
    }; // L1
}

L1
Stack                              Heap
main
  user1.active         | true
  user1.username       | *--------> "someusername123"
  user1.email          | *--------> "someone@example.com"
  user1.sign_in_count  | 1
```

- To get a specific value from a struct, we use dot notation. If the instance is mutable, we change a field with dot notation and assigning into a particular field. Note that we use `let mut user1` to make the entire instance mutable; Rust doesn't allow us to mark only certain fields as mutable.
- You can build a struct with a function and return the struct if it is the last expression in the function body. This is so common that there is a *field init shorthand* syntax for doing this with less repetition:

```
// `build_user` without special syntax
fn build_user(email: String, username: String) -> User {
    User {
        active: true,
        username: username,
        email: email,
        sign_in_count: 1,
    }
}

// `build_user` with *field init shorthand*
fn build_user(email: String, username: String) -> User {
    User {
        active: true,
        username,
        email,
        sign_in_count: 1,
    }
}
```

- Because the parameters have the same name as the struct fields they are meant for, we only have to specify the parameter: `username,` instead of `username: username,`.
- Struct update syntax provides a convenient way to copy another struct and update certain fields. You use `..struct_name` to set unspecified fields as equal:

```rust
// without struct update syntax
fn main() {
    // --snip--

    let user2 = User {
        active: user1.active,
        username: user1.username,
        email: String::from("another@example.com"),
        sign_in_count: user1.sign_in_count,
    };
}

// with struct update syntax
fn main() {
    // --snip--

    let user2 = User {
        email: String::from("another@example.com"),
        ..user1
    };
}
```

- The `..user1` must come last and specifies all remaining fields should be set equal to `user1`.
- Note that you can specify a struct's fields and values in any order, if you haven't noticed we aren't using the same order as the definition.
- Note also that ownership still applies. `..user1` moves the `String` in `user1.username` into `user2`, so `user1` is partially invalidated: we can no longer use `user1` as a whole or `user1.username`, though its other fields are still usable. If we had given `user2` new values for both `email` and `username`, `user1` would still be fully valid, because `active` and `sign_in_count` implement the `Copy` trait and are copied rather than moved.
- Rust also supports *tuple structs*. They have a name but don't have names associated with their fields; rather, they just have the types of the fields. Tuple structs are useful when you want to give the whole tuple a name and make the tuple a different type from other tuples, and when naming each field as in a regular struct would be verbose or redundant.

```
struct Color(i32, i32, i32);
struct Point(i32, i32, i32);

fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0, 0, 0);
}
```

- Above `black` and `origin` have the same values but are different types because they're instances of different tuple structs. Each struct you define is its own type, even though the fields within the struct might have the same types. A function that takes a parameter of type `Color` cannot take a `Point` as an argument even though both are three `i32` values. Tuple structs still support `.` syntax to access an individual value like `black.0`, and you can still destructure them but need to specify the type of the struct when doing so like `let Point(x, y, z) = origin;`.

- You can also define *unit-like structs* that lack any data fields. They can be useful if you need to implement a trait on some type but don't have any data that you want to store in the type itself:

```
struct AlwaysEqual;

fn main() {
    let subject = AlwaysEqual;
}
```

- We used the owned `String` type rather than `&str` slice type in the `User` struct defined above. This is a deliberate choice because we want each instance of this struct to own all of its data and for that data to be valid for as long as the entire struct is valid. Structs can also store references to data owned by something else, but to do this requires *lifetimes*, a Rust feature we will discuss in Chapter 10.

```rust
// DOES NOT COMPILE! error[E0106]: missing lifetime specifier
struct User {
    active: bool,
    username: &str,
    email: &str,
    sign_in_count: u64,
}

fn main() {
    let user1 = User {
        active: true,
        username: "someusername123",
        email: "someone@example.com",
        sign_in_count: 1,
    };
}
```

- Rust's borrow checker will track ownership permissions both at the struct-level and field-level. If you borrow field `x` of a `Point` structure `p`, it will affect both permissions of `p` and `p.x`, but not `p.y`.

```rust
struct Point { x: i32, y: i32 }
let mut p = Point { x: 0, y: 0 };   // p   [init]   +R +W +O
                                    // p.x [init]   +R +W +O
                                    // p.y [init]   +R +W +O
let x = &mut p.x;                   // p.x [needs]   R  W
                                    // p   [borrow] -R -W -O
                                    // p.x [borrow] -R -W -O
                                    // x   [init]   +R  . +O
                                    // *x  [init]   +R +W  .
*x += 1;                            // *x  [needs]   R  W
                                    // x   [end]    -R  . -O
                                    // *x  [end]    -R -W  .
                                    // p   [regain] +R +W +O
                                    // p.x [regain] +R +W +O
println!("{}, {}", p.x, p.y);       // p.x [needs]   R
                                    // p.y [needs]   R
```

- If we had tried to use `p` while `p.x` was mutably borrowed, we would have gotten `error[E0502]: cannot borrow `p` as immutable because it is also borrowed as mutable`. Generally, if you encounter an ownership error that involves a struct, you should consider which fields of your structure are supposed to be borrowed with which

permissions. But be aware of the borrow checker's limitations, since Rust may sometimes assume more fields are borrowed than they actually are.

5.2 - An Example Program Using Structs

- Consider a simple program to calculate the area of rectangles:

```
// ch05_rectangles/src/main.rs
fn main() {
    let width1 = 30;
    let height1 = 50;

    println!(
        "The area of the rectangle is {} square pixels.",
        area(width1, height1)
    );
}

fn area(width: u32, height: u32) -> u32 {
    width * height
}
```

- While this works, we have a problem that `area` calculates the area of one rectangle but here it takes two parameters and it's not clear they are related. We could make it a little more clear with a tuple:

```
fn main() {
    let rect1 = (30, 50);

    println!(
        "The area of the rectangle is {} square pixels.",
        area(rect1)
    );
}

fn area(dimensions: (u32, u32)) -> u32 {
    dimensions.0 * dimensions.1
}
```

- Now we have our data all in one argument, but it's hard for a newcomer to remember width is the first piece of data and height the second without labels. This doesn't matter for our

area calculation, but if we were trying to draw the rectangle, it could prove disastrous.

```
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!(
        "The area of the rectangle is {} square pixels.",
        area(&rect1)
    );
}

fn area(rectangle: &Rectangle) -> u32 {
    rectangle.width * rectangle.height
}
```

- Now `area` takes one parameter and both pieces of data are labeled. Note that `rectangle` parameter is an immutable borrow of a struct `Rectangle` instance. This way, `main` retains ownership and can continue using `rect1`. Note that accessing the fields of a borrowed struct instance does not move the field values, which is why you often see borrows of structs.

- Imagine if to debug we wanted to print our rectangle above, like `println!("rect1 is {rect1}");`. This would fail to compile with an `error[E0277]: `Rectangle` doesn't implement `std::fmt::Display`` . `println!` macro supports multiple types of formatting, but by default the curly brackets tell it to use a formatting known as `Display`, output intended for direct end user consumption. Instead we can try to print our rectangle using `Debug` which is meant for developers. To do this, we replace `{rect1}` with `{rect1:?}` like `println!("rect1 is {rect1:?}");` and this changes our compile error to `error[E0277]: `Rectangle` doesn't implement `Debug`` . There is a default implementation of `Debug` however and we can implement it by adding `#[derive(Debug)]` above the definition of our `Rectangle` struct. This will give us output `rect1 is Rectangle { width: 30, height: 50 }`.

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!("rect1 is {rect1:?}");
}
```

- To display it split across multiple lines, use `{:#?}` instead of `{:?}`. There is also a `dbg!` macro that takes ownership of an expression, prints the file and line number of where the macro occurs, and prints the resultant value of that expression, then returns ownership of the value. It prints to `stderr`.

```
// ch05_rectangles/src/main.rs
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let scale = 2;
    let rect1 = Rectangle {
        width: dbg!(30 * scale),
        height: 50,
    };

    dbg!(&rect1);
}
```

```
$ cargo run
   Compiling rectangles v0.1.0 (file:///projects/rectangles)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.61s
     Running `target/debug/rectangles`
[src/main.rs:10:16] 30 * scale = 60
[src/main.rs:14:5] &rect1 = Rectangle {
    width: 60,
    height: 50,
}
```

- The `width` field gets the same value because `dbg!` returns ownership. However we don't want `dbg!` to take ownership of `rect1`, so we use a reference to `rect1` in the next call.
- It's cool we're printing out our struct, but we forgot all about `area`. Let's fix that...

5.3 - Method Syntax

- Methods are similar to functions but are defined within the context of a struct, enum, or trait and their first parameter is always `self`.

```rust
// ch05_rectangles/src/main.rs
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!(
        "The area of the rectangle is {} square pixels.",
        rect1.area()
    );
}
```

- To define a function within the context of `Rectangle`, we start an `impl` block for `Rectangle`. Everything within this `impl` block will be associated with the `Rectangle` type. `area` is mostly the same as when it was a separate function except for the new `&self` parameter instead of taking an `&Rectangle` and using `self` to access the data elements. You call `area` using the `.` syntax on an instance of the struct type. `&self` is actually shorthand for `self: &Self`. Within an `impl` block, `Self` is an alias for the type that the `impl` block is for (here `Rectangle`). Methods can take ownership of `self`, borrow `self` immutably (like we did here), or borrow `self` mutably just as they can any other parameter. It is very rare to take ownership of `self`, and we didn't need `&mut self` here to read values. Methods can have the same name as data fields, and Rust knows which you are accessing by whether or not you use parentheses. Often when we do this, we want the method to return the value of the field and do nothing else; *getters*. This is useful to designate a field as private but give it a public getter so that it is read-only, and we will cover public/private in Chapter 7.

- Methods are a type of *associated function*. Everything in an `impl` block is an associated function with the type named after the `impl`. But we can define functions without a `self` parameter that are not methods if they do not need an instance of the type to work with. `String::from` is an example of an associated function defined on the `String` type. This functionality is often used for constructors that return a new instance of the struct, and they are often called `new`, but this isn't a special name in Rust. The associated function is namespaced by the struct and you use the `::` syntax to call it. The `::` syntax is also used for namespaces created by modules which we will discuss in Chapter 7. Method calls with the `.` syntax are just syntactic sugar for calling the method with `::` and providing `&r` for an instance `r`.

- It is valid to have multiple `impl` blocks for the same type. We will see a use for this when we discuss generic types and traits.

- Rust inserts as many references and dereferences as needed to make types match up for the `self` parameter. In the following code, we need an immutable borrow plus two dereferences (one for the mutable reference and one for the box):

```
let r = &mut Box::new(Rectangle {
    width: 1,
    height: 2
});
let area1 = r.area();
let area2 = Rectangle::area(&**r);
assert_eq!(area1, area2);
```

- To study methods and ownership, we will use a running example of these three methods that take `&self`, `&mut self`, and `self`.

```
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn set_width(&mut self, width: u32) {
        self.width = width;
    }

    fn max(self, other: Rectangle) -> Rectangle {
        Rectangle {
            width: self.width.max(other.width),
            height: self.height.max(other.height),
        }
    }
}
```

- If we make an owned rectangle with `let rect = Rectangle { ... }`, then `rect` has R and O permissions so we can call `area` and `max` methods but not `set_width` which would require W. Trying the latter would give `error[E0596]: cannot borrow `rect` as mutable, as it is not declared as mutable`. It would be fine to call `set_width` through a mutable reference, `let rect_ref = &mut rect;` (which requires declaring `let mut rect`), but we would get a similar error through an immutable reference, `let rect_ref = ▭`. Calling `max` moves ownership, so we can no longer use our original variable (unless the struct implements `Copy`). Once we call `rect.max(..)`, we move `rect` and so lose all permission on it. Trying to use it afterwards would give `error[E0382]: borrow of moved value: `rect``. We would also get an error, `error[E0507]: cannot move out of `*self` which is behind a mutable reference`, if we wrote a `&mut self` method that did `*self = self.max(other);`. Calling `max` requires ownership, which we do not have through `*self`. This would actually be safe since our `Rectangle` doesn't own any heap data, and you could make it work by adding `#[derive(Copy, Clone)]` to the `struct` definition, so calling `max` copies `*self` instead of moving it. So remember: when you see an error like "cannot move out of `*self`", that's usually because you're trying to call a `self` method on a reference like `&self` or `&mut self` and Rust is protecting you.

```rust
// ch05_rectangles/src/main.rs
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn set_width(&mut self, width: u32) {
        self.width = width;
    }

    fn max(self, other: Rectangle) -> Rectangle {
        Rectangle {
            width: self.width.max(other.width),
            height: self.height.max(other.height),
        }
    }
}

fn main() {
    let scale = 2;
    let rect1 = Rectangle {
        width: dbg!(30 * scale),
        height: 50,
    };

    dbg!(&rect1);

    println!(
        "The area of the rectangle is {} square pixels.",
        rect1.area()
    );
}
```

Chapter 6 - Enums and Pattern Matching

- Enums allow you to define a type by enumerating its possible variants.

6.1 - Defining an Enum

- Enums give you a way of saying a value is one of a possible set of values.
- Consider an IP address. It can either be a version four or a version six address, but not both at the same time. When you define an enum, it becomes a custom data type that can be used anywhere in your code.

```rust
enum IpAddrKind {
    V4,
    V6,
}

// Using the enum
let four = IpAddrKind::V4;
let six = IpAddrKind::V6;
fn route(ip_kind: IpAddrKind) {}

// Instead of embedding the enum in a struct, we can also carry data
enum IpAddr {
    V4(String),
    V6(String),
}

let home = IpAddr::V4(String::from("127.0.0.1"));
let loopback = IpAddr::V6(String::from("::1"));

// The two don't have to take the same arguments
enum IpAddr {
    V4(u8, u8, u8, u8),
    V6(String),
}

let home = IpAddr::V4(127, 0, 0, 1);
let loopback = IpAddr::V6(String::from("::1"));

// Enums can even contain structs, like the standard library `IpAddr`
struct Ipv4Addr { /* --snip-- */ }
struct Ipv6Addr { /* --snip-- */ }
enum IpAddr {
    V4(Ipv4Addr),
    V6(Ipv6Addr),
}

// Here's an enum with four variants with different types:
enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

// We can also define methods on enums with `impl`
```

```
impl Message {
    fn call(&self) {
        // method body would be defined here
    }
}

let m = Message::Write(String::from("hello"));
m.call();
```

- As you can see above, the name of each enum variant becomes a function that constructs an instance of the enum. It can optionally take data arguments.
- The `Option` enum is included in the standard library and encodes when a value could be something or nothing. `Option` is included in the prelude and does not need to be brought into scope explicitly. Even its variants are included in the prelude, so you can use `Some` and `None` without the `Option::` prefix.
- Rust does not have `null` or `nil`. It instead has `Option<T>`.

```
enum Option<T> {
    None,
    Some(T),
}

let some_number = Some(5);              // Option<i32>
let some_char = Some('e');              // Option<char>
let absent_number: Option<i32> = None;
```

- You cannot add an `i32` to an `Option<i32>` without converting the latter. But how do you get the `i32` out of an `Option<i32>`? In general, you need code to handle each variant. That way you explicitly deal with the `None` case. `match` is an easy way to do so.

6.2 - The `match` Control Flow Construct

- `match` allows you to compare a value against a series of patterns and then execute code based on which pattern matches. Patterns can be made up of literal values, variable names, wildcards, and many other things; Chapter 19 is all about patterns. The power of `match` comes from the expressiveness of patterns and the fact that the compiler confirms that all possible cases are handled. The code block associated with the first matched pattern is executed.

```
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter,
}

fn value_in_cents(coin: Coin) -> u8 {
    match coin {
        Coin::Penny => 1,
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter => 25,
    }
}
```

- A `match` is said to have arms. An arm is a pattern and some code. The `=>` operator separates the pattern and the code to run. Each arm is separated from the next with a comma. The code associated with each arm is an expression, and the resultant value of the expression in the matching arm is the value that gets returned for the entire `match` expression. In our example, each expression is short, so we do not use curly brackets. If you want multiple lines of code in a match arm, you must use curly brackets and the comma following the arm is then optional.

```
fn value_in_cents(coin: Coin) -> u8 {
    match coin {
        Coin::Penny => {
            println!("Lucky penny!");
            1
        }
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter => 25,
    }
}
```

- Patterns can bind to the parts of the values that match the pattern.

```rust
#[derive(Debug)]  // so we can inspect the state in a minute
enum UsState {
    Alabama,
    Alaska,
    // --snip--
}

enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}

fn value_in_cents(coin: Coin) -> u8 {
    match coin {
        Coin::Penny => 1,
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter(state) => {
            println!("State quarter from {state:?}!");
            25
        }
    }
}
```

- You can see how this would be useful for pulling the value out of an `Option`.

```rust
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        None => None,
        Some(i) => Some(i + 1),
    }
}

let five = Some(5);
let six = plus_one(five);
let none = plus_one(None);
```

- Matches in Rust are *exhaustive*. The arms of a `match` must cover all possibilities. A `match` that only handles the `Some` case will not compile. If you don't care about some values, you can use a catch-all pattern:

```rust
let dice_roll = 9;
match dice_roll {
    3 => add_fancy_hat(),
    7 => remove_fancy_hat(),
    other => move_player(other), // matches all other values of `u8`
}

fn add_fancy_hat() {}
fn remove_fancy_hat() {}
fn move_player(num_spaces: u8) {}
```

- If we want the catch-all without binding a value, use `_`.
- If you do not want to do anything in an arm, you can use the `()` expression.
- If an enum contains non-copyable data like a String, then you should be careful with whether a match will move or borrow that data. A match on an `Option<String>` will move non-ignored fields. The idiomatic solution is to match on a reference.

```rust
let opt: Option<String> =
    Some(String::from("Hello world"));  // opt        [init]   +R  . +0
                                        // opt@Some.0 [init]   +R  . +0
match &opt {                            // opt        [needs]   R
                                        // opt        [borrow]  R  . -0
                                        // opt@Some.0 [borrow]  R  . -0
    Some(s) => println!("Some: {}", s), // s          [init]   +R  . +0
                                        // s          [needs]   R
                                        // s          [end]    -R  . -0
                                        // opt        [regain]  R  . +0
                                        // opt@Some.0 [regain]  R  . +0
    None => println!("None!"),          // opt        [regain]  R  . +0
                                        // opt@Some.0 [regain]  R  . +0
};
println!("{:?}", opt);                  // opt        [needs]   R
                                        // opt        [end]    -R  . -0
                                        // opt@Some.0 [end]    -R  . -0
```

- If we had done the above without the reference, `Some(s)` would have moved the `String` out of `opt` into `s`, and the `println!` wouldn't compile: `error[E0382]: borrow of partially moved value: `opt`` . Rust will "push down" the reference from the outer enum, `&Option<String>`, to the inner field, `&String`. Therefore `s` has type `&String` and `opt` can be used after the match.

6.3 - Concise Control Flow with if let and let...else

- `if let` syntax lets you combine `if` and `let` into a less verbose way to handle values that match one pattern while ignoring the rest.

```
let config_max = Some(3u8);
match config_max {
    Some(max) => println!("The maximum is configured to be {max}"),
    _ => (),
}

// This is equivalent to above
let config_max = Some(3u8);
if let Some(max) = config_max {
    println!("The maximum is configured to be {max}");
}
```

- You can also include an `else` with an `if let`, and the block of code associated with `else` is the same as a block of code associated with the `_`.

```
let mut count = 0;
match coin {
    Coin::Quarter(state) => println!("State quarter from {state:?}!"),
    _ => count += 1,
}

// equivalently
let mut count = 0;
if let Coin::Quarter(state) = coin {
    println!("State quarter from {state:?}!");
} else {
    count += 1;
}
```

- `let...else` lets us bind values from a pattern if it matches, and execute an early-exit `else` block if it does not. `let...else` takes a pattern on the left side and an expression on the right, but it does not have an `if` branch, only an `else`. If the pattern matches, it will bind the value from the pattern, but if it does not match it will follow the `else` arm, which must diverge (usually by returning from the function):

```rust
fn describe_state_quarter(coin: Coin) -> Option<String> {
    let Coin::Quarter(state) = coin else {
        return None;
    };

    if state.existed_in(1900) {
        Some(format!("{state:?} is pretty old, for America!"))
    } else {
        Some(format!("{state:?} is relatively new."))
    }
}
```

- Using custom types defined by `enum` or `struct` ensures type safety: The compiler will make certain your functions only get values of the type each function expects.

6.4 - Ownership Inventory #1

- *Note: Ownership Inventory #1 has been moved to the Appendix at the end of these notes for high-yield review.*

Chapter 7 - Packages, Crates, and Modules

- As a project grows, you should organize code by splitting it into multiple modules and then multiple files. As a package grows, you can extract parts into separate crates that become external dependencies.
- A package can contain multiple binary crates and optionally one library crate. For very large projects comprising a set of interrelated packages that evolve together, Cargo provides workspaces which will be covered in Chapter 14.
- Features for managing your code's organization, collectively referred to as the *module system*, include:
  - **Packages**: A Cargo feature that lets you build, test, and share crates
  - **Crates**: A tree of modules that produces a library or executable
  - **Modules and use**: Let you control the organization, scope, and privacy of paths
  - **Paths**: A way of naming an item, such as a struct, function, or module

7.1 - Packages and Crates

- A *crate* is the smallest amount of code that the Rust compiler considers at a time. Crates can contain modules, and the modules may be defined in other files that get compiled with the crate. A crate can come in two forms: a binary crate or a library crate. *Binary crates* are programs you can compile to an executable that you can run. Each must have a function called `main` that defines what happens when the executable runs. *Library crates* don't have a `main` function and don't compile to an executable. Instead they define functionality intended to be shared with multiple projects. Most of the time when Rustaceans say "crate," they mean library crate, and they use "crate" interchangeably with the general programming concept of "library".
- The *crate root* is a source file that the Rust compiler starts from and makes up the root module of your crate.
- A *package* is a bundle of one or more crates that provides a set of functionality. A package contains a *Cargo.toml* file that describes how to build those crates. A package can contain as many binary crates as you like, but at most only one library crate. A package must contain at least one crate.
- When you run `cargo new my-project`, Cargo creates a `my-project` directory and in it a *Cargo.toml* and *src/main.rs*. *Cargo.toml* doesn't mention the source file. Cargo follows the convention that *src/main.rs* is the crate root of a binary crate with the same name as the package. Likewise, Cargo knows that if the package directory contains *src/lib.rs*, the package contains a library crate with the same name as the package and *src/lib.rs* is the crate root. Cargo passes crate root files to `rustc` to build the library or binary. This default new project is a binary crate named `my-project`. If a package contains both *src/main.rs* and *src/lib.rs*, it has two crates: a binary and a library, both with the same name as the package. A package can have multiple binary crates by placing files in the *src/bin* directory: each file will be a separate binary crate.

7.2 - Control Scope and Privacy with Modules

- **Start from the crate root**: When compiling a crate, the compiler first looks in the crate root file (usually *src/lib.rs* for a library crate and *src/main.rs* for a binary crate) for code to compile.
- **Declaring modules**: In the crate root file, you can declare new modules, say for example "garden", with `mod garden;`. The compiler will look for the module's code in:
  - Inline, within curly brackets that replace the semicolon following `mod garden`.
  - In the file *src/garden.rs*

    - In the file *src/garden/mod.rs*
- **Declaring submodules**: In any file other than the crate root, you can declare submodules. For example, you might declare `mod vegetables;` in *src/garden.rs*. The compiler will look for the submodule's code within the directory name for the parent module in these places:
    - Inline, directly following `mod vegetables`, within curly brackets instead of the semicolon
    - In the file *src/garden/vegetables.rs*
    - In the file *src/garden/vegetables/mod.rs*
- **Paths to code in modules**: Once a module is part of your crate, you can refer to code in that module from anywhere else in that same crate, as long as the privacy rules allow, using the path to the code. For example, an `Asparagus` type in the garden vegetables module would be found at `crate::garden::vegetables::Asparagus`.
- **Private vs public**: Code within a module is private from its parent modules by default. To make a module public, declare it with `pub mod` instead of `mod`. To make items within a public module public as well, use `pub` before their declarations.
- **The `use` keyword**: Within a scope, the `use` keyword creates shortcuts to items to reduce repetition of long paths. In any scope that can refer to `crate::garden::vegetables::Asparagus`, you can create a shortcut with `use crate::garden::vegetables::Asparagus`, and from then on you only need to write `Asparagus` to make use of that type in the scope.
- *Modules* let us organize code within a crate for readability and easy reuse. Modules also allow us to control the *privacy* of items because code within a module is private by default.
- The source code provides an extended example of a `restaurant` library in `ch07_restaurant`.
- The contents of either *src/main.rs* or *src/lib.rs* are *crate roots* and the contents of either of these two files form a module named `crate` at the root of the crate's module structure, known as the *module tree*.

7.3 - Paths for Referring to an Item in the Module Tree

- An *absolute path* is the full path starting from a crate root; for code from an external crate, the absolute path begins with the crate name, and for code from the current crate, it starts with the literal `crate`.
- A *relative path* starts from the current module and uses `self`, `super`, or an identifier in the current module.

- Both absolute and relative paths are followed by one or more identifiers separated by double colons (`::`).
- Using absolute vs relative path comes down to if you're more likely to move item definition code separately from or together with the code that uses an item. The preference in general is to specify absolute paths because it's more likely we'll want to move code definitions and item calls independently.
- In Rust if you want to make an item private, you put it in a module. By default all items are private even to parent modules. However, items in child modules can use the items in their ancestor modules. Sibling modules are accessible, but only their `pub` items are visible to siblings. A private item is visible in the module where it's defined and all of that module's descendants.
- Modules are containers so making a module public doesn't do much on its own without making items within it public as well.
- Typically a crate with a binary and a library will have just enough code in the binary to start an executable that calls code defined in the library. The library can be shared, so other projects benefit from most of the functionality that the package provides. The module tree should be defined in *src/lib.rs*. Then, any public items can be used in the binary crate by starting paths from the name of the package. The binary crate becomes a user of the library crate just like a completely external crate would use the library crate: It can only use the public API.
- We can construct relative paths that begin in the parent module, rather than the current module or crate root, by using `super` at the start of the path. This makes rearranging the module tree easier when the module is closely related to the parent but the parent might be moved elsewhere.
- If we have a struct with a private field, the struct needs to provide a public associated function that constructs an instance or we can't create an instance of the struct outside the module since we couldn't set the value of the private field. If we make an enum public, all its variants are public.

7.4 - Bringing Paths into Scope with the `use` Keyword

- We can create a shortcut to a path with the `use` keyword once and then use the shorter name everywhere else in scope. `use` commands only create shortcuts for the particular scope in which the `use` occurs. Submodules have separate scopes and won't see `use` declared in a parent module without calling `super::`.

- The idiomatic way to bring a function into scope is to bring the function's parent module into scope with `use`. This makes it clear the function isn't locally defined (since calls to it are preceded with the parent module name) while still minimizing repetition of the full path. It is idiomatic to specify the full path for structs, enums, and other items with `use`.

```
use std::collections::HashMap;

fn main() {
    let mut map = HashMap::new();
    map.insert(1, 2);
}
```

- Bringing two items with the same name into the same scope requires using their parent modules or else use the `as` keyword. `as` lets you specify a new local name, or *alias*, for the type.

- *Re-exporting* is combining `pub` and `use` so that we bring the item into our scope but also make that item available for others to bring into their scope. It is useful when the internal structure of your code is different from how programmers calling your code would think about the domain.

- To use an external package we put the library and version in our *Cargo.toml* then `use` the items we want to bring into scope, or prefix them with the package name and `::`. We don't have to specify `std` in our *Cargo.toml*, but we do need to `use` items from there to bring them into scope. For example, `use std::collections::HashMap` allows you to refer to `HashMap`.

- We can `use` two or more items in the same path by specifying the shared portion of the path followed by `::{item1, item2}`. For example, `use std::{cmp::Ordering, io}`. We can `use std::io::{self, Write}` to bring both `std::io` and `std::io::Write` into scope.

- If we want to bring all public items defined in `std::collections` into scope, we use the glob operator (`*`) like `use std::collections::*`. It is often used when testing to bring everything under test into the `tests` module.

- Here's the full code of the project before splitting it into different files:

```rust
// ch07_restaurant/src/lib.rs
mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}
        fn seat_at_table() {}
    }

    mod serving {
        fn take_order() {}
        fn serve_order() {}
        fn take_payment() {}
    }
}

mod back_of_house {
    pub struct Breakfast {
        pub toast: String,
        seasonal_fruit: String,
    }

    impl Breakfast {
        pub fn summer(toast: &str) -> Breakfast {
            Breakfast {
                toast: String::from(toast),
                seasonal_fruit: String::from("peaches"),
            }
        }
    }

    pub enum Appetizer {
        Soup,
        Salad,
    }

    fn fix_incorrect_order() {
        cook_order();
        super::deliver_order();
    }

    fn cook_order() {}
}

pub use crate::front_of_house::hosting;

pub fn eat_at_restaurant() {
```

```
    // Absolute path
    crate::front_of_house::hosting::add_to_waitlist();

    // Relative path
    front_of_house::hosting::add_to_waitlist();

    // Order a breakfast in the summer with Rye toast.
    let mut meal = back_of_house::Breakfast::summer("Rye");
    // Change our mind about what bread we'd like
    meal.toast = String::from("wheat");
    println!("I'd like {} toast please", meal.toast);

    // The next line won't compile if we uncomment it; we're not allowed
    // to see or modify the seasonal fruit that comes with the meal.
    // meal.seasonal_fruit = String::from("blueberries");

    let order1 = back_of_house::Appetizer::Soup;
    let order2 = back_of_house::Appetizer::Salad;

    // Because of earlier `use` command this works
    hosting::add_to_waitlist();
}

fn deliver_order() {}
```

7.5 - Separating Modules into Different Files

- The book shows moving `front_of_house` to *src/front_of_house.rs*. It also moves its child `front_of_house::hosting` to *src/front_of_house/hosting.rs*. I have made these changes to the code in `ch07_restaurant` and it compiles. There is an older style still supported where `front_of_house` goes within *src/front_of_house/mod.rs* and `hosting` goes in *src/front_of_house/hosting/mod.rs*. The main drawback of the old style is having many open files named `mod.rs`.
- You only need to load a file using a `mod` declaration *once* in your module tree. Other files refer to it using a path to where it was declared.
- Not covered in the book: there are more visibility levels than private and `pub`. `pub(crate)` is the common one, for sharing an item between modules without making it part of the crate's public API:

Visibility	Visible to
(none)	This module and its child modules
`pub(super)`	Also the parent module
`pub(crate)`	The whole crate, but not other crates
`pub(in crate::some::path)`	Everything under that module
`pub`	Everyone, including other crates

Chapter 8 - Common Collections

- Collections can contain multiple values and, unlike the built-in array and tuple types, the data is stored on the heap so the amount does not need to be known at compile time and can grow or shrink as the program runs. We will discuss three collections in this chapter:
  - A *vector* allows you to store a variable number of values next to each other.
  - A *string* is a collection of characters.
  - A *hash map* allows you to associate a value with a specific key. It's a particular implementation of the more general data structure called a *map*.

8.1 - Storing Lists of Values with Vectors

- Vectors (`Vec<T>`) are useful when you have a list of items of the same type.
- To create a new, empty vector call `Vec::new` . For example, `let v: Vec<i32> = Vec::new();` . Because we aren't inserting values right away, Rust needs the type annotation here. More often, you'll create a `Vec<T>` with initial values and Rust will infer the type. Rust provides the `vec!` macro to create a new vector that holds the values it is given. For example, `let v = vec![1, 2, 3];` . Rust will infer the type to be `Vec<i32>` .
- As with any variable, if we want to be able to change a vector, we need to make it mutable using the `mut` keyword: `let mut v = Vec::new();` . You can add new values to a mutable vector with the `push` method. If we push `i32` values after declaring the new vector, we don't need the type annotation where it is declared.
- You can access values by indexing like `let third: &i32 = &v[2];` or as an option from using get like `let third: Option<&i32> = v.get(2);` . Indexing outside of range will cause a panic, but using `get` outside of range returns `None` .

- Recall that you can't have a mutable reference while an immutable reference is still in use.

```
// DOES NOT COMPILE! error[E0502]: cannot borrow `v` as mutable because it is
//                                 also borrowed as immutable
let mut v = vec![1, 2, 3, 4, 5];
let first = &v[0];
v.push(6);
println!("The first element is: {first}");
```

- To access each element in a vector in turn, we can use a `for` loop to get immutable references to each element:

```
let v = vec![100, 32, 57];
for i in &v {
    println!("{i}");
}
```

- We can also iterate over mutable references of a mutable vector in order to make changes to all the elements. To get the value in `i` and add 1 to it, we need to dereference `i` with `*`:

```
let mut v = vec![100, 32, 57];
for i in &mut v {
    *i += 50;
}
```

- An iterator is a pointer that moves through each element of a vector. Each call to `next` returns `Some` with the current element and advances the pointer, or returns `None` at the end. Rust does not allow you to add or remove elements from a vector during iteration. A `push` operation could cause reallocation and invalidate the pointer. You can iterate over the range of indexes into a vector without using a pointer to the vector.
- The variants of an `enum` are the same type, so different variants can be in the same vector:

```
enum SpreadsheetCell {
    Int(i32),
    Float(f64),
    Text(String),
}

let row = vec![
    SpreadsheetCell::Int(3),
    SpreadsheetCell::Text(String::from("blue")),
    SpreadsheetCell::Float(10.12),
];
```

- Vector has many methods and is documented at: https://doc.rust-lang.org/std/vec/struct.Vec.html. In addition to `push` which adds to the end, there is `pop` that returns an `Option` containing the final element and removes it.

```
let mut stack = Vec::new();
stack.push(1);
stack.push(2);
stack.push(3);
while let Some(top) = stack.pop() {
    // Prints 3, 2, 1
    println!("{top}");
}
```

- When a vector is dropped, all of its contents are also dropped. The borrow checker ensures that any reference to contents of a vector is only used while the vector itself is valid.

```
{
    let v = vec![1, 2, 3, 4];
    // do stuff with v
} // <- v goes out of scope and is freed here
```

- `&[T]` works like `&str` for any list of elements: functions take a slice and can then accept a `Vec`, an array, or part of either. A `&Vec<T>` is converted to `&[T]` automatically, and slice syntax works on vectors too.

```rust
fn sum(list: &[i32]) -> i32 {
    let mut total = 0;
    for n in list {
        total += n;
    }
    total
}

let v = vec![1, 2, 3, 4, 5];
let a = [10, 20, 30];
sum(&v);       // 15, &Vec<i32> becomes &[i32]
sum(&a);       // 60, arrays work too
sum(&v[1..3]); // 5, a slice of the Vec: [2, 3]
```

8.2 - Storing UTF-8 Encoded Text with Strings

- Strings are implemented as a collection of bytes, plus some methods to provide useful functionality when those bytes are interpreted as text. Rust has only one string type in the core language, the string slice `str` most commonly seen in its borrowed form `&str`. The `String` type is part of the standard library and is a growable, mutable, owned, UTF-8 encoded string type. `String` is actually implemented as a wrapper around a vector of bytes with some extra guarantees, restrictions, and capabilities.

- `new` works the same as with `Vec`. `let mut s = String::new()` creates a new, empty string called `s`. Often we have some data to start from, so we use the `to_string` method, which is available on any type that implements the `Display` trait including string literals.

```rust
let data = "initial contents";
let s = data.to_string();
// The method also works on a literal directly:
let s = "initial contents".to_string();
// This is also equivalent
let s = String::from("initial contents");
```

- You can append to a string with `push_str`. You can use the `+` operator and `format!` macro to also concatenate string values. `push` adds a single character to a string.

```rust
let mut s1 = String::from("foo");
let s2 = "bar";
s1.push_str(s2); // s1 = "foobar"
println!("s2 is {s2}"); // "s2 is bar", push_str did not take ownership of s2

let s1 = String::from("Hello, ");
let s2 = String::from("world!");
let s3 = s1 + &s2; // s1 has been moved here and can no longer be used
// s3 = "Hello, world!"
// `+` uses the `add` method: fn add(self, s: &str) -> String {

// `format!` is useful for combining strings in more complicated ways
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");
let s = format!("{s1}-{s2}-{s3}");
```

- `add` takes a string slice, but we passed `&s2`, a `&String`. The compiler coerces the `&String` argument into a `&str`. `add` takes ownership of its first argument since it is `self` and not `&self`. `format!` works like `println!` but returns a `String` with the contents rather than printing output. The code generated by `format!` uses references so doesn't take ownership of any of its parameters.

- You cannot use indexing syntax to access individual characters of a string in Rust. `String` is a wrapper over `Vec<u8>`, and some characters are multi-byte. You can create a slice with the indexing syntax, like `let s = &hello[0..4]` for the first four bytes of a string. You will get a panic at runtime if you try to cut across a char boundary, so be careful doing this in code.

- Be explicit about whether you want characters or bytes when working with strings. The `chars` method lets you iterate across each character and the `bytes` method lets you iterate across each byte, and `char_indices` returns a pair of `(byte_index, char)`:

```rust
for c in "Зд".chars() {
    println!("{c}");
}

for b in "Зд".bytes() {
    println!("{b}");
}

for (i, c) in "Зд".char_indices() {
    println!("{i}: {c}"); // i is always a valid slice boundary
}
```

- The `str` page is at https://doc.rust-lang.org/std/primitive.str.html. Here are the methods you'll use most, all of which work on `&str` and therefore on `String` too:

```rust
let s = String::from("Hello, world");

// searching (return bool or Option<usize> byte index)
s.contains("world");            // true
s.starts_with("Hello");         // true
s.ends_with('d');               // true (char or &str patterns)
s.find("o");                    // Some(4)
s.rfind('o');                   // Some(8)

// transforming (return a new String)
s.replace("world", "Rust");     // "Hello, Rust"
s.replacen('l', "L", 2);        // "HeLLo, world"
s.to_uppercase();               // "HELLO, WORLD"
s.to_lowercase();               // "hello, world"
"ab".repeat(3);                 // "ababab"

// trimming and splitting (return &str slices, no allocation)
"  hi  ".trim();                // "hi" (also trim_start, trim_end)
s.split(", ");                  // iterator: "Hello", "world"
"a b  c".split_whitespace();    // iterator: "a", "b", "c"
"a\nb".lines();                 // iterator: "a", "b"
s.split_once(", ");             // Some(("Hello", "world"))
s.strip_prefix("Hello, ");      // Some("world")

// building and modifying (String only, needs `mut`)
let mut t = String::new();
t.push_str("foo");              // "foo"
t.push('!');                    // "foo!"
t.insert(0, '>');               // ">foo!"
t.pop();                        // Some('!'), t is ">foo"
t.clear();                      // ""
```

- **Methods that return pieces of the string** (`trim` , `split` , `strip_prefix`) return `&str` slices pointing into the original, so they're cheap.
- **Methods that change the text** (`replace` , `to_uppercase`) have to allocate and return a new `String` .
- **Searches give byte positions.** `find` returns a byte index, not a character index. That fits the UTF-8 discussion above, and it means you can pass the result straight to `&s[i..]` .
- **Splitting returns iterators,** so you'll usually end with `.collect::<Vec<_>>()` or loop over the result with `for` . Chapter 13 explains iterators properly.

8.3 - Storing Keys with Associated Values in Hash Maps

- A `HashMap<K, V>` is a mapping of keys of type `K` to values of type `V`. Like vectors, hash maps are homogeneous: All of the keys must have the same type, and all of the values must have the same type.
- You can use `new` and then add elements with `insert`. The `get` method allows you to get values back out:

```
use std::collections::HashMap;
let mut scores = HashMap::new();
scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);
let team_name = String::from("Blue");
let score = scores.get(&team_name).copied().unwrap_or(0); // score = 10
for (key, value) in &scores {
    println!("{key}: {value}");
}

// You can also build a HashMap from an array of pairs with HashMap::from
let scores = HashMap::from([
    ("Blue", 10),
    ("Yellow", 50),
]);
```

- `get` returns an `Option<&V>` so here `copied` is converting it from `Option<&i32>` to `Option<i32>` then `unwrap_or` is extracting the `i32` value or else returning a 0 for `None`.
- For types that implement the `Copy` trait, the values are copied into the hash map. For owned values, the values will be moved and the hash map will be the owner of those values. If we insert references to values into the hash map, the values won't be moved into the hash map. The values the references point to need to be valid for at least as long as the hash map is valid.

```
use std::collections::HashMap;
let field_name = String::from("Favorite color");
let field_value = String::from("Blue");
let mut map = HashMap::new();
map.insert(field_name, field_value);
// field_name and field_value are now invalid!
```

- Keys are unique, so there are several ways to deal with a key already existing in a hash map. Either you overwrite the old value, keep the old value, or combine the old and new value. `insert` overwrites old values with new. `entry` is used to check if a key exists and only insert if not. `or_insert` returns a mutable reference to a value for a key if it exists or else inserts the new key/value pair and returns a mutable reference to the new value. The mutable reference is useful if you want to update the value.

```rust
use std::collections::HashMap;
let mut scores = HashMap::new();
scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Blue"), 50);
println!("{scores:?}"); // {"Blue": 50}

let mut scores = HashMap::new();
scores.insert(String::from("Blue"), 10);
scores.entry(String::from("Yellow")).or_insert(50);
scores.entry(String::from("Blue")).or_insert(50);
println!("{scores:?}"); // {"Yellow": 50, "Blue": 10}

let text = "hello world wonderful world";
let mut map = HashMap::new();
for word in text.split_whitespace() {
    // `count` is a &mut i32 into the map; while it's in use the map is
    // mutably borrowed, but the borrow ends after its last use (*count += 1)
    let count = map.entry(word).or_insert(0);
    *count += 1;
}
println!("{map:?}"); // {"world": 2, "hello": 1, "wonderful": 1}

// Inside the loop, you could just update the map directly
*map.entry(word).or_insert(0) += 1;

counts.entry(word).or_default(); // uses the type's default: 0, "", empty Vec
// the following computes the default only if needed
counts.entry(word).or_insert_with(|| expensive());
counts.entry(word)
    .and_modify(|c| *c += 1)              // runs only if the key exists
    .or_insert(1);                        // runs only if it doesn't

// You can also iterate over a HashMap, `key` and `value` are references.
// Using `for (&key, &value) in &counts` matches and copies the values so you
// don't have to dereference them inside the loop body.
for (key, value) in &counts {
    println!("{key}: {value}");
}

// `entry` is also useful for grouping
let words = ["apple", "avocado", "banana", "blueberry", "cherry"];
let mut by_letter: HashMap<char, Vec<&str>> = HashMap::new();
for w in words {
    by_letter.entry(w.chars().next().unwrap())
        .or_insert_with(Vec::new)   // only builds the Vec if the key is new
```

```
        .push(w);
}
// {'a': ["apple", "avocado"], 'b': ["banana", "blueberry"], 'c': ["cherry"]}
```

- `or_insert_with(Vec::new)` is better than `or_insert(Vec::new())` because the second version builds a `Vec` every time, even when the key already exists, and then throws it away. `or_default()` is the shortest form when the type's default value is what you want.
- `BTreeMap` is also in `std::collections`. It's a map that keeps its keys sorted (a B-tree, not a hash map).
- The default hashing function provides resistance against denial of service attacks at the cost of performance. There are other hashing algorithms available or you could write your own.
- Exercises
    - Given a list of integers, use a vector and return the median (when sorted, the value in the middle position) and mode (the value that occurs most often; a hash map will be helpful here) of the list.

```rust
// ch08_collections/src/main.rs
use std::collections::HashMap;

fn main() {
    let v = vec![1, 2, 3, 4, 5, 5, 5];
    println!(
        "The median is {}",
        median(&v).expect("No median of an empty list!")
    );
    println!(
        "The mode is {}",
        mode(&v).expect("No mode of an empty list!")
    );
}

fn median(v: &[i32]) -> Option<f64> {
    if v.is_empty() {
        return None;
    }
    let mut v = v.to_vec();
    v.sort();
    let l = v.len();
    if l.is_multiple_of(2) {
        Some((v[l / 2 - 1] as f64 + v[l / 2] as f64) / 2.0)
    } else {
        Some(v[l / 2] as f64)
    }
}

fn mode(v: &[i32]) -> Option<i32> {
    if v.is_empty() {
        return None;
    }
    let mut counts = HashMap::new();
    for &item in v {
        *counts.entry(item).or_insert(0) += 1;
    }
    let mut mode = v[0];
    let mut mode_count = 0;
    for (&item, &count) in &counts {
        if count > mode_count || (count == mode_count && item < mode) {
            mode_count = count;
            mode = item;
        }
    }
```

```
        Some(mode)
    }
```

- Convert strings to Pig Latin. The first consonant of each word is moved to the end of the word and ay is added, so first becomes irst-fay. Words that start with a vowel have hay added to the end instead (apple becomes apple-hay). Keep in mind the details about UTF-8 encoding!
    - Here's my long-winded version:

```rust
// ch08_collections/src/bin/pig_latin.rs
fn main() {
    let sample_string =
    String::from("This is a test of the pig latin system!");
    println!("{}", pig_latinize(&sample_string));
}

fn pig_latinize(s: &str) -> String {
    let mut new_words = Vec::new();
    let mut word_count = 0;
    let l = s.split_whitespace().count();
    for word in s.split_whitespace() {
        let mut new_word = String::new();
        let mut first_letter = ' ';
        let mut punctuation = ' ';
        word_count += 1;
        for (i, c) in word.to_lowercase().chars().enumerate() {
            if i == 0 {
                first_letter = c;
            } else {
                if !c.is_ascii_punctuation() {
                    new_word.push(c);
                } else {
                    punctuation = c;
                }
                if i == word.chars().count() - 1 {
                    new_word.push('-');
                    new_word.push(first_letter);
                    new_word.push_str("ay");
                    if word_count == l && punctuation != ' ' {
                        new_word.push(punctuation);
                    }
                }
            }
            if "aeiouAEIOU".contains(first_letter) {
                new_word.push_str(word);
                new_word.push_str("-hay");
                break;
            }
        }
        new_words.push(new_word);
    }
    capitalize_first(&new_words.join(" "))
}
```

```rust
fn capitalize_first(s: &str) -> String {
    let mut new_string = String::new();
    for (i, c) in s.char_indices() {
        if i == 0 {
            new_string.push(c.to_ascii_uppercase());
        } else {
            new_string.push(c);
        }
    }
    new_string
}
```

- Here's AI's shorter version using iterators:

```rust
fn pig_latin_word(word: &str) -> String {
    let Some(first) = word.chars().next() else {
        return String::new();
    };
    let rest = &word[first.len_utf8()..];
    if "aeiouAEIOU".contains(first) {
        format!("{word}-hay")
    } else {
        format!("{rest}-{first}ay")
    }
}

fn pig_latinize(s: &str) -> String {
    let mut new_words = Vec::new();
    for word in s.split_whitespace() {
        new_words.push(pig_latin_word(word));
    }
    new_words.join(" ")
}
```

- And here's a version of `pig_latin_word` that uses match-guards:

```rust
fn pig_latin_word(word: &str) -> String {
    let mut first = None;
    let mut rest = String::new();
    for c in word.to_lowercase().chars() {
        if first.is_none() {
            first = Some(c);
        } else {
            rest.push(c);
        }
    }
    match first {
        None => String::new(),
        Some(f) if "aeiou".contains(f) => format!("{f}{rest}-hay"),
        Some(f) if rest.is_empty() => format!("{f}ay"),   // "b" -> "bay"
        Some(f) => format!("{rest}-{f}ay"),
    }
}
```

- Using a hash map and vectors, create a text interface to allow a user to add employee names to a department in a company; for example, "Add Sally to Engineering" or "Add Amir to Sales." Then, let the user retrieve a list of all people in a department or all people in the company by department, sorted alphabetically.

```rust
use std::collections::HashMap;
use std::io;

fn main() {
    let mut company: HashMap<String, Vec<String>> = HashMap::new();

    println!("Commands: Add <name> to <department>, List <department>,");
    println!("          List all, Quit");

    loop {
        let mut input = String::new();
        io::stdin()
            .read_line(&mut input)
            .expect("Failed to read line");

        let mut words = Vec::new();
        for word in input.split_whitespace() {
            words.push(word);
        }

        if words.is_empty() || words[0] == "Quit" {
            break;
        } else if words[0] == "Add"
            && words.len() >= 4
            && words[2] == "to"
        {
            add_employee(&mut company, words[1], &words[3..].join(" "));
        } else if words[0] == "List"
            && words.len() == 2
            && words[1] == "all"
        {
            list_all(&company);
        } else if words[0] == "List" && words.len() >= 2 {
            list_department(&company, &words[1..].join(" "));
        } else {
            println!("Sorry, I didn't understand that.");
        }
    }
}

fn add_employee(
    company: &mut HashMap<String, Vec<String>>,
    name: &str,
    dept: &str,
) {
```

```
    let people = company.entry(dept.to_string()).or_default();
    people.push(name.to_string());
    println!("Added {name} to {dept}.");
}

fn list_department(company: &HashMap<String, Vec<String>>, dept: &str) {
    match company.get(dept) {
        Some(people) => {
            let mut people = people.clone();
            people.sort();
            println!("{dept}:");
            for person in &people {
                println!("  {person}");
            }
        }
        None => println!("No department named {dept}."),
    }
}

fn list_all(company: &HashMap<String, Vec<String>>) {
    let mut depts = Vec::new();
    for dept in company.keys() {
        depts.push(dept);
    }
    depts.sort();
    for dept in depts {
        list_department(company, dept);
    }
}
```

- You could also use slice-pattern, but that hasn't been covered yet:

```
match words.as_slice() {
    ["Add", name, "to", dept @ ..] if !dept.is_empty() => {
        add_employee(&mut company, name, &dept.join(" "));
    }
    ["List", "all"] => list_all(&company),
    ["List", dept @ ..] if !dept.is_empty() => {
        list_department(&company, &dept.join(" "));
    }
    ["Quit"] | [] => break,
    _ => println!("Sorry, I didn't understand that."),
}
```

8.4 - Ownership Inventory #2

- *Note: Ownership Inventory #2 has been moved to the Appendix at the end of these notes for high-yield review.*

Chapter 9 - Error Handling

- Rust groups errors into two major categories: recoverable and unrecoverable errors. For a *recoverable error*, such as "file not found", we just want to report the error and allow the user to retry the operation. *Unrecoverable errors* are always symptoms of bugs, like trying to access a location beyond the end of an array, so we want to immediately stop the program. Rust doesn't have exceptions, which are the single way many other languages try to deal with both types of error. It has `Result<T, E>` for recoverable errors and `panic!` for unrecoverable errors.

9.1 - Unrecoverable Errors with `panic!`

- You can cause a panic by taking an action that causes a panic or by calling the `panic!` macro. By default, a panic prints a failure message, unwinds, cleans up the stack, and quits. Via an environment variable, you can have Rust display the call stack when a panic occurs.
- Unwinding (walking back up the stack and cleaning up each function's data) is a lot of work and makes the binary larger. You can instead abort immediately and let the operating system reclaim the memory by adding `panic = 'abort'` to the appropriate `[profile]` sections in your *Cargo.toml*.
- You can get a backtrace in the event of a panic by setting `RUST_BACKTRACE=1`. The key to reading the backtrace is to start from the top and read until you see files you wrote. That's the spot where the problem originated. To get a backtrace, debug symbols must be enabled. They are enabled by default when building without the `--release` flag.

9.2 - Recoverable Errors with `Result`

- The `Result` enum has two variants: `Ok` and `Err`:

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

- Consider the function `File::open` from `std::fs::File`. It returns a `Result<T, E>` where the `T` is `std::fs::File` which is a file handle on success and the `E` is a `std::io::Error`. We can `match` on these variants.
- The `Result` enum and its variants are brought into scope by the prelude.

```
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file_result = File::open("hello.txt");

    let greeting_file = match greeting_file_result {
        Ok(file) => file,
        Err(error) => match error.kind() {
            ErrorKind::NotFound => match File::create("hello.txt") {
                Ok(fc) => fc,
                Err(e) => panic!("Problem creating the file: {e:?}"),
            },
            _ => {
                panic!("Problem opening the file: {error:?}");
            }
        },
    };
}
```

- `io::Error` is a struct provided by the standard library. It has a method `kind` that we can call to get an `io::ErrorKind` value. The enum `io::ErrorKind` is also provided by the standard library and has variants for the different kinds of errors that might result from an `io` operation. Here we are prepared to deal with `NotFound` by creating the file and willing to crash for other errors.
- `match` is very useful but also very primitive. `Result` has methods that can be more convenient, often using closures which we'll introduce in Chapter 13. Here is the above rewritten:

```rust
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file = File::open("hello.txt").unwrap_or_else(|error| {
        if error.kind() == ErrorKind::NotFound {
            File::create("hello.txt").unwrap_or_else(|error| {
                panic!("Problem creating the file: {error:?}");
            })
        } else {
            panic!("Problem opening the file: {error:?}");
        }
    });
}
```

- If you don't have any error handling code and just want to panic, the `unwrap` and `expect` methods are an easy way to get the value out of the `Result` and panic if it's an error. Both return the value inside for an `Ok`, but panic for an `Err`, but `expect` allows you to specify a panic message. Most Rustaceans prefer `expect` to `unwrap` to give more context about why the operation is expected to always succeed.

```rust
use std::fs::File;

fn main() {
    let greeting_file = File::open("hello.txt")
        .expect("hello.txt should be included in this project");
}
```

- Instead of dealing with the `Err` immediately, we may choose to propagate the error to calling code where there might be more context on how to deal with the issue. This is a long way to propagate the error; we will see a short way later:

```rust
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let username_file_result = File::open("hello.txt");

    let mut username_file = match username_file_result {
        Ok(file) => file,
        Err(e) => return Err(e),
    };

    let mut username = String::new();

    match username_file.read_to_string(&mut username) {
        Ok(_) => Ok(username),
        Err(e) => Err(e),
    }
}
```

- This code propagates the error. It returns an `io::Error`, because that is the error type of both calls that could fail. On success it returns `Ok` wrapping the username. Recall that `return` is unnecessary when you are returning the final expression of a function. The calling code may decide to panic, use a default username, or look up the username elsewhere, for example. We don't know what the calling code is trying to do, so we propagate all the success and error information upward for it to handle appropriately.
- The pattern of propagating errors is so common in Rust that Rust provides the question mark operator `?` to make this easier. Here is the last example rewritten using it:

```rust
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let mut username_file = File::open("hello.txt")?;
    let mut username = String::new();
    username_file.read_to_string(&mut username)?;
    Ok(username)
}
```

- In the above code, if the value of the `Result` is `Ok`, the value inside the `Ok` will be returned by the `?` expression, but if the value is an `Err`, the `Err` will be returned from

the whole function as if we had used the `return` keyword. There is one difference between this and the longer example. Error values that have `?` called on them go through the `from` function defined in the `From` trait of the standard library, which converts values from one type into another. The error type received is thus converted into the error type defined in the return type of the current function. This is useful when a function returns one error type to represent all the ways a function might fail, even if parts might fail for many different reasons. For example we could define `OurError` as the return type and also define `impl From<io::Error> for OurError` to construct `OurError` from an `io::Error`, and the `?` operator would convert error types.

- We could make this example even shorter by chaining method calls:

```
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let mut username = String::new();

    File::open("hello.txt")?.read_to_string(&mut username)?;

    Ok(username)
}
```

- And we could make that even shorter by using `fs::read_to_string`, but that wouldn't have given a chance to explain `?`:

```
use std::fs;
use std::io;

fn read_username_from_file() -> Result<String, io::Error> {
    fs::read_to_string("hello.txt")
}
```

- The `?` operator can only be used in functions whose return type is compatible with the value the `?` is used on, because it is defined to perform an early return of a value out of the function.

```rust
// DOES NOT COMPILE! error[E0277]: the `?` operator can only be used in a
//                                 function that returns `Result` or `Option`
use std::fs::File;

fn main() {
    let greeting_file = File::open("hello.txt")?;
}
```

- You can make `main` return a `Result<(), E>`. If it returns an `Err`, the program prints the error and exits with a failure code. Just remember to return `Ok(())` at the end. A `Box<dyn Error>` allows `main` to return any type of error. `main` may return any types that implement the `std::process::Termination` trait which contains a function `report` that returns an `ExitCode`. The standard library documentation has more on implementing the `Termination` trait for your own types.
- As you can see from the error, `?` can also be used with `Option<T>` values if that is what your function returns.

```rust
fn last_char_of_first_line(text: &str) -> Option<char> {
    text.lines().next()?.chars().last()
}
```

- Note you can't mix and match between `Result` and `Option`. You can either use `?` on `Result` values in a function that returns `Result`, or you can use `?` on `Option` values in a function that returns `Option`.
- (Not in the book) Converting between wrappers (`Option` and `Result`):
  - Rust provides adapter methods to freely convert between the two:

From	To	Method	Conversion
`Result<T, E>`	`Option<T>`	`.ok()`	`Ok(v)` -> `Some(v)`
`Result<T, E>`	`Option<E>`	`.err()`	`Err(e)` -> `Some(e)`
`Option<T>`	`Result<T, E>`	`.ok_or(e)`	`None` -> `Err(e)`
`Option<T>`	`Result<T, E>`	`.ok_or_else()`	`None` -> `Err(f())`

  - Chaining a `Result`-returning function from an `Option`:

- Path A: Discard the error and stay in `Option` (using `.ok()`):

```
let opt: Option<&str> = Some("42");
let val: Option<i32> = opt.and_then(|s| s.parse::<i32>().ok());
```

- Path B: Preserve distinct errors by converting `None` to `Err` first with `.ok_or_else()`, then using `Result::and_then()`.
  - If `opt` was `None`, `.ok_or_else()` produces `Err("missing input")`, and `Result::and_then()` short-circuits immediately--the parse closure is never called!
  - If `opt` was `Some("abc")`, `ok_or_else()` produces `Ok("abc")`, the closure runs, and `.parse()` fails with its own parse error.
  - This gives descriptive errors for both "nothing was provided" and "invalid input was provided":

```
let opt: Option<&str> = Some("42");

let val: Result<i32, String> = opt
    .ok_or_else(|| String::from("missing input"))
    .and_then(|s| s.parse::<i32>().map_err(|e| e.to_string()));
```

9.3 - To `panic!` or Not to `panic!`

- When you choose to return a `Result`, you are giving the calling code options, so `Result` is a good default choice for a function that might fail. In examples, prototype code, and tests, it's more appropriate to panic. Often examples are obfuscated by too much error handling, prototypes can have `expect` and `unwrap` replaced by real error handling later, and panic is an expedient way to mark a test as failed. `expect` is also appropriate in code you know cannot fail and documenting the reason you think you will never get an `Err` variant:

```
use std::net::IpAddr;

let home: IpAddr = "127.0.0.1"
    .parse()
    .expect("Hardcoded IP address should be valid");
```

- If the above IP came from user input and did have the possibility of failure, we would want to handle it robustly. But we can see that the hardcoded value will parse, so we can unwrap

the value.

- Panic anytime your code could end up in a bad state. In this context, a *bad state* is when some assumption, guarantee, contract, or invariant has been broken, such as when invalid values, contradictory values, or missing values are passed in to your code - plus one of the following:
    - The bad state is something that is unexpected as opposed to something that will likely happen occasionally, such as a user entering data in the wrong format.
    - Your code after this point needs to rely on not being in this bad state, rather than checking for the problem at every step.
    - There's not a good way to encode this information in the types you use.
- If someone calls your code with values that don't make sense, it is best to return an error and let the user decide what they want to do, but if continuing could be insecure or harmful, the best choice might be to call `panic!` and alert the user to the bug in their code so it can be fixed immediately. Similarly, `panic!` can be appropriate if you're calling external code that is out of your control and returns an invalid state that you have no way of fixing. However, when failure is expected, return `Result` instead of calling `panic!`.
- Let's imagine for our chapter 2 guessing game, we wanted to implement a `Guess` type that restricts guesses to values between 1 and 100 and otherwise panics (which would be documented in our API docs). Imagine we have a *src/lib.rs* that declares a `mod guessing_game` where we implement this struct in *src/guessing_game.rs*. By being in a module, its `value` field would be private and only set through the public `Guess::new` method and only accessible through the public `value` method. You could pass this around safely knowing you had a value between 1 and 100. Any of our functions relying on this value can take or return a `Guess` without having to validate the value is within range.

```rust
// imaginary /src/guessing_game.rs
pub struct Guess {
    value: i32,
}

impl Guess {
    pub fn new(value: i32) -> Guess {
        if value < 1 || value > 100 {
            panic!("Guess value must be between 1 and 100, got {value}.");
        }

        Guess { value }
    }

    pub fn value(&self) -> i32 {
        self.value
    }
}
```

- (Not in the book) Mapping over containers, tuple structs, and why Rust has no general `Functor` trait:
    - In functional languages like Haskell, `Functor` defines `fmap` over any type constructor of kind `* -> *` (`class Functor f where fmap :: (a -> b) -> f a -> f b`). This allows uniform mapping over `Maybe`, `Either`, `[]`, and custom containers.
    - Rust lacks Higher-Kinded Types (HKTs): generic parameters in Rust only represent concrete types (kind `*`), lifetimes, or constants. You cannot write a trait generic over an unapplied container `F<_>`.
    - Instead, Rust uses an idiomatic naming convention: types like `Option`, `Result`, and `Iterator` each implement their own inherent `.map()` method independently.
    - `Option` and `Result` also provide `.and_then()` (equivalent to monadic bind `>>=`) to chain operations that can fail without nesting wrappers (`Option<Option<T>>`), as well as `.map_err()` on `Result`:

```rust
// Option combinators:
// Note: Option::and_then always requires a function returning an Option!
// Here, Result::ok converts the inner Result<i32, _> into Option<i32>.
let opt: Option<&str> = Some("42");
let doubled: Option<i32> = opt
    .map(|s| s.parse::<i32>())  // 1. Produces Option<Result<i32, _>>
    .and_then(Result::ok)       // 2. Inner Result -> Option<i32>, flattens
    .map(|n| n * 2);            // 3. Doubles inner i32 -> Some(84)

// Non-destructive mapping using as_ref():
let original = Some(String::from("hello"));
let len = original.as_ref().map(|s| s.len()); // Some(5), original intact

// Result combinators:
let res: Result<&str, &str> = Ok("42");
let parsed: Result<i32, &str> = res
    .and_then(|s| s.parse::<i32>().map_err(|_| "parse failed"))
    .map(|n| n + 1);            // Ok(43)
```

- By convention, `map` methods consume `self` by value (`FnOnce(T) -> U`). This allows moving or transforming heap data without cloning. To map over a borrowed value without consuming the original container, borrow it first with `.as_ref()` (as shown above).
- Custom tuple structs and newtypes can implement `.map()` the same way:

```rust
// Newtype for meters
struct Meter(f64);

impl Meter {
    pub fn map<F>(self, f: F) -> Meter
    where
        F: FnOnce(f64) -> f64,
    {
        Meter(f(self.0))
    }
}

// Generic wrapper tuple struct
struct Wrapper<T>(T);

impl<T> Wrapper<T> {
    pub fn map<U, F: FnOnce(T) -> U>(self, f: F) -> Wrapper<U> {
        Wrapper(f(self.0))
    }
}
```

Chapter 10 - Generic Types, Traits, and Lifetimes

- *Generics* are abstract stand-ins for concrete types or other properties. We can express the behavior of generics or how they relate to other generics without knowing what will be in their place when compiling and running code. Generics allow us to replace specific types with a placeholder that represents multiple types to remove code duplication.
- First let's talk about extracting a function to remove duplication. Here are the steps:
    1. Identify duplicate code.
    2. Extract the duplicate code into the body of the function, and specify the inputs and return values of that code in the function signature.
    3. Update the instances of duplicated code to call the function instead.

```
fn largest(list: &[i32]) -> &i32 {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let result = largest(&number_list);
    println!("The largest number is {result}");

    let number_list = vec![102, 34, 6000, 89, 54, 2, 43, 8];

    let result = largest(&number_list);
    println!("The largest number is {result}");
}
```

- Just like a function body can operate on an abstract `list` instead of specific values, generics allow code to operate on abstract types.

10.1 - Generic Data Types

- When defining a function that uses generics, we place the generics in the signature of the function where we would usually specify the data types of the parameters and return value.
- Continuing our `largest` example, imagine you had a function to determine the largest `i32` in a list and another to determine the largest `char`.

```
fn largest_i32(list: &[i32]) -> &i32 {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn largest_char(list: &[char]) -> &char {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let result = largest_i32(&number_list);
    println!("The largest number is {result}");

    let char_list = vec!['y', 'm', 'a', 'q'];

    let result = largest_char(&char_list);
    println!("The largest char is {result}");
}
```

- The functions do the same thing but do it to different types. Introducing a generic type parameter will allow us to eliminate the duplicate and use a single function for both. To parameterize the types we need to name the type parameter. You can use any identifier as a type parameter name but by convention type parameter names are short, often just one letter, and UpperCamelCase. We will simply use `T`, short for type. We have to declare the type parameter in the signature before we use it and we do so within angle brackets, `<>`, between the name of the function and the parameter list: `fn largest<T>(list: &[T]) ->`

`&T {`. We read this definition as "The function `largest` is generic over some type `T`". This function has one parameter named `list`, which is a slice of values of type `T`. The `largest` function will return a reference to a value of the same type `T`. Using it would look something like this:

```
// DOES NOT COMPILE! error[E0369]: binary operation `>` cannot be applied to
//                                 the type `&T`
fn largest<T>(list: &[T]) -> &T {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let result = largest(&number_list);
    println!("The largest number is {result}");

    let char_list = vec!['y', 'm', 'a', 'q'];

    let result = largest(&char_list);
    println!("The largest char is {result}");
}
```

- The issue is that when `largest` takes a slice `&[T]` as input, the function cannot assume *anything* about the type `T`. It could be `i32` or `String`, but it could also be `File`. Can you say one file is greater than another? `largest` requires that `T` is something you can compare with `>` (i.e. that `T` implements `PartialOrd`, a trait we are about to explain). Some types are comparable and others are not. Rust requires you to state the expected capabilities of generic types up front. If `T` needs to be comparable, then `largest` must say so. Also, unlike Java where every object has a set of core methods like `Object.toString()`, there are no core methods in Rust. Without restrictions, a generic type `T` has no capabilities: it cannot be printed, cloned, or mutated (although it can be dropped). We will come back to how to fix this in the next section, but let's see other uses of generics.

- We can define structs to use a generic type parameter in one or more fields by using the `<>` syntax. A `Point<T>` can hold `x` and `y` coordinate values of any type:

```
struct Point<T> {
    x: T,
    y: T,
}

fn main() {
    let integer = Point { x: 5, y: 10 };
    let float = Point { x: 1.0, y: 4.0 };
}
```

- Like a function definition, the `<T>` comes just after the name of the struct, then you use the generic type in the struct definition where we would otherwise specify concrete data type. Because we used only one generic type `T` to define `Point<T>`, this definition says that the struct is generic over some type `T` and that the fields `x` and `y` are both the same type. If you tried to make one `i32` and the other `f64`, you would get a compile error: `error[E0308]: mismatched types`:

```
// DOES NOT COMPILE! error[E0308]: mismatched types
struct Point<T> {
    x: T,
    y: T,
}

fn main() {
    let wont_work = Point { x: 5, y: 4.0 };
}
```

- However, we could allow `x` and `y` to be different types by using multiple generic type parameters:

```
struct Point<T, U> {
    x: T,
    y: U,
}

fn main() {
    let both_integer = Point { x: 5, y: 10 };
    let both_float = Point { x: 1.0, y: 4.0 };
    let integer_and_float = Point { x: 5, y: 4.0 };
}
```

- You can use as many generic type parameters in a definition as you want, but more than a few makes your code hard to read. If you find you need many generic types, your code may need restructuring into smaller pieces.
- We have already seen several enums that use generics, so let's review the definitions of `Option<T>` and `Result<T, E>`. Because `Option<T>` is generic, we can use this abstraction no matter what the type of the optional value is:

```
enum Option<T> {
    Some(T),
    None,
}


enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

- When you recognize situations in your code with multiple struct or enum definitions that differ only in the types of values they hold, you can avoid duplication by using generic types instead.
- You can also use generics in method definitions on structs and enums:

```rust
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

fn main() {
    let p = Point { x: 5, y: 10 };

    println!("p.x = {}", p.x());
}
```

- Note we specify `<T>` right after the `impl`. You could use a different letter than the struct declaration, but it is conventional to use the same name. We can also specify constraints on generic types when defining methods on the type. For example, we could implement a method only on `Point<f32>` instances:

```rust
impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}
```

- You cannot simultaneously implement specific and generic methods of the same name this way. For example, if you had a generic `distance_from_origin` for all types `T` and a specific `distance_from_origin` for `f32` the compiler would reject your program. More generally, Rust does not have inheritance-like mechanisms for specializing methods as you might find in an object-oriented language, with the one exception of default trait methods discussed in the next section.

- Generic type parameters in a struct definition aren't always the same as those you use in that same struct's method signatures. Consider the following with `X1` and `Y1` in the struct definition and `X2` and `Y2` for the `mixup` method signature. The method creates a new `Point` instance with the `x` value from the `self` `Point` (of type `X1`) and the `y` value from the passed-in `Point` (of type `Y2`).

```rust
struct Point<X1, Y1> {
    x: X1,
    y: Y1,
}

impl<X1, Y1> Point<X1, Y1> {
    fn mixup<X2, Y2>(self, other: Point<X2, Y2>) -> Point<X1, Y2> {
        Point {
            x: self.x,
            y: other.y,
        }
    }
}

fn main() {
    let p1 = Point { x: 5, y: 10.4 };
    let p2 = Point { x: "Hello", y: 'c' };

    let p3 = p1.mixup(p2);

    println!("p3.x = {}, p3.y = {}", p3.x, p3.y);
}
```

- In the above code, `p3` will have an `x` value of type `i32` and a `y` value of type `char`, so the output will be `p3.x = 5, p3.y = c`. In this example, `X1` and `Y1` are declared after `impl` because they go with the struct definition. `X2` and `Y2` are declared after `fn mixup` because they're only relevant to the method.

- Generic types will not make your code run slower than concrete types. Rust accomplishes this through monomorphization of the code using generics at compile time. *Monomorphization* is the process of turning generic code into specific code by filling in the concrete types that are used when compiled. An `Option<T>` in the source code becomes a concrete `Option<i32>` or `Option<f64>` (or whatever) at compilation based on its types used, so there is no performance penalty at runtime as if we had duplicated each definition by hand.

10.2 - Defining Shared Behavior with Traits

- A *trait* defines the functionality a particular type has and can share with other types. We can use traits to define shared behavior in an abstract way. We can use *trait bounds* to

specify that a generic type can be any type that has certain behavior. Traits are similar to a feature often called interfaces in other languages, although with some differences.

Defining a Trait

- A type's behavior consists of the methods we can call on that type. Different types share the same behavior if we can call the same methods on all those types. Trait definitions are a way to group method signatures together to define a set of behaviors necessary to accomplish some purpose.
- For example, let's say you have a `NewsArticle` struct that holds a news story filed in a particular location and a `SocialPost` that can have at most 280 characters along with metadata. We want to make a media aggregator library crate named `aggregator` that can display summaries of both. To do this, we need a summary from each type, and we'll request the summary by calling a `summarize` method on an instance. The `Summary` definition below expresses this behavior:

```
pub trait Summary {
    fn summarize(&self) -> String;
}
```

- We declare a trait with the `trait` keyword and then the trait's name which is `Summary` in this case. We also declared it `pub` so that crates depending on this crate can make use of it. Inside the curly brackets, we declare the method signatures that describe the behaviors of the types that implement this trait, which in this case is `fn summarize(&self) -> String`. After the method signature we use a semicolon instead of providing an implementation. Each type implementing this trait must provide its own custom behavior for the body of the method. The compiler will enforce that any type that has the `Summary` trait will have the method `summarize` defined with this signature exactly. A trait can have multiple methods in its body: The method signatures are listed one per line, and each line ends in a semicolon.

Implementing a Trait on a Type

- Below are the implementations of `summarize` for our `NewsArticle` and `SocialPost` structs:

```rust
// imaginary aggregator/src/lib.rs
pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("{}, by {} ({})", self.headline, self.author, self.location)
    }
}

pub struct SocialPost {
    pub username: String,
    pub content: String,
    pub reply: bool,
    pub repost: bool,
}

impl Summary for SocialPost {
    fn summarize(&self) -> String {
        format!("{}: {}", self.username, self.content)
    }
}
```

- Implementing a trait is similar to implementing regular methods. The difference is that after `impl` we put the trait name we want to implement then use the `for` keyword and specify the name of the type we want to implement the trait for. Within the `impl` block, we put the method signatures that the trait definition has defined. Instead of the semicolon after each signature, we fill in the method body with the specific behavior that we want the methods of the trait to have for the particular type.
- Now that the traits are implemented, users of the crate can call the trait methods on instances of `NewsArticle` and `SocialPost` like normal methods, but first they must bring the trait into scope as well as the types.

```
use aggregator::{SocialPost, Summary};

fn main() {
    let post = SocialPost {
        username: String::from("horse_ebooks"),
        content: String::from(
            "of course, as you probably already know, people",
        ),
        reply: false,
        repost: false,
    };

    println!("1 new post: {}", post.summarize());
}
```

- Other crates that depend on `aggregator` can also bring the `Summary` trait into scope to implement `Summary` on their own types. We can implement a trait on a type only if either the trait or the type, or both, are local to our crate. For example, we can implement standard library traits like `Display` on a custom type like `SocialPost` in our `aggregator` crate because `SocialPost` is local to our `aggregator` crate. We can also implement `Summary` on `Vec<T>` in our `aggregator` crate because the trait `Summary` is local to our `aggregator` crate. But we can't implement external traits on external types. For example, we can't implement `Display` on `Vec<T>` because both are defined in the standard library and not local to our `aggregator` crate. This restriction is part of a property called *coherence* and more specifically the *orphan rule*, so named because the parent type is not present. This rule ensures that other people's code can't break your code and vice versa. Without the rule, two crates could implement the same trait for the same type and Rust wouldn't know which implementation to use.

Using Default Implementations

- Sometimes it is useful to have default behavior for some or all the methods in a trait instead of requiring implementations for all methods on every type. We can implement a default method body when declaring the trait, and those implementing the trait can choose to keep or override the default behavior:

```
pub trait Summary {
    fn summarize(&self) -> String {
        String::from("(Read more...)")
    }
}
```

- To use a default implementation for `NewsArticle`, we specify an empty `impl` block with `impl Summary for NewsArticle {}`. The syntax for overriding the default implementation is the same as the syntax for implementing a trait without a default implementation, so we don't have to change `SocialPost`. Default implementations can call other methods in the same trait, even if those other methods don't have a default implementation.

```
pub trait Summary {
    fn summarize_author(&self) -> String;

    fn summarize(&self) -> String {
        format!("(Read more from {}...)", self.summarize_author())
    }
}

impl Summary for SocialPost {
    fn summarize_author(&self) -> String {
        format!("@{}", self.username)
    }
}
```

- We only have to implement `summarize_author` and we get the default implementation of `summarize` for free. Note however you cannot call the default implementation from an overriding implementation in the same method.

Using Traits as Parameters

- Now that we know how to define and implement traits, we can explore how to use traits to define functions that accept many different types. We will continue using the `Summary` trait as an example. Imagine we wanted a `notify` function that takes a type that implements `Summary` and calls its `summarize` function. We would specify this with the `impl Trait` syntax:

```rust
pub fn notify(item: &impl Summary) {
    println!("Breaking news! {}", item.summarize());
}
```

- This parameter will accept any type that implements the specified trait. In the function body, we can use any methods on `item` that come from the `Summary` trait such as `summarize`. We could call it with a `NewsArticle` or `SocialPost`, but it would fail to compile if called with a `String` or `i32` because those types don't implement `Summary`.
- The `impl Trait` syntax is actually syntactic sugar for a longer form known as a *trait bound*:

```rust
pub fn notify<T: Summary>(item: &T) {
    println!("Breaking news! {}", item.summarize());
}
```

- The longer form gives us more control. If we declared two parameters using `impl Summary`, those two parameters could be different types. Where if we declared them `<T: Summary>` then used `T` for both, they would be required to be the same type.
- In either syntax we can specify more than one trait bound using the `+` syntax:

```rust
pub fn notify(item: &(impl Summary + Display)) { /* ... */ }

pub fn notify<T: Summary + Display>(item: &T) { /* ... */ }
```

- With both traits specified, the caller can call `summarize` and use `{}` to format `item`.
- Functions with multiple generic type parameters can contain lots of trait bound information, so Rust has alternate syntax for specifying trait bounds inside a `where` clause after the function signature:

```rust
// Instead of
fn some_function<T: Display + Clone, U: Clone + Debug>(t: &T, u: &U) -> i32 {
    /* ... */
}
// We can say
fn some_function<T, U>(t: &T, u: &U) -> i32
where
    T: Display + Clone,
    U: Clone + Debug,
{
    /* ... */
}
```

Returning Types That Implement Traits

- We can use the `impl Trait` syntax in the return position to return a value of some type that implements a trait without specifying the concrete type:

```rust
fn returns_summarizable() -> impl Summary {
    SocialPost {
        username: String::from("horse_ebooks"),
        content: String::from(
            "of course, as you probably already know, people",
        ),
        reply: false,
        repost: false,
    }
}
```

- The ability to specify a return type only by the trait it implements is especially useful in the context of closures and iterators, which we cover in Chapter 13. Closures and iterators create types that only the compiler knows or types that are very long to specify, but the `impl Trait` syntax allows you to specify that a function returns some type that implements the `Iterator` trait without needing to write out a very long type. However, you can only use `impl Trait` if you're returning a single type. Returning either a `NewsArticle` or a `SocialPost` isn't allowed due to restrictions on how `impl Trait` is implemented in the compiler.

Using Trait Bounds to Conditionally Implement Methods

- By using a trait bound with an `impl` block that uses generic type parameters, we can implement methods conditionally for types that implement the specified traits. In the following example, `Pair<T>` always implements the `new` function to return a new instance of `Pair<T>`, but it only implements `cmp_display` when its inner type `T` implements the `PartialOrd` trait that enables comparison and the `Display` trait that enables printing:

```
use std::fmt::Display;

struct Pair<T> {
    x: T,
    y: T,
}

impl<T> Pair<T> {
    fn new(x: T, y: T) -> Self {
        // Recall `Self` in an `impl` is a type alias for `Pair<T>` here:
        Self { x, y }
    }
}

impl<T: Display + PartialOrd> Pair<T> {
    fn cmp_display(&self) {
        if self.x >= self.y {
            println!("The largest member is x = {}", self.x);
        } else {
            println!("The largest member is y = {}", self.y);
        }
    }
}
```

- We can also conditionally implement a trait for any type that implements another trait. Implementations of a trait on any type that satisfies the trait bounds are called *blanket implementations* and are used extensively by the Rust standard library. For example, the standard library implements the `ToString` trait on any type that implements the `Display` trait. The `impl` block looks similar to:

```
impl<T: Display> ToString for T {
    // --snip--
}
```

- Because of this, we can call the `to_string` method defined in `ToString` on any type that implements the `Display` trait. Because integers implement `Display` we can convert them to `String` values as follows:

```
let s = 3.to_string();
```

- Blanket implementations appear in the documentation for the trait in the "Implementors" section.

- Not covered in the book: implementing `Display` for your own type. It can't be derived (the user-facing format is up to you), but it's short. `fmt` writes into a `Formatter` with `write!` (like `format!`, but into `f`) and returns `fmt::Result`. Thanks to the blanket implementation above, `to_string()` comes for free:

```
use std::fmt;

struct Rectangle {
    width: u32,
    height: u32,
}

impl fmt::Display for Rectangle {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}x{} rectangle", self.width, self.height)
    }
}

fn main() {
    let rect = Rectangle { width: 30, height: 50 };
    println!("{rect}");        // 30x50 rectangle
    let s = rect.to_string();  // "30x50 rectangle", via ToString
}
```

- Traits and trait bounds let us write code that uses generic type parameters to reduce duplication but also specify to the compiler that we want the generic type to have particular behavior. The compiler can then use the trait bound information to check that all the concrete types used with our code provide the correct behavior. In dynamically typed languages, we would get an error at runtime if we called a method on a type that didn't define the method. But Rust moves these errors to compile time so that we're forced to fix the problems before our code is even able to run. Additionally, we don't have to write code

that checks for behavior at runtime, because we've already checked at compile time. Doing so improves performance without having to give up the flexibility of generics.

10.3 - Validating References with Lifetimes

- Lifetimes are another kind of generic that we've already been using. They ensure that references are valid as long as we need them to be. Every reference in Rust has a lifetime, which is the scope for which that reference is valid. Most of the time they are implicit and inferred, just like most of the time types are inferred. Just like we are only required to annotate types when multiple types are possible, we must annotate lifetimes when the lifetimes of references could be related in a few different ways. Rust requires us to annotate the relationships using generic lifetime parameters to ensure that the actual references used at runtime will definitely be valid.

Dangling References

- The main aim of lifetimes is to prevent dangling references which, if they were allowed to exist, would cause a program to reference data other than the data it's intended to reference.

```
// DOES NOT COMPILE! error[E0597]: `x` does not live long enough
fn main() {
    let r;

    {
        let x = 5;
        r = &x;
    }

    println!("r: {}", r);
}
```

- Because `r` has a larger scope than `x`, we say that it "lives longer". If Rust allowed this code to work, `r` would be referencing memory that was deallocated when `x` went out of scope.

The Borrow Checker Ensures Data Outlives Its References

- The Rust compiler has a *borrow checker* that compares scopes to determine whether all borrows are valid. This is the same code with annotations showing the lifetimes of the

variables:

```
// DOES NOT COMPILE! error[E0597]: `x` does not live long enough
fn main() {
    let r;                // ---------+-- 'a
                          //          |
    {                     //          |
        let x = 5;        // -+-- 'b  |
        r = &x;           //  |       |
    }                     // -+       |
                          //          |
    println!("r: {r}");   //          |
}                         // ---------+
```

- `'b` is shorter than `'a`: The subject of the reference doesn't live as long as the reference. So the program is rejected. Here is a fixed version:

```
fn main() {
    let x = 5;            // ----------+-- 'b
                          //           |
    let r = &x;           // --+-- 'a  |
                          //   |       |
    println!("r: {r}");   //   |       |
                          // --+       |
}                         // ----------+
```

- Here `x` still has lifetime `'b`, but `'b` is larger than `'a`.

Generic Lifetimes in Functions

- We'll write a function that returns the longer of two string slices. After it is written, the output of the following program will be `The longest string is abcd`.

```
fn main() {
    let string1 = String::from("abcd");
    let string2 = "xyz";

    let result = longest(string1.as_str(), string2);
    println!("The longest string is {result}");
}
```

- We want `longest` to take string slices, which are references, rather than strings, because we don't want the `longest` function to take ownership of its parameters. The following will not compile:

```
// DOES NOT COMPILE! error[E0106]: missing lifetime specifier
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
```

- The error message tells us we need a generic lifetime parameter on the return type because Rust can't tell whether the reference being returned refers to `x` or `y`. But we actually don't know that either and also don't know the lifetimes of concrete references that will be passed in. To fix this error, we'll add generic lifetime parameters that define the relationship between the references so that the borrow checker can perform analysis.

Lifetime Annotation Syntax

- Lifetime annotations don't change how long references live. They instead describe the relationships of the lifetimes of multiple references to each other without affecting lifetimes. Just like a function can accept any type when the type signature specified a generic type parameter, functions can accept references with any lifetime by specifying a generic lifetime parameter.

- Lifetime annotations start with an apostrophe `'` and are usually all lowercase and very short, like generic types. Most people use `'a` for the first lifetime annotation. We place lifetime annotations after the `&` of a reference, using a space to separate the annotation from the reference's type. Here are some examples:

```
&i32        // a reference
&'a i32     // a reference with an explicit lifetime
&'a mut i32 // a mutable reference with an explicit lifetime
```

- One lifetime annotation by itself doesn't have much meaning, because the annotations are meant to tell Rust how generic lifetime parameters of multiple references relate to each other.

In Function Signatures

- To use lifetime annotations in function signatures, we need to declare the generic lifetime parameters inside angle brackets between the function name and the parameter list, just as we did with generic type parameters. We want the signature to express the following constraint: The returned reference will be valid as long as both parameters are valid:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

- Now the function signature tells Rust that for some lifetime `'a`, the function takes two parameters, both of which are string slices that live at least as long as lifetime `'a`. It also tells Rust that the string slice returned from the function will live at least as long as lifetime `'a`. In practice, this means the lifetime of the reference returned by the `longest` function is the same as the smaller of the lifetimes of the values passed in as arguments. Lifetime annotations go in the function signature, not in the function body, and they become part of the contract of the function. The generic lifetime `'a` will get the concrete lifetime that is equal to the smaller of the lifetimes of `x` and `y`, and the returned reference will be valid for the length of the smaller of the lifetimes of `x` and `y`.

```
// This example works fine, since `result` doesn't outlive either parameter
fn main() {
    let string1 = String::from("long string is long");

    {
        let string2 = String::from("xyz");
        let result = longest(string1.as_str(), string2.as_str());
        println!("The longest string is {result}");
    }
}

// DOES NOT COMPILE! error[E0597]: `string2` does not live long enough
fn main() {
    let string1 = String::from("long string is long");
    let result;
    {
        let string2 = String::from("xyz");
        result = longest(string1.as_str(), string2.as_str());
    }
    println!("The longest string is {result}");
}
```

Relationships

- If `longest` always returned the first parameter, we would only need to specify the lifetime of the first parameter and return type. When returning a reference from a function, the lifetime parameter for the return type must match the lifetime parameter for one of the parameters. If the reference does not refer to one of the parameters, it must refer to a value created within the function, but this would be a dangling reference because the value would go out of scope at the end of the function. It doesn't work to specify a lifetime parameter only for the return value when it relates to neither parameter:

```
// DOES NOT COMPILE! error[E0515]: cannot return value referencing local
//                                 variable `result`
fn longest<'a>(x: &str, y: &str) -> &'a str {
    let result = String::from("really long string");
    result.as_str()
}
```

- We're trying to return a reference to a local variable that is cleaned up at the end of the function, and there is no way to specify lifetime parameters to make this legal, so Rust

won't allow it. In this case, the best fix would be to return an owned data type rather than a reference.

- Ultimately, lifetime syntax is about connecting the lifetimes of various parameters and return values of functions. Once they're connected, Rust has enough information to allow memory-safe operations and disallow operations that would create dangling pointers or otherwise violate memory safety.

In Struct Definitions

- So far, the structs we've defined all hold owned types. We can define structs to hold references, but in that case we would need to add a lifetime annotation on every reference in the struct's definition:

```
struct ImportantExcerpt<'a> {
    part: &'a str,
}

fn main() {
    let novel = String::from("Call me Ishmael. Some years ago...");
    let first_sentence = novel.split('.').next().unwrap();
    let i = ImportantExcerpt {
        part: first_sentence,
    };
}
```

- This struct has a single field `part` that holds a string slice which is a reference. An instance of `ImportantExcerpt` cannot outlive the reference it holds in its `part` field. Here `novel` exists before `i` and doesn't go out of scope until after `i` does, so this is valid.

Lifetime Elision

- The *lifetime elision rules* are particular patterns that the compiler is programmed to recognize, so you can leave out lifetime annotations. Lifetimes on function or method parameters are called *input lifetimes*, and lifetimes on return values are called *output lifetimes*. The compiler follows three rules to figure out lifetimes of references when there aren't explicit annotations. The first rule applies to input lifetimes, and the second and third rules apply to output lifetimes. If the compiler gets to the end of the three rules and there are still references for which it can't figure out lifetimes, the compiler will stop with an error. These rules apply to `fn` definitions as well as `impl` blocks.

- The first rule is that the compiler assigns a different lifetime parameter to each lifetime in each input type. For example:
  - The function `fn foo(x: &i32)` would get one lifetime parameter and become `fn foo<'a>(x: &'a i32)`.
  - The function `fn foo(x: &i32, y: &i32)` would get two lifetime parameters and become `fn foo<'a, 'b>(x: &'a i32, y: &'b i32)`.
  - The function `fn foo(x: &ImportantExcerpt)` would get two lifetime parameters and become `fn foo<'a, 'b>(x: &'a ImportantExcerpt<'b>)`.
- The second rule is that if there is exactly one input lifetime parameter, that lifetime is assigned to all output lifetime parameters: `fn foo<'a>(x: &'a i32) -> &'a i32`.
- The third rule is that if there are multiple input lifetime parameters but one of them is `&self` or `&mut self` because this is a method, the lifetime of `self` is assigned to all output lifetime parameters. The third rule makes methods much nicer to read and write because fewer symbols are necessary.
- Consider the `first_word` function from Chapter 4. It did not need annotations because of these rules. The signature starts without any lifetimes: `fn first_word(s: &str) -> &str {`. Then the compiler applies the first rule which specifies that each parameter gets its own lifetime: `fn first_word<'a>(s: &'a str) -> &str {`. The second rule applies because there is exactly one input lifetime, so the lifetime of the input parameter is assigned to the output parameter: `fn first_word<'a>(s: &'a str) -> &'a str {`. Now all the references in the signature have lifetimes so the compiler can continue analysis without needing programmer annotations.
- Now consider the `longest` function before we added lifetime parameters. `fn longest(x: &str, y: &str) -> &str {`. The first rule is every input parameter gets its own lifetime: `fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &str {`. The second rule does not apply since there is more than one input lifetime. The third rule doesn't apply because this is a function rather than a method so there is no `self`. We have worked through all three rules without figuring out the return type's lifetime, so we get a compiler error.

In Method Definitions

- The third rule means we don't need to annotate method signatures often. When we implement methods on a struct with lifetimes, we use the same syntax as that of generic type parameters. Where we declare and use the lifetime parameters depends on whether they are related to the struct fields or the method parameters and return values. Lifetime names for struct fields always need to be declared after the `impl` keyword and then used after the struct's name because those lifetimes are part of the struct's type. In method

signatures inside the `impl` block, references might be tied to the lifetime of references in the struct's fields or they might be independent.

- Consider this method `level` whose only parameter is a reference to `self` and whose return value is an `i32` defined on `ImportantExcerpt` from the earlier example:

```
impl<'a> ImportantExcerpt<'a> {
    fn level(&self) -> i32 {
        3
    }
}
```

- The lifetime parameter after `impl` and after the struct name are required, but because of the first elision rule, we're not required to annotate the lifetime of the reference to `self`.

- Here is an example where the third lifetime elision rule applies:

```
impl<'a> ImportantExcerpt<'a> {
    fn announce_and_return_part(&self, announcement: &str) -> &str {
        println!("Attention please: {announcement}");
        self.part
    }
}
```

- The first rule applies and gives both `&self` and `announcement` their own lifetimes. Then because one of the parameters is `&self`, the return type gets the lifetime of `&self` and all lifetimes have been accounted for.

The Static Lifetime

- `'static` is a special lifetime that denotes the affected reference *can* live for the entire duration of the program. All string literals have the `'static` lifetime. Here is an annotated string:

```
let s: &'static str = "I have a static lifetime.";
```

- The text of this string is stored directly in the program's binary, which is always available. Therefore, the lifetime of all string literals is `'static`. An error message might suggest using `'static` but it is seldom the answer.

Generic Type Parameters, Trait Bounds, and Lifetimes

- Here is an example that combines specifying generic type parameters, trait bounds, and lifetimes all in one function:

```
use std::fmt::Display;

fn longest_with_an_announcement<'a, T>(
    x: &'a str,
    y: &'a str,
    ann: T,
) -> &'a str
where
    T: Display,
{
    println!("Announcement! {ann}");
    if x.len() > y.len() { x } else { y }
}
```

- This is the `longest` function from earlier, but it also takes an `ann` parameter of any type that implements the `Display` trait. Because lifetimes are a type of generic, both `'a` and `T` go in the angle brackets after the function name.

10.4 - Ownership Inventory #3

- *Note: Ownership Inventory #3 has been moved to the Appendix at the end of these notes for high-yield review.*

Chapter 11 - Writing Automated Tests

- Tests are Rust functions that verify that the non-test code is functioning in the expected manner. The bodies of test functions typically perform these three actions:
  - Set up any needed data or state.
  - Run the code you want to test.
  - Assert that the results are what you expect.

11.1 - How to Write Tests

- At its simplest, a test in Rust is a function that's annotated with the `test` attribute. Attributes are metadata about pieces of Rust code; one example is the `derive` attribute

we used with structs in Chapter 5. To change a function into a test function, add `#[test]` on the line before `fn`. When you run your tests with `cargo test`, Rust builds a test runner binary that runs the annotated function and reports on whether each test function passes or fails.

- When you make a new library project with Cargo (`cargo new library-name --lib`), a test module with a test function in it is automatically generated.
- `cargo test` outputs a line per test, but `cargo test -q` outputs a `.` per test unless there's a failure.
- One way to fail a test is to call `panic!`. Another is to give the `assert!` macro an argument that evaluates `false`. You could also compare two unequal values with `assert_eq!` or two equal values with `assert_ne!`.
- Here's a basic library with a couple of tests for some code we've seen:

```rust
// ch11_adder/src/lib.rs
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}

pub fn add_two(a: u64) -> u64 {
    a + 2
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn larger_can_hold_smaller() {
        let larger = Rectangle {
            width: 8,
            height: 7,
        };
        let smaller = Rectangle {
            width: 5,
            height: 1,
        };

        assert!(larger.can_hold(&smaller));
    }

    #[test]
    fn smaller_cannot_hold_larger() {
        let larger = Rectangle {
            width: 8,
            height: 7,
        };
        let smaller = Rectangle {
            width: 5,
            height: 1,
        };
```

```
        assert!(!smaller.can_hold(&larger));
    }

    #[test]
    fn it_adds_two() {
        let result = add_two(2);
        assert_eq!(result, 4);
    }
}
```

- The tests are in their own `tests` module but they can see everything in their parent module. They `use super::*` so they can directly call anything in their parent. These tests pass.
- The `assert_eq!` macro calls its arguments `left` and `right` and doesn't care which is the expected vs actual value. When `assert_eq!` or `assert_ne!` fail, they print their arguments using debug formatting. Also, they do their comparison with `==` and `!=` so the values being compared need to implement the `Debug` and `PartialEq` traits. Both traits are derivable, so usually you only need to add `#[derive(PartialEq, Debug)]` annotation to your struct or enum definition. Any arguments after the value (for `assert!`) or two values (for `_eq!` and `_ne!`) being tested are passed to the `format!` macro for display. You can pass a format string that contains `{}` placeholders and values to go in those placeholders. Custom messages are useful for documenting what an assertion means; when a test fails, you'll have a better idea of what the problem is with the code.

```rust
pub fn greeting(name: &str) -> String {
    format!("Hello {name}!")
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn greeting_contains_name() {
        let result = greeting("Carol");
        assert!(
            result.contains("Carol"),
            "Greeting did not contain name, value was `{result}`"
        );
    }
}
```

- Now when `greeting_contains_name` fails, we get an informational message that includes the result without using `assert_eq!` to test exact equality. You can refer to any variables in scope, and you can also provide format strings for `assert_eq!` and `assert_ne!`.
- The `#[track_caller]` attribute works like Go's `t.Helper()` for test helper functions: a failing assertion inside the helper is reported at the line in the test that called it, not inside the helper.

```rust
#[track_caller]
fn assert_even(n: i32) {
    assert!(n % 2 == 0, "{n} is not even");
}

#[test]
fn three_is_even() {
    assert_even(3); // the failure points at this line
}
```

- Recall our `Guess` struct that takes a guess between 1 and 100 and panics otherwise. This is our contract, and it would be good to test. We can test it with `should_panic`. We place `#[should_panic]` after the `#[test]` attribute and before the test function it applies to.

```rust
pub struct Guess {
    value: i32,
}

impl Guess {
    pub fn new(value: i32) -> Guess {
        if value < 1 || value > 100 {
            panic!("Guess value must be between 1 and 100, got {value}.");
        }

        Guess { value }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    #[should_panic]
    fn greater_than_100() {
        Guess::new(200);
    }
}
```

- Tests relying on `#[should_panic]` can be imprecise and pass even when they panic for an unexpected reason. `should_panic` takes an `expected` argument that will check the panic message for an expected substring. It can further check your expectations, but couples you to a specific `panic` message.

```rust
#[test]
#[should_panic(expected = "between 1 and 100")]
fn greater_than_100() {
    Guess::new(200);
}
```

- Instead of panicking, tests can optionally return a `Result<(), String>`. This enables you to use the question mark operator in the body of tests, which can be a convenient way to write tests that should fail if any operation within them returns an `Err` variant. You cannot use `#[should_panic]` with a test that returns a `Result`. To check that an operation returns an `Err` variant, you should use `assert!(value.is_err())`.

```rust
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() -> Result<(), String> {
        let result = add_two(2);

        if result == 4 {
            Ok(())
        } else {
            Err(String::from("two plus two does not equal four"))
        }
    }
}
```

- Here is an example of a test calling `Result` returning functions and passing the `Err` upwards:

```rust
// without Result: unwrap on every fallible step
#[test]
fn loads_config() {
    let text = fs::read_to_string("tests/data/config.toml").unwrap();
    let config = Config::parse(&text).unwrap();
    assert_eq!(config.port, 8080);
}

// with Result: ? passes any failure up, which fails the test
#[test]
fn loads_config() -> Result<(), Box<dyn Error>> {
    let text = fs::read_to_string("tests/data/config.toml")?;
    let config = Config::parse(&text)?;
    assert_eq!(config.port, 8080);
    Ok(())
}
```

- This has two benefits:
    - Setup reads like normal code. Integration tests often read files, parse input or connect to things, and `?` keeps that setup short.
    - Failures show the error itself. A failing test prints the returned error (for example, the parse error), while an `unwrap()` panic shows mostly the panic location.

11.2 - Controlling How Tests Are Run

- The default behavior of `cargo test` is to run all your tests in parallel and capture output generated during test runs to make it easier to read test output. All tests in a test binary are run by default and all test failures are reported at the end. `cargo test -- --fail-fast` stops starting new tests after the first failure. Some arguments go to `cargo test` and some go to the resultant test binary. You separate options to each by `--`. `cargo test --help` displays the options you can use with `cargo test`, and `cargo test -- --help` displays the options you can use after the separator. Options are documented in the Tests section of the rustc book: https://doc.rust-lang.org/rustc/tests/index.html.

- If you don't want to run the tests in parallel, you run: `cargo test -- --test-threads=1`. It is slower, but the tests won't interfere with one another if they share state.

- Rust captures output of tests and only shows it on test failure. So if the test passes, you only see the success message and no `println!` output, but if it fails you see both the failure message and captured output. If you want to see printed output for passing tests run: `cargo test -- --show-output`.

- Running `cargo test` will run all tests. Passing a string after `cargo test`, like `cargo test greeting` will run all tests that contain the substring "greeting" in their function name or module name. Specifying `cargo test module_name` is a good way to run all the tests in a module.

- Not in the book: Cargo can also select tests by *target*, which is better for running fast unit tests on every change and slower integration tests before committing:

Command	Runs
`cargo test --lib`	Unit tests in the library (*src/lib.rs* tree)
`cargo test --bins`	Unit tests in *src/main.rs* and *src/bin*
`cargo test --test api`	Only the integration test file *tests/api.rs*
`cargo test --test '*'`	All integration tests
`cargo test --doc`	Only doc tests
`cargo test`	Everything

Targets and name filters combine: `cargo test --lib parser::` runs just the parser module's unit tests. For a TDD loop, `bacon test -- --lib`. Avoid `--tests` (plural) for this: it runs unit and integration tests together.

- The `#[ignore]` attribute marks a test to not be run unless you call either `cargo test -- --ignored` to run only the ignored tests or `cargo test -- --include-ignored` to run the normal test suite along with the ignored tests.

11.3 - Test Organization

- *Unit tests* are small, focused tests that test one module in isolation at a time and can test private interfaces. *Integration tests* are entirely external to your library and use your code in the same way any other external code would, using only the public interface and potentially exercising multiple modules per test. Both types of test are important to ensure the pieces of your library do what you expect them to, separately and together.

- You put unit tests in the *src* directory in each file with the code that they're testing. The convention is to create a module named `tests` in each file to contain the test functions and to annotate the module with `#[cfg(test)]`. Because of the annotation, unit tests are not compiled unless you run `cargo test`. Integration tests don't need this annotation because they are in a different directory. The annotation applies to all code in the `tests` module, including test helpers. If you make a separate test helper module, annotate it `#[cfg(test)]` as well (see Testing Larger Projects below).

- Rust's test conventions allow you to test private functions. It is up to you if that is a good idea.

- In Rust, integration tests are entirely external to your library, and they use your library through the public API, like any other code would. You put integration tests in the *tests* directory. Each file in the *tests* directory is a separate crate, so we need to bring our library into each test crate's scope with `use`. Cargo automatically only compiles code in the *tests* directory when you run `cargo test`, so `#[cfg(test)]` is unnecessary.

- In the test output, unit tests then integration tests and finally doc tests are reported on separately. If any test in a section fails, the following sections will not be run (`cargo test --no-fail-fast` runs them anyway). Each integration test has its own section, so if you add more files to the *tests* directory, there will be more integration test sections. To only run the tests in one particular integration test, you can run `cargo test --test integration_test` for a test named `integration_test`.

```rust
// ch11_adder/tests/integration_test.rs
use adder::add_two;

#[test]
fn it_adds_two() {
    let result = add_two(2);
    assert_eq!(result, 4);
}
```

- It is likely you will need a place for shared integration test code such as a shared `setup` function. If you place test helpers in *tests/common/mod.rs*, it will not show up as a separate integration test. The name tells Rust to not treat it as an integration test file. Here is our same integration test using a shared helper:

```rust
// ch11_adder/tests/common/mod.rs
pub fn setup() {
    // setup code specific to your library's tests would go here
}

// ch11_adder/tests/integration_test.rs
use adder::add_two;

mod common;

#[test]
fn it_adds_two() {
    common::setup();

    let result = add_two(2);
    assert_eq!(result, 4);
}
```

- Only library crates expose functions that other crates can `use`; binary crates are meant to be run on their own. So integration tests have no way to bring functions defined in `src/main.rs` into scope. This is why we keep the code in our `src/main.rs` straightforward and call logic that lives in *src/lib.rs*. Using that structure, integration tests can test the library crate with `use`. If the important functionality works, the small amount of code in the *src/main.rs* file will work as well.

- Testing larger projects (not in the book): each module gets its own `#[cfg(test)] mod tests` child, because a test module only sees the private items of its own parent;

`crate::tests` can't see inside sibling modules like `crate::parser`. To keep tests out of the source file (like Go's *parser_test.go* beside *parser.go*), end the module with `#[cfg(test)] mod tests;` and put the tests in *src/parser/tests.rs*; it is still a child module with private access.

- `pub(crate)` makes an item visible anywhere in the crate but not to other crates. Use it for code shared between modules, and for shared unit-test helpers: `#[cfg(test)] pub(crate) mod test_utils;` (no standard name; `test_utils` is the most common). Integration tests can't see `test_utils`, since they are separate crates; they share helpers through *tests/common/mod.rs* instead.
- Besides one integration test binary per `tests/*.rs` file, Cargo also builds `tests/<name>/main.rs` as a single binary that can pull in sibling files as modules. Linking fewer binaries is faster, so big projects often group their integration tests this way, named after what they test (or *tests/it/* for one all-in-one binary). Other subdirectories are only modules. Each workspace member has its own *tests/*.

```
src/
|-- lib.rs            // #[cfg(test)] pub(crate) mod test_utils;
|-- test_utils.rs     // shared unit-test helpers
|-- parser.rs         // ends with: #[cfg(test)] mod tests;
|-- parser/
|   `-- tests.rs      // crate::parser::tests (sees parser's private items)
|-- lexer.rs          // ends with: #[cfg(test)] mod tests;
`-- lexer/
    `-- tests.rs      // crate::lexer::tests
tests/
|-- api.rs            // one integration test binary
|-- common/
|   `-- mod.rs        // shared integration helpers, not a test binary
`-- parser/
    |-- main.rs       // one binary for all parser integration tests
    `-- errors.rs     // pulled in with `mod errors;`
```

- Even though Rust's type system and ownership rules help prevent some kinds of bugs, tests are still important to reduce logic bugs having to do with how your code is expected to behave.

Beyond the Book

- Naming tests: test names are function names, so no spaces, but nested modules give RSpec-style `describe` / `context` / `it` names joined by `::`. Each module needs its own `use super::*;`.

```
#[cfg(test)]
mod mode {                          // describe "mode"
    use super::*;

    mod when_the_list_is_empty {    // context "when the list is empty"
        use super::*;

        #[test]
        fn returns_none() {         // it "returns none"
            assert_eq!(mode(&[]), None);
        }
    }
}
// cargo test prints: mode::when_the_list_is_empty::returns_none
```

- rstest - pytest-style fixtures and parametrized tests. `#[case(...)]` runs one test per row (like Go's table-driven tests or Spock's `where:` blocks), `#[values(...)]` runs every combination, and `#[fixture]` functions inject shared setup. The docs have good examples of all three.

```rust
use rstest::rstest;

#[rstest]
#[case::rectangle(Shape::Rectangle { width: 12.0, height: 6.0 }, 72.0)]
#[case::triangle(Shape::Triangle { base: 12.0, height: 6.0 }, 36.0)]
fn area(#[case] shape: Shape, #[case] want: f64) {
    // floats: compare within a tolerance, not with assert_eq!
    let got = shape.area();
    assert!((got - want).abs() < 1e-9, "got {got}, want {want}");
}
// names: area::case_1_rectangle, area::case_2_triangle

#[rstest]
fn scaling_keeps_area_positive(
    #[values(1.0, 2.5, 10.0)] width: f64,
    #[values(1.0, 0.5)] scale: f64,   // 3 x 2 = 6 tests
) {
    let rect = Shape::Rectangle { width: width * scale, height: 4.0 };
    assert!(rect.area() > 0.0);
}
```

- test-case - like `#[case]`, but each case gets a plain-English description that becomes the test name. `=>` gives the expected return value (an implicit `assert_eq!`).

```rust
use test_case::test_case;

#[test_case(&[] => None ; "returns none for an empty list")]
#[test_case(&[3, 1, 2] => Some(2.0) ; "returns the middle of an odd list")]
fn median_works(v: &[i32]) -> Option<f64> {
    median(v)
}
```

- pretty_assertions - drop-in `assert_eq!` with colored diffs; add `use pretty_assertions::assert_eq;` to the test module.
- approx - `assert_relative_eq!` and `assert_abs_diff_eq!` for float comparisons.
- proptest - property-based testing, the Rust take on QuickCheck/Hypothesis.
- Benchmarks: Rust's built-in `#[bench]` has been unstable for years and only works on the nightly compiler. On stable Rust, use the `criterion` crate (the long-standing standard,

modeled on Haskell's criterion) or `divan` (newer and simpler). Go's built-in `testing.B` is more convenient here.

Chapter 12 - An I/O Project: Building a Command Line Program

- Project time! We are going to build `grep` (globally search a regular expression and print). In the simplest case `grep` searches a specified file for a specified string. To do so, `grep` takes as its arguments a file path and a string. Then, it reads the file, finds lines in that file that contain the string argument, and prints those lines.

12.1 - Accepting Command Line Arguments

- Our first task after creating the new project is to make `minigrep` accept its two command line arguments: the file path and a string to search for. We will pass these arguments by calling `cargo run` with `--`: `cargo run -- searchstring example-filename.txt`. There are many good command-line argument parsing libraries, but we are going to do it ourselves.

- To enable `minigrep` to read the values of command line arguments we pass it, we will use the `std::env::args` function provided by Rust's standard library. It returns an iterator of the command line arguments passed to `minigrep`. Iterators will be covered in Chapter 13, but it is enough to know that iterators produce a series of values, and we can call the `collect` method on an iterator to turn it into a collection, such as a vector, which contains all the elements the iterator produces.

```rust
// ch12_minigrep/src/main.rs
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();
    dbg!(args);
}
```

- `collect` can return many varieties of collection. Although you don't have to annotate types in Rust often, `collect` is one function you do often need to annotate because Rust isn't able to infer the kind of collection you want.

```
$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `target/debug/minigrep`
[ch12_minigrep/src/main.rs:5:5] args = [
    "target/debug/minigrep",
]

$ cargo run needle haystack
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `target/debug/minigrep needle haystack`
[ch12_minigrep/src/main.rs:5:5] args = [
    "target/debug/minigrep",
    "needle",
    "haystack",
]
```

- So let's pull out the first two args where we can use them:

```
// ch12_minigrep/src/main.rs
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();

    let query = &args[1];
    let file_path = &args[2];

    println!("Searching for {query}");
    println!("In file {file_path}");
}
```

- We know array indexing starts at 0, but `args[0]` is the filename of the program we're running.

12.2 - Reading a File

- Next we actually need to read a file, so we'll create a `poem.txt` in the root of our project.

```rust
// ch12_minigrep/src/main.rs
use std::env;
use std::fs;

fn main() {
    // --snip--
    println!("In file {file_path}");

    let contents = fs::read_to_string(file_path)
        .expect("Should have been able to read the file");

    println!("With text:\n{contents}");
}
```

- Now if we `cargo run -- the poem.txt`, we get the contents of our text file.

12.3 - Refactoring to Improve Modularity and Error Handling

- To improve our program, we'll fix four problems that have to do with the program's structure and how it's handling potential errors. First, our `main` function now performs two tasks: it parses arguments and reads files. As our program grows, the number of separate tasks the `main` function handles will increase. As the function gains responsibilities, it becomes more difficult to reason about, harder to test, and harder to change without breaking one of its parts. It's best to separate functionality so that each function is responsible for one task.

- This issue also ties into the second problem: Although `query` and `file_path` are configuration variables to our program, variables like `contents` are used to perform the program's logic. The longer `main` becomes, the more variables we'll need to bring into scope; the more variables we have in scope, the harder it will be to keep track of the purpose of each. It's best to group the configuration variables into one structure to make their purpose clear.

- The third problem is that we've used `expect` to print an error message when reading the file fails, but the error message just prints `Should have been able to read the file.` Reading a file can fail in a number of ways: For example, the file could be missing, or we might not have permission to open it. Right now, regardless of the situation, we'd print the same error message for everything, which wouldn't give the user any information.

- Fourth, we use `expect` to handle an error, and if the user runs our program without specifying enough arguments, they'll get an `index out of bounds` error from Rust that doesn't clearly explain the problem. It would be best if all the error-handling code were in one place so that future maintainers had only one place to consult the code if the error-handling logic needed to change. Having all the error-handling code in one place will also ensure that we're printing messages that will be meaningful to our end users.

- The organization problem of allocating responsibility for multiple tasks to the `main` function is common to many binary projects. As a result, many Rust programmers find it useful to split up the separate concerns of a binary program when the `main` function starts getting large. This process has the following steps:

    - Split your program into a *main.rs* file and a *lib.rs* file and move your program's logic to *lib.rs*.
    - As long as your command line parsing logic is small, it can remain in the `main` function.
    - When the command line parsing logic starts getting complicated, extract it from the `main` function into other functions or types.
- The responsibilities that remain in the `main` function after refactoring should be limited to the following:

    - Calling the command line parsing logic with the argument values
    - Setting up any other configuration
    - Calling a `run` function (the book says in *lib.rs*, but in the current version `run` stays in *main.rs* and only `search` moves to *lib.rs*)
    - Handling the error if `run` returns an error
- This pattern is about separating concerns: *main.rs* handles running the program and *lib.rs* handles all the logic of the task at hand. Because you cannot test the `main` function directly, this structure lets you test all of your program's logic by moving it out of the `main` function. The code that remains in the `main` function will be small enough to verify by inspection.

- First we need a `parse_config` function to handle arguments:

```rust
// ch12_minigrep/src/main.rs
fn main() {
    let args: Vec<String> = env::args().collect();

    let (query, file_path) = parse_config(&args);

    // --snip--
}

fn parse_config(args: &[String]) -> (&str, &str) {
    let query = &args[1];
    let file_path = &args[2];

    (query, file_path)
}
```

- At the moment, we're returning a tuple, but then we immediately break that tuple down into individual parts again. This is a sign that perhaps we don't have the right abstraction yet. We'll instead put the two values in one struct and give each of the struct fields a meaningful name.

```rust
// ch12_minigrep/src/main.rs
use std::env;
use std::fs;

fn main() {
    let args: Vec<String> = env::args().collect();

    let config = parse_config(&args);

    println!("Searching for {}", config.query);
    println!("In file {}", config.file_path);

    let contents = fs::read_to_string(config.file_path)
        .expect("Should have been able to read the file");

    println!("With text:\n{contents}");
}

struct Config {
    query: String,
    file_path: String,
}

fn parse_config(args: &[String]) -> Config {
    let query = args[1].clone();
    let file_path = args[2].clone();

    Config { query, file_path }
}
```

- Now we bundle our configuration parameters in a `Config` struct. We transfer ownership of our new struct when we return. We also clone our two `arg` values so our struct can contain owned `String` values. It would've been a violation of the borrowing rules if `Config` tried to take ownership of the values in `args` directly. Cloning isn't the most efficient solution, but it makes our code very straightforward because we don't have to manage the lifetimes of references.

- The primary point of `parse_config` is to create a struct now, so it's a constructor. We can make our code more idiomatic by naming it `new` and associating it with the `Config` struct.

```rust
// ch12_minigrep/src/main.rs
fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::new(&args);

    // --snip--
}

// --snip--

impl Config {
    fn new(args: &[String]) -> Config {
        let query = args[1].clone();
        let file_path = args[2].clone();

        Config { query, file_path }
    }
}
```

- We don't have tests yet, but we're refactoring in small steps while keeping working code and running it between changes. Next up is fixing the panic when not enough arguments are provided. How about we `panic!` with a better message:

```rust
// ch12_minigrep/src/main.rs
// --snip--
    fn new(args: &[String]) -> Config {
        if args.len() < 3 {
            panic!("not enough arguments");
        }
        // --snip--
```

- Now the error message is better, but it's still a panic with line numbers of our code included. Let's try using `Result`. We're also going to change the name of our constructor from `new` to `build`, because many programmers expect `new` functions to never fail.

```rust
// ch12_minigrep/src/main.rs
use std::env;
use std::fs;
use std::process;

fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::build(&args).unwrap_or_else(|err| {
        println!("Problem parsing arguments: {err}");
        process::exit(1);
    });
    // --snip--
}

impl Config {
    fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("not enough arguments");
        }
        let query = args[1].clone();
        let file_path = args[2].clone();

        Ok(Config { query, file_path })
    }
}
```

- We are exiting with a nonzero status since it is the convention to signal to the process that called our program that the program exited with an error state.
- Now we want to extract a function named `run` that will hold all the logic currently in `main` that isn't involved with setting up configuration or handling errors. It stays in *main.rs*; despite the book's earlier description, only the search logic ends up in *lib.rs*.

```rust
// ch12_minigrep/src/main.rs
fn main() {
    // --snip--

    println!("Searching for {}", config.query);
    println!("In file {}", config.file_path);

    run(config);
}

fn run(config: Config) {
    let contents = fs::read_to_string(config.file_path)
        .expect("Should have been able to read the file");

    println!("With text:\n{contents}");
}
// --snip--
```

- Ok, so the only logic left to move was reading the file. But we want to again improve error handling. Our `run` could return a `Result` too and let `main` sort out errors.

```rust
// ch12_minigrep/src/main.rs
use std::error::Error;

// --snip--

fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let contents = fs::read_to_string(config.file_path)?;

    println!("With text:\n{contents}");

    Ok(())
}
```

- For the return type, we left the success type of `run` as `()` but wrapped it in an `Ok`. Returning `()` or when using `Result` returning `Ok(())` is the idiomatic way to indicate we are calling `run` for its side effects only. For the error type, we used the trait object `Box<dyn Error>` and brought `std::error::Error` into scope. Trait objects will be covered in Chapter 18, but for now just know that `Box<dyn Error>` means the function will return a type that implements the `Error` trait, but we don't have to specify the particular type. This

gives us flexibility to return error values of different types for different error cases. This is a lot cleaner than the panic caused by `expect`.

- If we run this now, we get a warning that we are ignoring the resulting value and that it might indicate an error occurred. It's so helpful:

```
// ch12_minigrep/src/main.rs
fn main() {
    // --snip--

    println!("Searching for {}", config.query);
    println!("In file {}", config.file_path);

    if let Err(e) = run(config) {
        println!("Application error: {e}");
        process::exit(1);
    }
}
```

- In this case, `run` doesn't return a value we want to unwrap, so we'll just check for the `Err`. Now it's time to start our library code:

```
// ch12_minigrep/src/lib.rs
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    unimplemented!();
}

// ch12_minigrep/src/main.rs
use minigrep::search;
// --snip--
fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let contents = fs::read_to_string(config.file_path)?;

    for line in search(&config.query, &contents) {
        println!("{line}");
    }

    Ok(())
}
```

- The `unimplemented!` macro is just a placeholder that panics. It's also a good time to remove the two `println!` calls in `main` that print our arguments. Now we're moving on

with more module code that handles errors well, and the work that is left is mostly in *src/lib.rs*.

12.4 - Adding Functionality with Test-Driven Development

- We're working in the *src/lib.rs* and we're working test first.

```
// ch12_minigrep/src/lib.rs
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    unimplemented!();
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn one_result() {
        let query = "duct";
        let contents = "\
Rust:
safe, fast, productive.
Pick three.";

        assert_eq!(vec!["safe, fast, productive."], search(query, contents));
    }
}
```

- First, we make our `search` function return an empty `vec!` so everything will compile and we can see our assertion fail:

```
// ch12_minigrep/src/lib.rs
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    vec![]
}
```

- It's worth mentioning our lifetime annotations. We're simply making it clear to the compiler that the string slices we return in the `Vec` refer to `contents` not `query` and will be valid as long as `contents` is valid. If we failed to include the annotation, the compiler would helpfully suggest we annotate all the arguments and return value with the same lifetime, but

we know that our references will be to `contents`, not `query`, so `query` doesn't need a lifetime.

- So now we need `search` to do something for real to make the test pass:

```
// ch12_minigrep/src/lib.rs
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    let mut results = Vec::new();

    for line in contents.lines() {
        if line.contains(query) {
            results.push(line);
        }
    }

    results
}
```

- And our tests are green. We could use this opportunity to refactor, but `search` doesn't look half bad. There are some useful iterator features we aren't using, so we will revisit this project in Chapter 13. So basically by passing one test, we've implemented the whole program! Try it at the command line, it greps text just fine.

12.5 - Working with Environment Variables

- We will add a feature to turn on case insensitive searching by setting an environment variable. We could make it a command line option, but with an environment variable it can be set once for that terminal session.

```rust
// ch12_minigrep/src/lib.rs
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn case_sensitive() {
        let query = "duct";
        let contents = "\
Rust:
safe, fast, productive.
Pick three.
Duct tape.";

        assert_eq!(vec!["safe, fast, productive."], search(query, contents));
    }

    #[test]
    fn case_insensitive() {
        let query = "rUsT";

        let contents = "\
Rust:
safe, fast, productive.
Pick three.
Trust me.";

        assert_eq!(
            vec!["Rust:", "Trust me."],
            search_case_insensitive(query, contents)
        );
    }
}
```

- Note that we modified our first test case by renaming it and adding a line to `contents` that says "Duct tape." and shouldn't match "duct" in all lowercase. So we need a new function. First we make it a carbon copy of `search` to make sure our new test fails with a good error. Then we modify it to lowercase both its `query` and `contents`:

```rust
// ch12_minigrep/src/lib.rs
pub fn search_case_insensitive<'a>(
    query: &str,
    contents: &'a str,
) -> Vec<&'a str> {
    let query = query.to_lowercase();
    let mut results = Vec::new();

    for line in contents.lines() {
        if line.to_lowercase().contains(&query) {
            results.push(line);
        }
    }

    results
}
```

- Since `to_lowercase` creates new data rather than referencing existing data, `query` is now a `String` instead of a string slice and we need to pass a reference to `contains` which takes a string slice.
- So our new function works. We are going to need to call it from `run` but first we need to add a field to our struct for the new option. Note that this will break the build, because we aren't initializing the field anywhere yet:

```rust
// ch12_minigrep/src/main.rs
struct Config {
    query: String,
    file_path: String,
    ignore_case: bool,
}

// same file
use minigrep::{search, search_case_insensitive};

// --snip--
fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let contents = fs::read_to_string(config.file_path)?;

    let results = if config.ignore_case {
        search_case_insensitive(&config.query, &contents)
    } else {
        search(&config.query, &contents)
    };

    for line in results {
        println!("{line}");
    }

    Ok(())
}
```

- We've already done `use std::env` so we have access to the functions to work with environment variables.

```rust
// ch12_minigrep/src/main.rs
impl Config {
    fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("not enough arguments");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        let ignore_case = env::var("IGNORE_CASE").is_ok();

        Ok(Config {
            query,
            file_path,
            ignore_case,
        })
    }
}
```

- We don't actually care about the value of `IGNORE_CASE`, just that it is set. So we can just call `is_ok` to make sure `var` returns an `Ok` and not an `Err`. Testing the changes (beyond the tests of the library that pass) is as simple as running `cargo run -- to poem.txt` and `IGNORE_CASE=1 cargo run -- to poem.txt`.

12.6 - Redirecting Errors to Standard Error

- The `println!` macro only prints to `stdout`, so we need something else to print to `stderr`. If we run `cargo run > output.txt` we see our error goes to the file instead of the screen. All our error handling code is in `main`, so it's a pretty simple update. We use `eprintln!` to print to `stderr`:

```rust
// ch12_minigrep/src/main.rs
fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::build(&args).unwrap_or_else(|err| {
        eprintln!("Problem parsing arguments: {err}");
        process::exit(1);
    });

    if let Err(e) = run(config) {
        eprintln!("Application error: {e}");
        process::exit(1);
    }
}
```

Chapter 13 - Functional Language Features: Iterators and Closures

13.1 - Closures

- Rust's closures are anonymous functions you can save in a variable or pass as arguments to other functions. Unlike functions, closures can capture values from the scope in which they're defined.

```rust
#[derive(Debug, PartialEq, Copy, Clone)]
enum ShirtColor {
    Red,
    Blue,
}

struct Inventory {
    shirts: Vec<ShirtColor>,
}

impl Inventory {
    fn giveaway(&self, user_preference: Option<ShirtColor>) -> ShirtColor {
        user_preference.unwrap_or_else(|| self.most_stocked())
    }

    fn most_stocked(&self) -> ShirtColor {
        let mut num_red = 0;
        let mut num_blue = 0;

        for color in &self.shirts {
            match color {
                ShirtColor::Red => num_red += 1,
                ShirtColor::Blue => num_blue += 1,
            }
        }
        if num_red > num_blue {
            ShirtColor::Red
        } else {
            ShirtColor::Blue
        }
    }
}

fn main() {
    let store = Inventory {
        shirts: vec![ShirtColor::Blue, ShirtColor::Red, ShirtColor::Blue],
    };

    let user_pref1 = Some(ShirtColor::Red);
    let giveaway1 = store.giveaway(user_pref1);
    println!(
        "The user with preference {:?} gets {:?}",
        user_pref1, giveaway1
    );
```

```
    let user_pref2 = None;
    let giveaway2 = store.giveaway(user_pref2);
    println!(
        "The user with preference {:?} gets {:?}",
        user_pref2, giveaway2
    );
}
```

- Here `unwrap_or_else` is being passed a closure. It takes one argument: a closure without any arguments that returns a value `T` (the same `T` in the `Some` variant of `Option<T>`). If the option is a `Some` variant, it returns the value from within the `Some`. If it is `None`, then it calls the closure and returns the value returned by the closure. The closure calls `self.most_stocked()` on the current `Inventory` instance. The closure captures an immutable reference to the `self Inventory` instance and passes it with the code we specify to the `unwrap_or_else` method.
- Closures don't usually require you to annotate the types of the parameters or the return value like normal functions do. As we see above, brackets are also optional, but only when the closure body has only one expression.
- The following causes `s` to immediately be dropped:

```
let f = |_| (); // sometimes called the "toilet closure"
let s = String::from("Hello");
f(s);
```

- Closures can capture values in three ways: borrowing immutably, borrowing mutably, and taking ownership. The closure will decide which of the three to do based upon the body of the function. Remember, a mutable borrow disables read access on the original.
- If your closure only needs a borrow, but you want to transfer ownership (often useful when doing concurrency), you can specify the `move` keyword: `thread::spawn(move || println!("From thread: {list:?}")).join().unwrap()` transfers ownership of `list` so if `main` finishes first it isn't deallocated.
- Traits are how functions and structs can specify what kinds of closures they can use. Closures will automatically implement one, two, or all three of these `Fn` traits depending on how the closure's body handles values from the environment:

    - `FnOnce` - All closures implement at least this trait. It applies to closures that can be called once. A closure that moves captured values out of its body will implement this and no other `Fn` traits, because it can only be called once.
    - `FnMut` - applies to closures that don't move captured values out of their body but might mutate the captured values. These closures can be called more than once.
    - `Fn` - applies to closures that don't move captured values out of their body and don't mutate captured values, as well as closures that capture nothing from their environment. These closures can be called more than once without mutating their environment, which is important in cases such as calling a closure multiple times concurrently.
- As an example, we will look at the standard library definition on `unwrap_or_else` defined on `Option<T>`:

```
impl<T> Option<T> {
    pub fn unwrap_or_else<F>(self, f: F) -> T
    where
        F: FnOnce() -> T
    {
        match self {
            Some(x) => x,
            None => f(),
        }
    }
}
```

- Because `unwrap_or_else` specifies `FnOnce`, and all closures implement `FnOnce`, it works with any closure. Indeed, you can even specify the name of a function to pass to this method, such as `Vec::new`, and the compiler would automatically implement the correct trait(s) for the function definition. By specifying `FnOnce`, `unwrap_or_else` will not call `f` more than once.

- Now we will look at `sort_by_key`. It specifies `FnMut` instead of `FnOnce`:

```rust
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    list.sort_by_key(|r| r.width);
    println!("{list:#?}");
}
```

- The reason it specifies `FnMut` is because it calls its closure more than once, even if it doesn't capture, mutate, or move anything out of its environment, so it meets the trait bound requirements. Here we try to pass it a function that implements `FnOnce` since it moves a value out of its environment:

```rust
// DOES NOT COMPILE! error[E0507]: cannot move out of `value`, a captured
//                   variable in an `FnMut` closure
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    let mut sort_operations = vec![];
    let value = String::from("closure called");

    list.sort_by_key(|r| {
        sort_operations.push(value);
        r.width
    });
    println!("{list:#?}");
}
```

- The closure captures `value` and then moves `value` out of the closure by transferring ownership of `value` to the `sort_operations` vector. This closure can only be called once; trying to call it a second time wouldn't work, because `value` would no longer be in the environment to be pushed into `sort_operations` again! Therefore the closure implements `FnOnce`.
- Instead we could close over a mutable variable and mutate that:

```rust
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    let mut num_sort_operations = 0;
    list.sort_by_key(|r| {
        num_sort_operations += 1;
        r.width
    });
    println!("{list:#?}, sorted in {num_sort_operations} operations");
}
```

13.2 - Processing a Series of Items with Iterators

- The iterator pattern allows you to perform some task on a sequence of items in turn without manually reimplementing the logic. In Rust, iterators are lazy, meaning they have no effect until you call methods that consume the iterator to use it up. On a `Vector`, the `iter` method returns an iterator.
- All iterators implement the `Iterator` trait that is defined in the standard library:

```rust
pub trait Iterator {
    type Item;

    fn next(&mut self) -> Option<Self::Item>;

    // methods with default implementations elided
}
```

- The `Item` type is an associated type that is returned from the iterator. We will go into the details of associated types in Chapter 20. The `Iterator` trait only requires implementation of a `next` method which returns one item wrapped in `Some` and when iteration is over

returns `None`. An iterator variable needs to be mutable since `next` changes internal state. We don't need to do this in a `for` loop despite it using an iterator behind the scenes, since `for` takes ownership of the iterator and makes it mutable behind the scenes.

- `Iterator` provides a number of different methods with default implementations. Some of these call `next` and are called *consuming adapters* because calling them uses up the iterator. `sum` is one example. It takes ownership of the iterator and uses it up. *Iterator adapters* don't consume the iterator but instead produce different iterators by changing some aspect of the original iterator. Iterator adapters like `map` are lazy and don't do anything on their own. To apply their changes, you must consume the iterator. Because iterator adapters are lazy, you can chain multiple before consuming them to produce results. `collect` is a handy way to consume an iterator and collect the resultant values into a collection type.

```
let v1: Vec<i32> = vec![1, 2, 3];
let v2: Vec<_> = v1.iter().map(|x| x + 1).collect();
assert_eq!(v2, vec![2, 3, 4]);
```

- (Not in the book) Advanced flexibility of `collect`:
  - `collect` is powered by the `FromIterator` trait, which allows it to collect into any type implementing it based on type inference.
  - Rustlings `iterators2`: Collecting into `String` vs `Vec<String>`. The exact same pipeline can build a vector or concatenate directly into a single `String` with zero changes to the function body:

```rust
fn capitalize_first(s: &str) -> String {
    let mut chars = s.chars();
    match chars.next() {
        None => String::new(),
        Some(c) => c.to_uppercase().to_string() + chars.as_str(),
    }
}

// Collects into a vector:
fn capitalize_vec(words: &[&str]) -> Vec<String> {
    words.iter().map(|w| capitalize_first(w)).collect()
}

// Collects into a single String (identical body, different return type):
fn capitalize_str(words: &[&str]) -> String {
    words.iter().map(|w| capitalize_first(w)).collect()
}
```

  - Rustlings `iterators3`: Transposing containers of `Result`s. An iterator of `Result<T, E>` can be collected into a list of results, or inverted into a single `Result` that fails fast on the first error:

```rust
fn divide(a: i64, b: i64) -> Result<i64, &'static str> {
    if b == 0 {
        Err("divide by zero")
    } else {
        Ok(a / b)
    }
}

let nums = [27, 297, 81];

// List of results: Vec<Result<i64, &str>>
let list: Vec<Result<i64, _>> =
    nums.into_iter().map(|n| divide(n, 27)).collect();

// Inverted: returns Ok(Vec<i64>) if all succeed, or the first Err:
let res: Result<Vec<i64>, _> =
    nums.into_iter().map(|n| divide(n, 27)).collect();
```

- Many iterator adapters take closures as arguments, and commonly those closures capture their environment. Here is an example of `filter` with a closure that captures the `shoe_size` variable:

```rust
#[derive(PartialEq, Debug)]
struct Shoe {
    size: u32,
    style: String,
}

fn shoes_in_size(shoes: Vec<Shoe>, shoe_size: u32) -> Vec<Shoe> {
    shoes.into_iter().filter(|s| s.size == shoe_size).collect()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn filters_by_size() {
        let shoes = vec![
            Shoe {
                size: 10,
                style: String::from("sneaker"),
            },
            Shoe {
                size: 13,
                style: String::from("sandal"),
            },
            Shoe {
                size: 10,
                style: String::from("boot"),
            },
        ];

        let in_my_size = shoes_in_size(shoes, 10);

        assert_eq!(
            in_my_size,
            vec![
                Shoe {
                    size: 10,
                    style: String::from("sneaker")
                },
                Shoe {
                    size: 10,
                    style: String::from("boot")
                },
            ]
```

```
        );
    }
}
```

- `shoes_in_size` takes ownership of a vector of shoes and a shoe size as parameters. It returns a vector containing only shoes of the specified size. The call to `into_iter` creates an iterator that takes ownership of the vector. The `filter` adapts that into a new iterator. The closure captures the `shoe_size` parameter, and the call to `collect` gathers the values returned into a vector.

13.3 - Improving our I/O Project

- Recall last chapter where we had a `Config::build` function that had to clone its arguments. We needed `clone` here because we have a slice with `String` elements in the parameter `args`, but the `build` function doesn't own `args`:

```rust
// ch12_minigrep/src/main.rs
impl Config {
    fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("not enough arguments");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        let ignore_case = env::var("IGNORE_CASE").is_ok();

        Ok(Config {
            query,
            file_path,
            ignore_case,
        })
    }
}

// This is called from main here:
fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::build(&args).unwrap_or_else(|err| {
        eprintln!("Problem parsing arguments: {err}");
        process::exit(1);
    });

    // --snip--
}
```

- Once `Config::build` takes ownership of the iterator and stops using indexing operations that borrow, we can move the `String` values from the iterator into `Config`. `env::args` returns an iterator we were `collect`ing into a vector before. Now we pass ownership of the iterator to `Config::build` directly:

```rust
// ch13_minigrep_iter/src/main.rs
// First we will change where we call it to pass an iterator
fn main() {
    let config = Config::build(env::args()).unwrap_or_else(|err| {
        eprintln!("Problem parsing arguments: {err}");
        process::exit(1);
    });

    // --snip--
}


// We need to make `args` `mut`, and it can be any type that implements the
// `Iterator<Item = String>` trait, so any Iterator returning String values.
impl Config {
    fn build(
        mut args: impl Iterator<Item = String>,
    ) -> Result<Config, &'static str> {
        args.next();  // first value is program name

        let query = match args.next() {
            Some(arg) => arg,
            None => return Err("Didn't get a query string"),
        };

        let file_path = match args.next() {
            Some(arg) => arg,
            None => return Err("Didn't get a file path"),
        };

        let ignore_case = env::var("IGNORE_CASE").is_ok();

        Ok(Config {
            query,
            file_path,
            ignore_case,
        })
    }
}
```

- We can also take advantage of iterators in our `search` function. Here's the code from last chapter:

```rust
// ch12_minigrep/src/lib.rs
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    let mut results = Vec::new();

    for line in contents.lines() {
        if line.contains(query) {
            results.push(line);
        }
    }

    results
}
```

- Change it to:

```rust
// ch13_minigrep_iter/src/lib.rs
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    contents
        .lines()
        .filter(|line| line.contains(query))
        .collect()
}
```

- We can also update the `search_case_insensitive` function:

```rust
pub fn search_case_insensitive<'a>(
    query: &str,
    contents: &'a str,
) -> Vec<&'a str> {
    let query = query.to_lowercase();

    contents
        .lines()
        .filter(|line| line.to_lowercase().contains(&query))
        .collect()
}
```

Additional Project: Returning Iterators Directly

- (Not in the book) The book notes that `search` and `search_case_insensitive` could return an iterator directly instead of collecting into a `Vec<&'a str>`. Doing so eliminates

the heap allocation of the `Vec` inside the search functions, letting the caller lazily stream lines as needed.

- This change requires addressing a few subtleties:
  - **Return type**: Because closure types in iterator adapters like `filter` are anonymous and unnameable, the return type must be written as `impl Iterator<Item = &'a str>`.
  - **Lifetimes**: `query` in `search` must live as long as the returned iterator (`&'a str`) because the lazy closure retains a reference to it until iteration is finished.
  - **Ownership in `search_case_insensitive`**: `query.to_lowercase()` creates a local `String`. To return an iterator without returning a dangling reference to that local string, the closure must own it via `move`.
  - **Branching in `run`**: Because each closure produces a distinct concrete iterator type, `let results = if ... else ...` fails with mismatched types (`error[E0308]`). Instead, we can run a separate `for` loop inside each branch (or use dynamic dispatch with `Box<dyn Iterator<Item = &str>>`).
  - **Unit tests**: `assert_eq!` cannot compare a `Vec<&str>` directly against an iterator type. In tests, we either call `.collect::<Vec<_>>()` on the returned iterator or use `Iterator::eq`.

```rust
// ch13_minigrep_iter/src/lib.rs
pub fn search<'a>(
    query: &'a str,
    contents: &'a str,
) -> impl Iterator<Item = &'a str> {
    contents.lines().filter(move |line| line.contains(query))
}

pub fn search_case_insensitive<'a>(
    query: &str,
    contents: &'a str,
) -> impl Iterator<Item = &'a str> {
    let query = query.to_lowercase();

    contents
        .lines()
        .filter(move |line| line.to_lowercase().contains(&query))
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn case_sensitive() {
        let query = "duct";
        let contents = "\
Rust:
safe, fast, productive.
Pick three.
Duct tape.";

        assert_eq!(
            vec!["safe, fast, productive."],
            search(query, contents).collect::<Vec<_>>()
        );
    }

    #[test]
    fn case_insensitive() {
        let query = "rUsT";

        let contents = "\
Rust:
safe, fast, productive.
```

```
Pick three.
Trust me.";

        assert_eq!(
            vec!["Rust:", "Trust me."],
            search_case_insensitive(query, contents).collect::<Vec<_>>()
        );
    }
}
```

- And in *main.rs*, `run` branches into two `for` loops rather than assigning to a single `results` variable:

```
// ch13_minigrep_iter/src/main.rs
fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let contents = fs::read_to_string(config.file_path)?;

    if config.ignore_case {
        for line in search_case_insensitive(&config.query, &contents) {
            println!("{line}");
        }
    } else {
        for line in search(&config.query, &contents) {
            println!("{line}");
        }
    }

    Ok(())
}
```

13.4 - Performance in Loops vs Iterators

- Iterators are one of Rust's *zero-cost abstractions*. Using the abstraction imposes no additional runtime overhead. In many cases, Rust's code using iterators compiles down to the same assembly you'd write by hand.
- In an audio decoder benchmark from the book, the iterator implementation was actually slightly faster than a manual loop because the compiler was able to eliminate array bounds checks and unroll the loop.

Chapter 14 - More About Cargo and Crates.io

14.1 - Customizing Builds with Release Profiles

- Cargo has two main profiles: the `dev` profile Cargo uses when you run `cargo build`, and the `release` profile Cargo uses when you run `cargo build --release`. Cargo has default settings for each profile that apply when you haven't explicitly added any `[profile.*]` sections to your project's *Cargo.toml* file. Default settings are covered in the Cargo reference.
- The primary profile configuration shown is `opt-level`, which controls how many compiler optimizations are applied (defaults to `0` for `dev` to keep compiles fast, and `3` for `release` for maximum execution speed).

14.2 - Publishing a Crate to Crates.io

- Documentation comments use three slashes, `///`, and support Markdown notation for formatting the text. Place them just before the item they are documenting.

```
/// Adds one to the number given.
///
/// # Examples
///
/// ```
/// let arg = 5;
/// let answer = my_crate::add_one(arg);
///
/// assert_eq!(6, answer);
/// ```
pub fn add_one(x: i32) -> i32 {
    x + 1
}
```

- Running `cargo doc --open` compiles the HTML documentation for your current crate and all dependencies, and opens it directly in your browser.
- Examples in your comments are run as tests when you run `cargo test`.
- Commonly used sections include Examples, Panics, Errors, and Safety.

- `//!` comments are for documenting the item that *contains* the comments rather than the item *following* the comments. They are normally used inside the crate root (*src/lib.rs*) or inside a module to document the crate or module as a whole. This is to document `My Crate`:

```
//! # My Crate
//!
//! `my_crate` is a collection of utilities to make performing certain
//! calculations more convenient.

/// Adds one to the number given.
// --snip--
```

- Make your API convenient for users by re-exporting internal items with `pub use`.
- After registering, you can get your crates.io API key at https://crates.io/me/ then use it with `cargo login`.
- Publishing a version to crates.io is permanent: code cannot be modified or deleted once published, guaranteeing projects depending on that version never break.
- `cargo yank` prevents new projects from adding a specific version as a new dependency, while allowing existing projects with a `Cargo.lock` to continue downloading it. Use `cargo yank --undo` to un-yank a version.

14.3 - Cargo Workspaces

- A *workspace* is a set of packages that share the same *Cargo.lock* and output directory.
- I already have a workspace for this book, so I'll use its file as an example:

```
[workspace]
members = ["ch*_*"]
exclude = ["code"]
resolver = "3"
```

- The book source code is in `code/`, so I don't want it included. All my projects live in folders like `ch05_rectangles`, so I match them with a glob, and the resolver level is 3; the latest and greatest. You don't have to make your projects match a glob though. Making a new project in the workspace folder with `cargo new` will add the new project to *Cargo.toml*.

- Let's say we had a workspace with a binary `adder` that depends on two libraries: `add_one` and `add_two`. We could make our top level folder anything, so let's make it `add`. Then we would do `cargo new adder`, `cargo new add_one --lib`, and `cargo new add_two --lib`. Now if `adder` uses the function `add_one` in the same project, in its `adder/Cargo.toml` file you would add a line as follows:

```
[dependencies]
add_one = { path = "../add_one" }
```

- Cargo builds projects in a workspace together but doesn't assume they depend on each other unless you list it explicitly. Also, now cargo needs help knowing what to run, so to run `adder`, we `cargo run -p adder`.

- If we need an external dependency in more than one project, it needs to be listed in each project's *Cargo.toml*. The *Cargo.lock* at the workspace level will keep the version the same between projects if possible.

- When you run `cargo build` or `cargo test` at the workspace level, it builds or tests all projects and puts executables in a top-level *target* directory, in our running example `add/target`. You can run tests just for the `add_one` project from the top-level with `cargo test -p add_one`. Crates have to be published to crates.io separately, but you can run `cargo publish -p add_one` at the top level to publish `add_one`.

14.4 - Installing Binaries with cargo install

- `cargo install` lets you install and use binary crates locally. The binary by default goes to `$HOME/.cargo/bin`.

14.5 - Extending Cargo with Custom Commands

- Any executable in your path named `cargo-something` can be run using the command `cargo something`. They also show up in `cargo --list`.

Chapter 15 - Smart Pointers

- *Smart pointers* are data structures that act like a pointer but also have additional metadata and capabilities. Rust has a variety of smart pointers defined in the standard library that provide functionality beyond that provided by references. For example, there is a *reference counting* pointer that enables you to have multiple owners by keeping track of the number

of owners and cleaning up the data when no owners remain. While references only borrow data, in many cases smart pointers own the data they point to.

- Smart pointers are usually implemented using structs, but unlike a normal struct, they implement the `Deref` and `Drop` traits. The `Deref` trait allows them to behave like a reference so code can be written to work with both references and smart pointers. The `Drop` trait allows you to customize the code that's run when an instance of the smart pointer goes out of scope.
- There are too many smart pointers to cover them all as it is a common design pattern in Rust. Many libraries provide their own smart pointers, and you can even write your own. We will focus on the most common smart pointers in the standard library:
  - `Box<T>`, for allocating values on the heap
  - `Rc<T>`, a reference counting type that enables multiple ownership
  - `Ref<T>` and `RefMut<T>`, accessed through `RefCell<T>`, a type that enforces the borrowing rules at runtime instead of compile time
- In addition, we will cover the *interior mutability* pattern where an immutable type exposes an API for mutating an interior value. We'll also discuss reference cycles: how they can leak memory and how to prevent them.

15.1 - Using `Box<T>` to Point to Data on the Heap

- Boxes allow you to store data on the heap rather than the stack, and they have no performance overhead beyond heap access, but they don't have extra capabilities. You use them most often in these situations:
  - When you have a type whose size can't be known at compile time, and you want to use a value of that type in a context that requires an exact size
  - When you have a large amount of data, and you want to transfer ownership but ensure that the data won't be copied when you do so
  - When you want to own a value, and you care only that it's a type that implements a particular trait rather than being a specific type

```
// Storing an i32 on the heap with a `Box`
fn main() {
    let b = Box::new(5);
    println!("b = {b}");
}
```

- Storing a single value on the heap isn't very useful, so you won't see code like the above often, but it demonstrates the simplicity of using a `Box`.
- A value of a *recursive type* can have another value of the same type as part of itself. Since a recursive type could theoretically be nested infinitely, Rust can't determine at compile time how much space they take up. Because boxes have a known size, we can enable recursive types by inserting a box in the recursive type definition.
- A *cons list* is a data structure that is made up of nested pairs and is the Lisp version of a linked list. Most of the time in Rust, a `Vec<T>` is a better choice for a list of items.
- If we try the naive implementation of cons, we will get compiler errors: `error[E0072]: recursive type 'List' has infinite size` and `error[E0391]: cycle detected when computing when 'List' needs drop`.

```
// DOES NOT COMPILE! error[E0072]: recursive type `List` has infinite size
// DOES NOT COMPILE! error[E0391[: cycle detected when computing when `List`
//                                 needs drop
// We are implementing this to hold `i32` for simplicity but could easily
make
// it generic.
enum List {
    Cons(i32, List),
    Nil,
}

use crate::List::{Cons, Nil}

fn main() {
    let list = Cons(1, Cons(2, Cons(3, Nil)));
}
```

- The first compiler error helpfully points out to fix this my introducing indirection (e.g., a `Box`, `Rc`, or `&`) to break the cycle.

```
enum List {
    Cons(i32, Box<List>),
    Nil,
}

use crate::List::{Cons, Nil}

fn main() {
    let list = Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))));
}
```

- We have finally improved Lisp by adding even more parentheses. Now a cons cell has the size of one `i32` plus a `usize` pointer. Before its size was indeterminate, because it was one `i32` plus another `Cons` cell, which in turn was one `i32` plus another `Cons` cell, indefinitely.
- `Box<T>` is a smart pointer because it implements the `Deref` trait which allows `Box<T>` values to be treated like references. When a `Box<T>` goes out of scope, the heap data that the box is pointing to is cleaned up because of the `Drop` trait implementation.

15.2 - Treating Smart Pointers Like Regular References with the Deref Trait

- Implementing the `Deref` trait allows you to customize the behavior of the *dereference operator* (`*`). By implementing `Deref` so a smart point can be treated like a regular reference, you can write code that operates on references and use that code with smart pointers too.
- You can use the dereference operator to access the value inside the `Box`, in the same way that you use it to access the value pointed to by a reference.

```rust
// Dereferencing an `i32`
fn main() {
    let x = 5;
    let y = &x;

    assert_eq!(5, x);
    assert_eq!(5, *y);
}

// Dereferencing a `Box<T>`
fn main() {
    let x = 5;
    let y = Box::new(x);

    assert_eq!(5, x);
    assert_eq!(5, *y);
}
```

- The `MyBox<T>` we are designing here does not store it's data on the heap, so we can focus on `Deref`.

```rust
// A simple tuple-struct
struct MyBox<T>(T);

impl<T> MyBox<T> {
    fn new(x: T) -> MyBox<T> {
        MyBox(x)
    }
}

// Dereferencing a `MyBox<T>`
// DOES NOT COMPILE!
// error[E0614]: type `MyBox<{integer}>` cannot be dereferenced
fn main() {
    let x = 5;
    let y = MyBox::new(x);

    assert_eq!(5, x);
    assert_eq!(5, *y);
}
```

- To enable dereferencing with the `*` operator, we implement the `Deref` trait.

```rust
use std::ops::Deref;

impl<T> Deref for MyBox<T> {
    type Target = T;

    fn deref(&self) -> &Self::Target {
        &self.0
    }
}
```

- We file in the body of the `deref` method with `&self.0` so that `deref` returns a reference to the value we want to access with the `*` operator. Now the example above with `assert_eq!(5, *y);` will work.
- Without the `Deref` trait, the compiler can only dereference `&` references. The `deref` method gives the compiler the ability to take any value that implements `Deref` and call the `deref` method to get a reference that it knows how to dereference.
- *Deref coercion* converts a reference to a type that implements the `Deref` trait into a reference to another type. For example, deref coercion can convert `&String` to `&str` because `&String` implements the `Deref` trait such that it returns `&str`. Deref coercion is a convenience Rust performs on arguments to functions and methods, and it works only on types that implement the `Deref` trait. It happens automatically when we pass a reference to a particular type's value as an argument to a function or method that doesn't match the parameter type in the function or method definition. A sequence of calls to the `deref` method converts the type we provided to the type the parameter needs.

```rust
fn hello(name: &str) {
    println!("Hello, {name}!");
}

fn main() {
    let m = MyBox::new(String::from("Rust"));
    hello(&m);
}
```

- Here we call `hello` with `&m` which is a reference to a `MyBox<String>` value. But because we implemented the `Deref` trait on `MyBox<T>`, Rust can turn `&MyBox<String>` into `&String` by calling `deref`. And the standard library provides an implementation of `Deref`

on `String` that returns a string slice, so Rust calls `deref` again to turn `&String` into `&str`.

- Without deref coercion, we would of had to write this:

```
fn main() {
    let m = MyBox::new(String::from("Rust"));
    hello(&(*m)[..]);
}
```

- One dereference to get at the `String`, then `[..]` to turn it into a slice, then `&` to get a reference to the slice. When the `Deref` trait is defined for the types involved, Rust will analyze the types and use `Deref::deref` as many times as necessary to get a reference to match the parameter's type. The calls are inserted and resolved at compile time, so there is no runtime penalty for taking advantage of deref coercion.

- There is also a `DerefMut` trait to override the `*` operator on mutable references. Rust does deref coercion when it finds types and trait implementations in three cases:

    1. From `&T` to `&U` when `T: Deref<Target=U>`
    2. From `&mut T` to `&mut U` when `T: DerefMut<Target=U>`
    3. From `&mut T` to `&U` when `T: Deref<Target=U>`
- As we see, Rust will coerce a mutable reference into an immutable one. The reverse is not possible.

15.3 - Running Code on Cleanup with the Drop Trait

- `Drop` allows you to customize what happens when a variable goes out of scope. It can be used to release resources like files or network connections. It is often used with smart pointers to provide cleanup. In Rust, you won't leak resources by not manually cleaning up after yourself thanks to the `Drop` trait. It requires you to implement one method named `drop` that takes a mutable reference to `self`. Objects are dropped in the reverse order of creation.

```
struct CustomSmartPointer {
    data: String,
}

impl Drop for CustomSmartPointer {
    fn drop(&mut self) {
        println!("Dropping CustomSmartPointer with data `{}`!", self.data);
    }
}

fn main() {
    let c = CustomSmartPointer {
        data: String::from("my stuff"),
    };
    let d = CustomSmartPointer {
       data: String::from("other stuff"),
    };
    println!("CustomSmartPointers created");
}

/* OUTPUT:

   CustomSmartPointers created
   Dropping CustomSmartPointer with data `other stuff`!
   Dropping CustomSmartPointer with data `my stuff`!      */
```

- There is no easy way to call the `drop` method manually. Instead, you can call `std::mem::drop` to drop an object early. Trying to call `val.drop()` results in a compiler error `error[E0030]: explicit use of destructor method`.

15.4 - `Rc<T>`, the Reference Counted Smart Pointer

- Sometimes a single value might have multiple owners. Consider a graph data structure. A node is conceptually owned by all the incoming edges that point to it and shouldn't be cleaned up unless it doesn't have any incoming edges. `Rc<T>` gives us a way to model this, and the name is an abbreviation for *reference counting*. The `Rc<T>` type keeps track of the number of references to a value to determine whether or not it is still in use, allowing it to be cleaned up only when there are zero references. We use `Rc<T>` when we want to allocate some data on the heap for multiple parts of our program to read and we cannot determine at compile time which part will finish using the data last. If we knew, we would just make the last one owner. Note that `Rc<T>` is only used in single-thread scenarios and we will cover concurrency in Chapter 16.

- Lets reconsider our cons example. You could have two cons cells, `b` and `c`, with both pointed to a third cons cell `a` as their second element.

```
// DOES NOT COMPILE! error[E0382]: use of moved value: `a`
enum List {
    Cons(i32, Box<List>),
    Nil,
}

use crate::List::{Cons, Nil};

fn main() {
    let a = Cons(5, Box::new(Cons(10, Box::new(Nil))));
    let b = Cons(3, Box::new(a));
    let c = Cons(4, Box::new(a));
}
```

- We could implement this instead with `Rc<T>`. Every time `Rc::clone` is called, the reference count to the data within the `Rc<List>` will increase by 1. It starts at 1, so at the end it will be 3.

```
enum List {
    Cons(i32, Rc<List>),
    Nil,
}

use crate::List::{Cons, Nil};
use std::rc::Rc;

fn main() {
    let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))));
    let b = Cons(3, Rc::clone(&a));
    let c = Cons(4, Rc::clone(&a));
}
```

- We could have used `a.clone()` above, but Rust's convention is to use `Rc::clone` in this case. Most `clone` implementations make a deep copy of the data, but `Rc::clone` only increments the reference count. Clone calls are often a source of performance problems, but we don't need to worry with calls to `Rc::clone`. You can call `rc::strong_count(&a)` to see the reference count. Via immutable references, `Rc<T>` allows you to share data between multiple parts of your program for reading only. If we were to manually `drop` `a`,

its reference count would reduce by one and its `drop` method would not be called until after both `b` and `c` are dropped or out of scope.

15.5 - `RefCell<T>` and the Interior Mutability Pattern

- *Interior mutability* is a design pattern in Rust that allows you to mutate data even when there are immutable references to that data; normally, this action is disallowed by the borrowing rules. The pattern relies on `unsafe` code inside a data structure. We can use types with this pattern only when we can ensure that the borrowing rules will be followed at runtime despite the compiler being unable to guarantee that. The `unsafe` code involved is wrapped in a safe API, and the outer type is still immutable. The `RefCell<T>` type follows the interior mutability pattern.

- Unlike `Rc<T>`, the `RefCell<T>` type represents single ownership over the data it holds. Recall the borrowing rules:

    - At any given time, you can have either one mutable reference or any number of immutable references (but not both).
    - References must always be valid.
- With references and `Box<T>`, the borrowing rules' invariants are enforced at compile time. But with `RefCell<T>` these invariants are enforced at runtime. With `RefCell<T>`, if you break the rules, your program will panic and exit.

- Checking the borrowing rules at compile time is generally the superior strategy. It catches errors sooner and has no runtime performance penalty. Checking at runtime allows certain memory-safe scenarios that would have been blocked by compile-time checks. The `RefCell<T>` type is useful when you're sure your code follows the borrowing rules but the compiler is unable to understand and guarantee that. Similar to `Rc<T>`, `RefCell<T>` is only for single-thread scenarios and will give you a compile-time error if you are using it in a multithreaded context.

- Here is a recap of the reasons to choose `Box<T>`, `Rc<T>`, or `RefCell<T>`:

    - `Rc<T>` enables multiple owners of the same data; `Box<T>` and `RefCell<T>` have single owners.
    - `Box<T>` allows immutable or mutable borrows checked at compile time; `Rc<T>` allows only immutable borrows checked at compile time; `RefCell<T>` allows immutable or mutable borrows checked at runtime.

    - Because `RefCell<T>` allows mutable borrows checked at runtime, you can mutate the value inside the `RefCell<T>` even when the `RefCell<T>` is immutable.
- Mutating the value inside an immutable value is the interior mutability pattern. As you know, if you have an immutable value, you cannot borrow it mutably. Sometimes during testing a programmer will use a type in place of another type in order to observe particular behavior and assert that its implemented correctly. This placeholder is called a *test double*. *Mock objects* are specific types of test doubles that record what happens during a test so that you can assert that the correct actions took place. Rust doesn't have objects or build in mock object functionality, but you can definitely create a struct that will serve the same purposes as a mock object. We will test a library that tracks a value against a maximum value and sends messages based on how close to the maximum value the current value is. Our library will track how close to the maximum the value is and manage what messages are sent, but its to be decouple from sending messages by calling a class that implements the `Messenger` trait with a `send` method.

```rust
pub trait Messenger {
    fn send(&self, msg: &str);
}

pub struct LimitTracker<'a, T: Messenger> {
    messenger: &'a T,
    value: usize,
    max: usize,
}

impl<'a, T> LimitTracker<'a, T>
where
    T: Messenger,
{
    pub fn new(messenger: &'a T, max: usize) -> LimitTracker<'a, T> {
        LimitTracker {
            messenger,
            value: 0,
            max,
        }
    }

    pub fn set_value(&mut self, value: usize) {
        self.value = value;

        let percentage_of_max = self.value as f64 / self.max as f64;

        if percentage_of_max >= 1.0 {
            self.messenger.send("Error: You are over your quota!");
        } else if percentage_of_max >= 0.9 {
            self.messenger.send(
                "Urgent warning: You've used up over 90% of your quota!",
            );
        } else if percentage_of_max >= 0.75 {
            self.messenger.send(
                "Warning: You've used up over 75% of your quota!",
            );
        }
    }
}
```

- Now we see `Messenger` has a `send` method that takes an immutable reference to `&self` and the text of the message. The trait is what our mock object needs to implement so it can be used like the real object. The other important part is we want to test the behavior of the

`set_value` method on the `LimitTracker`. We can change what we pass for the `value` parameter, but the method doesn't return anything to make assertions on. We want to be able to say that if we create a `LimitTracker` with something that implements the `Messenger` trait and a particular value for `max`, the messenger is told to send the appropriate messages when we pass different numbers for `value`. Our mock object needs to keep track of what messages it's told to send.

```
// DOES NOT COMPILE! error[E0596]: cannot borrow `self.sent_messages` as
//                                 mutable, as it is behind a `&` reference
#[cfg(test)]
mod tests {
    use super::*;

    struct MockMessenger {
        sent_messages: Vec<String>,
    }

    impl MockMessenger {
        fn new() -> MockMessenger {
            MockMessenger {
                sent_messages: vec![],
            }
        }
    }

    impl Messenger for MockMessenger {
        fn send(&self, message: &str) {
            self.sent_messages.push(String::from(message));
        }
    }

    #[test]
    fn it_sends_an_over_75_percent_warning_message() {
        let mock_messenger = MockMessenger::new();
        let mut limit_tracker = LimitTracker::new(&mock_messenger, 100);

        limit_tracker.set_value(80);

        assert_eq!(mock_messenger.sent_messages.len(), 1);
    }
}
```

- We cannot modify `MockMessenger` to keep track of the messages, because the `send` method takes an immutable reference to `self`. And we do not want to change the `Messenger` trait to take `&mut self` solely for the sake of testing. This is where interior mutability can help. If we store the `sent_messages` in a `RefCell<T>` then the `send` method will be able to modify `sent_messages` to store the messages we've seen.

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::cell::RefCell;

    struct MockMessenger {
        sent_messages: RefCell<Vec<String>>,
    }

    impl MockMessenger {
        fn new() -> MockMessenger {
            MockMessenger {
                sent_messages: RefCell::new(vec![]),
            }
        }
    }

    impl Messenger for MockMessenger {
        fn send(&self, message: &str) {
            self.sent_messages.borrow_mut().push(String::from(message));
        }
    }

    #[test]
    fn it_sends_an_over_75_percent_warning_message() {
        // --snip--

        assert_eq!(mock_messenger.sent_messages.borrow().len(), 1);
    }
}
```

- We just had to wrap our `Vec<String>` in a `RefCell` in both the `MockMessenger` definition and initialization. Now in our `send` method, we `borrow_mut` on `sent_messages` and update the interior `Vec<String>`. We can access it in our test to check the value with a normal `borrow.`

- We use `borrow` and `borrow_mut` methods to get a reference to our `RefCell<T>`. They return a `Ref<T>` and `RecMut<T>` respectively and both implement the `Deref` trait so can be treated like regular references. The `RefCell<T>` keeps track of how many `Ref<T>` and `RefMut<T>` smart pointers are currently active. Just like compile-time borrowing, there can be many immutable borrows or one mutable borrow at any point in time. The `RefCell<T>` will panic if we violate these rules at runtime.

- It is common to use `RefCell<T>` with `Rc<T>`. Recall, `Rc<T>` lets you have multiple owners of some data but only gives immutable access. If you have a `Rc<T>` that holds a `RefCell<T>`, you can get a value that can have multiple owners and that you can mutate. Recall our cons example. Here it is modified so we can mutate the lists:

```
#[derive(Debug)]
enum List {
    Cons(Rc<RefCell<i32>>, Rc<List>),
    Nil,
}

use crate::List::{Cons, Nil};
use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    let value = Rc::new(RefCell::new(5));

    let a = Rc::new(Cons(Rc::clone(&value), Rc::new(Nil)));

    let b = Cons(Rc::new(RefCell::new(3)), Rc::clone(&a));
    let c = Cons(Rc::new(RefCell::new(4)), Rc::clone(&a));

    *value.borrow_mut() += 10;

    println!("a after = {a:?}");
    println!("b after = {b:?}");
    println!("c after = {c:?}");
}
```

- By using `RefCell<T>`, we have an outwardly immutable `List` value, but we can use the methods on `RefCell<T>` that provide access to its interior mutability so that we can modify our data when we need to. Note again that `RefCell<T>` does not work for multithreaded code, but `Mutex<T>` is a thread-safe version we will discuss in Chapter 16.

15.6 - Reference Cycles Can Leak Memory

- Rust does not promise safety from memory leaks. Indeed, they are possible given circular references. Consider this, the cons example but modified so we can mutate the second cell of the cons:

```rust
use crate::List::{Cons, Nil};
use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug)]
enum List {
    Cons(i32, RefCell<Rc<List>>),
    Nil,
}

impl List {
    fn tail(&self) -> Option<&RefCell<Rc<List>>> {
        match self {
            Cons(_, item) => Some(item),
            Nil => None,
        }
    }
}

fn main() {
    let a = Rc::new(Cons(5, RefCell::new(Rc::new(Nil))));

    println!("a initial rc count = {}", Rc::strong_count(&a));
    println!("a next item = {:?}", a.tail());

    let b = Rc::new(Cons(10, RefCell::new(Rc::clone(&a))));

    println!("a rc count after b creation = {}", Rc::strong_count(&a));
    println!("b initial rc count = {}", Rc::strong_count(&b));
    println!("b next item = {:?}", b.tail());

    if let Some(link) = a.tail() {
        *link.borrow_mut() = Rc::clone(&b);
    }

    println!("b rc count after changing a = {}", Rc::strong_count(&b));
    println!("a rc count after changing a = {}", Rc::strong_count(&a));

    // Uncomment the next line to see that we have a cycle;
    // it will overflow the stack.
    // println!("a next item = {:?}", a.tail());
    /* OUTPUT:

    a initial rc count = 1
    a next item = Some(RefCell { value: Nil })
```

```
    a rc count after b creation = 2
    b initial rc count = 1
    b next item = Some(RefCell { value: Cons(5, RefCell { value: Nil }) })
    b rc count after changing a = 2
    a rc count after changing a = 2 */
}
```

- Because `a` and `b` both have two reference counts, when both are dropped they are only reduced to 1 reference count each, so the memory they point to is never freed. If you have `RefCell<T>` values that contain `Rc<T>` values or similar nested combinations of types with interior mutability and reference counting, you must ensure you don't create cycles.
- You can create a weak reference to an `Rc<T>` by calling `Rc::downgrade` and passing it a reference to the `Rc<T>`. Weak references don't express ownership or prevent the `Rc<T>` instance from being cleaned up, so they won't cause a reference cycle. Because the `Rc<T>` may have been dropped, you need to call `Rc::upgrade` on the `Weak<T>` reference to get back an `Option<Rc<T>>` in order to do anything with the `Rc<T>` value.
- Consider a tree whose items know about their child items and their parent items.

```rust
use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    value: i32,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

fn main() {
    let leaf = Rc::new(Node {
        value: 3,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![]),
    });

    println!("leaf parent = {:?}", leaf.parent.borrow().upgrade());

    let branch = Rc::new(Node {
        value: 5,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![Rc::clone(&leaf)]),
    });

    *leaf.parent.borrow_mut() = Rc::downgrade(&branch);

    println!("leaf parent = {:?}", leaf.parent.borrow().upgrade());
}

/* OUTPUT:

  leaf parent = None
  leaf parent = Some(Node { value: 5, parent: RefCell { value: (Weak) },
  children: RefCell { value: [Node { value: 3,
  parent: RefCell { value: (Weak) },
  children: RefCell { value: [] } }] } }) */
```

- We want a `Node` to own its children, and we want to share that ownership with variables so that we can access each `Node` in the tree directly. We also want to modify which nodes are children of another node, so we wrap the `Vec<Rc<Node>>` in a `RefCell<T>` inside `children`. After creating `leaf` and `branch`, `leaf` has two owners: `leaf` and `branch`.

When we print `leaf.parent`, we see `leaf` inside `branch.children`, but because `parent` is a `Weak<Node>`, we avoid an infinite cycle.

15.7 - Chapter Summary: Smart Pointers

- Rust's core smart pointers balance ownership, borrow checking, and mutability:

Type	Owners	Check time	Mutability	Use case
`Box<T>`	Single	Compile time	Outward	Heap alloc, sized
`Rc<T>`	Multiple	Compile time	Immutable	Shared read (single)
`RefCell<T>`	Single	Runtime	Interior	Mocking, mutation
`Rc<RefCell<T>>`	Multiple	Runtime	Interior	Shared mutable data

- **Key traits and mechanisms**:
  - `Deref`: Overrides `*` operator and powers deref coercion (e.g. `&String` coerces automatically to `&str` when passed to functions).
  - `Drop`: Defines automatic cleanup when an instance leaves scope. Can trigger early cleanup via `drop(x)`, which is just a move into an empty function.
  - `Weak<T>`: Non-owning reference created with `Rc::downgrade`. Breaks reference cycles so memory can be reclaimed. Must call `.upgrade()` to obtain an `Option<Rc<T>>` before use.

Chapter 16 - Fearless Concurrency

- *Concurrent programming* is when different parts of a program execute independently. *Parallel programming* is when different parts of a program execute at the same time. We will use the two contexts interchangeably for this chapter and be more strict in the next. This chapter applies to concurrent and/or parallel programming. Programming in these contexts has historically been difficult and error-prone, but the Rust team discovered that by leveraging ownership and type checking, many concurrency errors become compile-time errors.
- Topics in this chapter:
  - How to create threads to run multiple pieces of code at the same time
  - *Message-passing* concurrency, where channels send messages between threads
  - *Shared-state* concurrency, where multiple threads have access to some piece of data

    - The `Sync` and `Send` traits, which extend Rust's concurrency guarantees to user-defined types provided by the standard library

16.1 - Using Threads to Run Code Simultaneously

- In most current operating systems, an executed program's code is run in a *process*. Within a program, you can also have independent parts that run simultaneously in separate *threads*. Because threads can run simultaneously, there's no inherent guarantee about the order in which parts of your code on different threads will run. This can lead to:
    - Race conditions, in which threads are accessing data or resources in an inconsistent order
    - Deadlocks, in which two threads are waiting for each other, preventing both threads from continuing
    - Bugs that only happen in certain situations and are hard to reproduce and fix reliably
- Rust standard library uses a *1:1* model of thread implementation, whereby a program uses one operating system thread per one language thread. There are crates that implement other models of threading that make different trade-offs to the 1:1 model, and Rust's async system provides another approach to concurrency.

- To create a new thread, we call `thread::spawn` and pass it a closure containing the code we want to run in the new thread. In Rust when the main thread of the program completes, all spawned threads are shutdown whether or not they've finished running. The following thread will only make it up to 4 or 5:

```rust
use std::thread;
use std::time::Duration;

fn main() {
    thread::spawn(|| {
        for i in 1..10 {
            println!("hi number {i} from the spawned thread!");
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("hi number {i} from the main thread!");
        thread::sleep(Duration::from_millis(1));
    }
}
```

- The return type of `thread::spawn` is a `JoinHandle<T>`. A `JoinHandle<T>` is an owned value that will wait for the thread to finish when we call the `join` method on it. The `join` call itself returns a `Result<T, Box<dyn Any + Send>>` because the spawned thread might have returned a value or panicked. To make sure our thread prints to ten, we can call `join`. We `unwrap` the result to propagate any panic or extract the `Ok(val)` (in this case just `Ok(())`):

```rust
use std::thread;
use std::time::Duration;

fn main() {
    let handle = thread::spawn(|| {
        for i in 1..10 {
            println!("hi number {i} from the spawned thread!");
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("hi number {i} from the main thread!");
        thread::sleep(Duration::from_millis(1));
    }

    handle.join().unwrap();
}
```

- Calling `join` on the handle blocks the thread currently running until the thread represented by the handle terminates.
- We'll often use the `move` keyword with closures passed to `thread::spawn` because the closure will then take ownership of the values it uses from the environment. To use data from the main thread in a spawned thread, the spawned thread's closure must capture the values it needs.

```
// DOES NOT COMPILE! error[E0373]: closure may outlive the current function,
//                                 but it borrows `v`, which is owned by the
//                                 current function
use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(|| {
        println!("Here's a vector: {v:?}");
    });

    handle.join().unwrap();
}
```

- Rust infers how to capture `v`, and because `println!` only needs a reference to `v`, the closure tries to borrow `v`. However, Rust doesn't know how long the spawned thread will run so cannot be sure the reference to `v` will always be valid. For example:

```
// DOES NOT COMPILE!
use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(|| {
        println!("Here's a vector: {v:?}");
    });

    drop(v); // oh no!

    handle.join().unwrap();
}
```

- By using the `move` keyword, we force the closure to take ownership of the values it is using.

```
use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(move || {
        println!("Here's a vector: {v:?}");
    });

    handle.join().unwrap();
}
```

- Now if we tried to `drop(v)` after passing ownership to the closure, we would get `error[E0382]: use of moved value: `v`` .
- In Rust, the move keyword on a closure doesn't mean "invalidate the outer variable." It means: "Capture variables by value instead of by reference (`&` or `&mut`)."
    - For non-`Copy` types (like `String` , `Vec` , `Box`): Capturing "by value" transfers ownership. The outer variable loses all permissions and can no longer be used.
    - For `Copy` types (like `i32` , `bool` , `char`): Capturing "by value" just makes an independent bitwise copy! The outer variable in main stays completely valid and retains its Read, Write, and Own permissions.
- So the following example has the unexpected result of printing 2.

```rust
use std::thread;

fn main() {
    let mut n = 1;

    let t = thread::spawn(move || {
        n = n + 1;
        thread::spawn(move || {
            n = n + 1;
        })
    });

    n = n + 1;
    t.join().unwrap().join().unwrap();
    println!("{n}");
}
```

- Intuition might suggest that `n` wouldn't be allowed to be used after moving, which is 100% correct for non-`Copy` types like `String` or `Box`.
- The quiz was specifically testing whether one remembered that `move` on a `Copy` type is just a duplicate bitwise copy.

16.2 - Transfer Data Between Threads with Message Passing

- Rust's standard library provides an implementation of channels for message-sending concurrency. We create a new channel using `mpsc::channel` function; `mpsc` stands for *multiple producer, single consumer*. The function returns a tuple where the first element is the transmitter and the second element is the receiver. The abbreviations `tx` and `rx` are traditionally used for *transmitter* and *receiver*.

```rust
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let val = String::from("hi");
        tx.send(val).unwrap();
    });

    let received = rx.recv().unwrap();
    println!("Got: {received}");
}
```

- We use `move` to transfer ownership of `tx`, because the spawned thread needs to own the transmitter to be able to send messages through the channel. The transmitter has a `send` method that returns a `Result<T, E>` type. It results in an error if the receiver has already been dropped and there's nowhere to send a value. After we sent a value down a channel, we transfer ownership of that value to the receiver, and we can no longer use it in the thread.

- The receiver has both `recv` and `try_recv` methods. `recv` will block the main thread's execution and wait until a value is sent down the channel. Once a value is sent, `recv` will return it in a `Result<T, E>`. When the transmitter closes, `recv` will return an error to signal that no more values will be coming. The `try_recv` method doesn't block and returns a `Result<T, E>` immediately: an `Ok` holding a value if one is available, and an `Err` if there aren't any messages this time. Using `try_recv` is useful if this thread has other work to do while waiting for messages: we could write a loop that calls `try_recv` every so often, handles a message if one is available, and otherwise does other work before checking again.

```rust
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let vals = vec![
            String::from("hi"),
            String::from("from"),
            String::from("the"),
            String::from("thread"),
        ];

        for val in vals {
            tx.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });

    for received in rx {
        println!("Got: {received}");
    }
}
```

- When you treat `rx` as an iterator, it blocks waiting for a value like manually calling `recv` and you don't need to `unwrap` it.
- To have multiple producers, you call `tx.clone()`.

```
// --snip--

let (tx, rx) = mpsc::channel();

let tx1 = tx.clone();
thread::spawn(move || {
    let vals = vec![
        String::from("hi"),
        String::from("from"),
        String::from("the"),
        String::from("thread"),
    ];

    for val in vals {
        tx1.send(val).unwrap();
        thread::sleep(Duration::from_secs(1));
    }
});

thread::spawn(move || {
    let vals = vec![
        String::from("more"),
        String::from("messages"),
        String::from("for"),
        String::from("you"),
    ];

    for val in vals {
        tx.send(val).unwrap();
        thread::sleep(Duration::from_secs(1));
    }
});

for received in rx {
    println!("Got: {received}");
}

// --snip--
```

- And here is an example of bidirectional communication. The output is 1.

```rust
use std::{sync::mpsc, thread};

enum ClientMessage { Incr, Get, Quit }
enum ServerMessage { Get(usize) }

fn main() {
    let (server_tx, client_rx) = mpsc::channel();
    let (client_tx, server_rx) = mpsc::channel();
    let server = thread::spawn(move || {

        let mut n = 0;
        loop {
            match server_rx.recv().unwrap() {
                ClientMessage::Quit => break,
                ClientMessage::Incr => n += 1,
                ClientMessage::Get => {
                    server_tx.send(ServerMessage::Get(n)).unwrap()
                }
            }
        }
    });

    for msg in [
        ClientMessage::Incr,
        ClientMessage::Get,
        ClientMessage::Quit,
    ] {
        client_tx.send(msg).unwrap();
    }

    if let ServerMessage::Get(n) = client_rx.recv().unwrap() {
        println!("{}", n)
    }

    server.join().unwrap();
}
```

- (Not in the book) `std::sync::mpsc` lacks a multiplexing `select` feature like Go's `select` statement. In real-world Rust, external crates solve this: `crossbeam-channel` provides a `crossbeam_channel::select!` macro for multi-channel synchronous code, and `tokio::select!` provides it for async.

16.3 - Shared-State Concurrency

- *Mutex* is an abbreviation for *mutual exclusion*, as in a mutex allows only one thread to access some data at any given time. To access the data in a mutex, a thread must first signal that it wants access by asking to acquire the mutex's lock. The *lock* is a data structure that is part of the mutex that keeps track of who currently has exclusive access to the data. Therefore, the mutex is described as *guarding* the data it holds via the locking system. When using a mutex there are two rules:

    1. You must attempt to acquire the lock before using the data.
    2. When you're done with the data that the mutex guards, you must unlock the data so that other threads can acquire the lock.
- Channels in any language are much like single ownership because once you send a value down a channel, you should no longer use that value. Shared-memory concurrency is like multiple ownership: multiple threads can access the same memory location at the same time. Rust's type system and ownership rules greatly assist in taming the complexity of multiple ownership. Rust believes in "fearless concurrency" but this is simply common primitives with some complexity tamed by ownership rules. The breakthrough isn't new concurrency primitives - it's that Rust's standard ownership rules apply across thread boundaries, which completely eliminates data races at compile time.

- Management of mutexes can be incredibly tricky to get right, which is why so many people are enthusiastic about channels. However, thanks to Rust's type system and ownership rules, you can't get locking and unlocking wrong.

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(5);

    {
        let mut num = m.lock().unwrap();
        *num = 6;
    }

    println!("m = {m:?}");
}
```

- We create a `Mutex` with `Mutex::new`. To access the data inside, we call the `lock` method to acquire the lock. This call will block the current thread so that it can't do any work until

it's our turn to have the lock. The call to `lock` would fail if another thread holding the lock panicked. In that case, no one would ever be able to get the lock, so we've chosen to `unwrap` and have this thread panic if we're in that situation. After we have acquired the lock, we can treat the value as a mutable reference to the data inside. The call to `lock` returns a type called `MutexGuard` wrapped in a `LockResult` that we handled with the call to `unwrap`. The `MutexGuard` implements `Deref` and `DerefMut` to provide mutable access to our inner data; the type also has a `Drop` implementation that releases the lock automatically when a `MutexGuard` goes out of scope, which happens at the end of the inner scope. So the lock release is automatic. Note that due to interior mutability, we can pass around immutable references to a `Mutex<T>` and they can all be locked and unlocked. `Mutex<T>` is basically a multi-threaded `RefCell<T>`.

- `Arc<T>` is a type like `Rc<T>` that is safe to use in concurrent situations. The `a` stands for *atomic*, meaning it's an *atomically reference-counted* type. `Arc` and `Rc` share the same API.

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();

            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("Result: {}", *counter.lock().unwrap());
}
```

- Using a `Mutex<T>` for simple numerical operations is overkill when there are simpler types provided by `std::sync::atomic`, but this pattern would be useful if we were dividing a

bigger job into parts and using a non-primitive type.

- Rust helps with concurrency problems, but it cannot stop you from creating deadlocks in code using `Mutex<T>`.
- The following example is unsafe and forbidden by the compiler, because an `Arc` is still not safe to use if its data contains a reference, since the reference could (in theory) be invalidated before all threads containing the `Arc` finish executing.

```
use std::{sync::Arc, thread};

fn main() {
    let s = String::from("Hello world");
    let a = Arc::new(&s);
    let a2 = Arc::clone(&a);
    let t = thread::spawn(move || a2.len());
    let len = t.join().unwrap();
    println!("{} {}", a, len);
}
```

- `thread::spawn` requires the closure and everything it captures to be `'static`. Because an OS thread could run indefinitely, Rust refuses to allow any references with a lifetime shorter than the entire program (`'static`) to be moved into a spawned thread.
- `Arc<T>` manages the lifetime of what it holds, so while it can technically hold a reference, it is almost always a mistake. The two easy ways to fix this are to declare the `Arc<T>` such that it owns the `String` in the first place, or pass `s` to `Arc<T>` instead of `&s` to transfer ownership to the `Arc<T>`:

```rust
use std::{sync::Arc, thread};

fn main() {
    let a = Arc::new(String::from("Hello world"));
    let a2 = Arc::clone(&a);
    let t = thread::spawn(move || a2.len());
    let len = t.join().unwrap();
    println!("{} {}", a, len);
}

// Or pass `s` by value without the `&`:
fn main() {
    let s = String::from("Hello world");
    let a = Arc::new(s); // moves ownership of `s` into `a`
    // ...
}
```

16.4 - Extensible Concurrency with Send and Sync

- Everything we have reviewed so far has been part of the standard library, not the language. Among the key concurrency concepts that are embedded in the language rather than the standard library are the `std::marker` traits `Send` and `Sync`. Despite being in `std`, they are core parts of the language with specific compiler support baked in.
- The `Send` marker trait indicates that ownership of a value of the type implementing `Send` can be transferred between threads. Almost every Rust type implements `Send`, but there are some exceptions like `Rc<T>`. Any type composed only of `Send` types is automatically marked as `Send`. Almost all primitives aside from raw pointers are `Send`.
- The `Sync` marker trait indicates that it is safe for the type implementing `Sync` to be referenced from multiple threads. In other words, any type `T` implements `Sync` if `&T` (an immutable reference to `T`) implements `Send`, meaning the reference can be sent safely to another thread. Similar to `Send`, primitive types all implement `Sync`, and types composed entirely of types that implement `Sync` also implement `Sync`. `Sync` is the most similar concept in Rust to the colloquial meaning of the phrase "thread-safe", i.e. that a particular piece of data can be safely used by multiple concurrent threads. The reason `Send` and `Sync` are separate is that a type can sometimes be one, or both, or neither. For example:
  - The smart pointer `Rc<T>` is neither `Send` nor `Sync`.
  - The `RefCell<T>` type and the family of related `Cell<T>` types are `Send` (if `T: Send`), but they are not `Sync`. A `RefCell` can be sent across a thread boundary, but

not accessed concurrently because the implementation of borrow checking that `RefCell<T>` does at runtime is not thread-safe.
    - The smart pointer `Mutex<T>` is `Send` and `Sync`, and can be used to share access with multiple threads.
    - The type `MutexGuard<'a, T>` that is returned by `Mutex::lock` is `Sync` (if `T: Sync`) but not `Send`. It is specifically not `Send` because some platforms mandate that mutexes are unlocked by the same thread that locked them.
- Because types composed entirely of other types that implement the `Send` or `Sync` traits also automatically implement `Send` and `Sync`, we don't have to implement those traits manually. As marker traits, they don't even have any methods to implement. They're just useful for enforcing invariants related to concurrency. Manually implementing these traits involves unsafe Rust code, which we're not talking about until Chapter 20. For now, it suffices to say that building new concurrent types not made up of `Send` and `Sync` parts requires careful thought to uphold the safety guarantees.
- Connecting `Send` and `Sync` back to our earlier compiler errors:
    - `Rc<T>` cannot be shared across threads because it is `!Send` and `!Sync`. Its internal reference counter uses normal non-atomic integer operations; sharing it would cause data races on the counter. `Arc<T>` uses atomic instructions, so it implements `Send` and `Sync` (provided `T: Send + Sync`).
    - `RefCell<T>` is `Send` (you can transfer an entire `RefCell` to another thread), but `!Sync`. Its borrow-checking counters are non-atomic, so multiple threads cannot hold `&RefCell<T>` simultaneously. `Mutex<T>` replaces runtime borrow flags with OS-level thread locks, making it `Sync`.
    - In other languages, thread safety is an honor system (e.g., remembering to call `mu.Lock()` before accessing shared data in Go or Java). In Rust, thread safety is enforced at compile time: you cannot send `!Send` data, share `!Sync` references, or access data in a `Mutex<T>` without locking.

16.5 - Chapter Summary: Fearless Concurrency

- Rust's standard library provides concurrency primitives where memory safety and thread safety are enforced at compile time via ownership, `Send`, and `Sync`:

Type	Scope	`Send`	`Sync`	Role / Notes
`Rc<T>`	Single	No	No	Non-atomic ref-counting
`Arc<T>`	Multi	Yes*	Yes*	Atomic ref-count (`Rc` equiv)

Type	Scope	`Send`	`Sync`	Role / Notes
`RefCell<T>`	Single	Yes*	No	Non-atomic borrow checks
`Mutex<T>`	Multi	Yes*	Yes*	Mutual exclusion (`RefCell` equiv)
`RwLock<T>`	Multi	Yes*	Yes*	N readers OR 1 writer
`Sender<T>`	Multi	Yes*	No	Cloneable channel producer
`Receiver<T>`	Multi	Yes*	No	Single channel consumer

`*` Trait implementation is conditional on the inner type `T` (e.g., `Arc<T>` is `Send` only if `T: Send + Sync`).

- **Key Takeaways**:
  - **Threads**: `thread::spawn` creates 1:1 OS threads. Its `'static` bound demands that spawned closures either take full ownership of captured data (`move || ...`) or only reference values that live for the entire program.
  - **Message Passing (Channels)**: `mpsc::channel()` enforces ownership transfer. Calling `tx.send(val)` consumes `val` by value, preventing the sending thread from accessing or mutating the data afterward.
  - **Shared State (`Mutex<T>`)**: Mutexes *wrap* their inner data rather than sitting beside it. Calling `.lock()` returns a `MutexGuard<T>` providing `Deref` and `DerefMut` to access data, and automatically unlocks the mutex via `Drop` when the guard goes out of scope.
  - **`Send` vs `Sync`**:
    - `Send` allows transferring *ownership* across thread boundaries.
    - `Sync` allows sharing *immutable references* (`&T`) across threads (i.e. `T` is `Sync` if `&T` is `Send`).
  - **Fearless, but not Flawless**: Rust eliminates data races at compile time, but cannot prevent **deadlocks** or logical race conditions.
  - As the book concludes: concurrent programming is no longer a concept to be afraid of--go forth and make your programs concurrent, fearlessly!

Chapter 17 - Async, Await, Futures, and Streams

- *Asynchronous programming* is an abstraction that lets us express our code in terms of potential pausing points and eventual results that takes care of the details of coordinating

the exact order of running different parts of the program. Rust's futures, streams, and the `async` and `await` syntax let us express how operations could be asynchronous, and there are third-party crates that implement asynchronous runtimes: code that manages and coordinates the execution of asynchronous operations.

- Your computer executes *CPU-bound* or *compute-bound* operations, where the processing power is the limit of how long the operation takes, and *I/O bound* operations that are limited by the speed of *input and output*. In both cases, the operating system's invisible interrupts provide a form of concurrency. Since we understand our programs at a more granular level than the operating system, we can spot other opportunities for concurrency. Rust's *async* abstraction gives us the ability to define a number of tasks we'd like our program to complete and allow the runtime to choose the best order and manner in which to run them.
- When an individual works on several different tasks before any of them is complete, this is *concurrency*. When a team splits up a group of tasks by having each member take one task and work on it alone, this is *parallelism*. Asynchronous code in Rust usually happens concurrently. It may also use parallelism under the hood depending on the hardware, the operating system, and the asynchronous runtime we are using.

17.1 - Futures and the Async Syntax

- The key elements of asynchronous programming in Rust are *futures* and Rust's `async` and `await` keywords. A *future* is a value that may not be ready now but will become ready at some point in the future. Rust provides a `Future` trait as a building block so that different async operations can be implemented with different data structures but a common interface. You can apply the `async` keyword to blocks and functions to specify that they can be interrupted and resumed. Within an async block or async function, you can use the `await` keyword to *await a future*. Any point where you await a future within an async block or function is a potential spot for that block or function to pause and resume. The process of checking with a future to see if its value is available yet is called *polling*. When writing async Rust, we use the `async` and `await` keywords most of the time, and Rust compiles them into equivalent code using the `Future` trait, in much the same way it compiles `for` loops into equivalent code using the `Iterator` trait. Because Rust provides the `Future` trait, you can also implement it for your own data types. Consider this simple async program that fetches two web pages concurrently and prints out whichever page finishes first.

- Futures in Rust are lazy. They don't do anything until you ask them to with the `await` keyword. `await` is a post-fix keyword that goes after the expression you're awaiting. It makes it nice for method chaining.

```rust
use trpl::Html;

async fn page_title(url: &str) -> Option<String> {
    let response = trpl::get(url).await;
    let response_text = response.text().await;
    // alternately, the last two steps could be chained
    // let response_text = trpl::get(url).await.text().await;
    Html::parse(&response_text)
        .select_first("title")
        .map(|title| title.inner_html())
}

// A simple main that calls `page_title` once:
fn main() {
    let args: Vec<String> = std::env::args().collect();

    trpl::block_on(async {
        let url = &args[1];
        match page_title(url).await {
            Some(title) => println!("The title for {url} was {title}"),
            None => println!("{url} had no title"),
        }
    })
}

// `page_title` is roughly equivalent to:
use std::future::Future;
use trpl::Html;

fn page_title(url: &str) -> impl Future<Output = Option<String>> {
    async move {
        let text = trpl::get(url).await.text().await;
        Html::parse(&text)
            .select_first("title")
            .map(|title| title.inner_html())
    }
}
```

- When Rust sees a block marked with the `async` keyword, it compiles it into a unique, anonymous data type that implements the `Future` trait. When Rust sees a function marked with `async`, it compiles it into a non-async function whose body is an async block. An async function's return type is the type of the anonymous data type the compiler creates for that async block. Thus writing `async fn` is equivalent to writing a function that returns

a future of the return type. `await` can only be used in async functions or blocks, and you cannot mark `main` as async. Asynchronous code needs a runtime. It can be initialized in main, but main itself isn't a runtime. Most languages that support async bundle a runtime, but Rust does not. Different runtimes are available and make different tradeoffs. `trpl` provides a `block_on` function that sets up a runtime using the `tokio` crate that is used to run the future passed in. It is similar to other runtime crates' `block_on` functions. Once the future completes, `block_on` returns whatever value the future produced. We could have passed `page_title` directly to `block_on` and matched on the resulting `Option<String>`, but most of the time we want to do more than make one async function call, so we pass `block_on` an `async` block and explicitly `await` the result.

- Each *await point*, that is, every place where the code uses the `await` keyword, represents a place where control is handed back to the runtime. An *executor* is the part of the runtime responsible for executing the async code.

```rust
use trpl::{Either, Html};

async fn page_title(url: &str) -> (&str, Option<String>) {
    let response_text = trpl::get(url).await.text().await;
    let title = Html::parse(&response_text)
        .select_first("title")
        .map(|title| title.inner_html());
    (url, title)
}

// Now we're (pod) racing!
fn main() {
    let args: Vec<String> = std::env::args().collect();

    trpl::block_on(async {
        let title_fut_1 = page_title(&args[1]);
        let title_fut_2 = page_title(&args[2]);

        let (url, maybe_title) =
            match trpl::select(title_fut_1, title_fut_2).await {
                Either::Left(left) => left,
                Either::Right(right) => right,
            };

        println!("{url} returned first");
        match maybe_title {
            Some(title) => println!("Its page title was: '{title}'"),
            None => println!("It had no title."),
        }
    })
}
```

- `trpl::select` returns a value to indicate which of the futures passed to it finishes first. This is simplified from the `select` in the `futures` crate.
- Here's the same example written in `tokio` and using `reqwest`:

```
use reqwest;
use std::time::Duration;

#[tokio::main]
async fn main() {
    let fut1 = reqwest::get("https://example.com");
    let fut2 = reqwest::get("https://rust-lang.org");

    tokio::select! {
        res1 = fut1 => println!("example.com finished first!"),
        res2 = fut2 => println!("rust-lang.org finished first!"),
    }
}
```

17.2 - Applying Concurrency with Async

- We will revisit some of the concurrency chapter examples using asynchronous code.

```
use std::time::Duration;

fn main() {
    trpl::block_on(async {
        trpl::spawn_task(async {
            for i in 1..10 {
                println!("hi number {i} from the first task!");
                trpl::sleep(Duration::from_millis(500)).await;
            }
        });

        for i in 1..5 {
            println!("hi number {i} from the second task!");
            trpl::sleep(Duration::from_millis(500)).await;
        }
    });
}
```

- All examples beyond this will start with the same `main` and `trpl::block_on` and opening `async {` then ending with `});` and a `}`.
- This too ends after printing no more than around 10 numbers, 5 from the `for` loop in the body of the main async block, and another 4-5 from the task spawned by `spawn_task`. When the `main` block ends, `spawn_task` is shut down. We can await the `handle` returned

by `spawn_task`. It is a `Future`. Its `Output` type is a `Result`, so we also unwrap it after awaiting it.

```
// ...
        let handle = trpl::spawn_task(async {
            for i in 1..10 {
                println!("hi number {i} from the first task!");
                trpl::sleep(Duration::from_millis(500)).await;
            }
        });

        for i in 1..5 {
            println!("hi number {i} from the second task!");
            trpl::sleep(Duration::from_millis(500)).await;
        }

        handle.await.unwrap();
```

- This version now prints all our numbers, as does the next version.
- `trpl::join` takes two `Future`s and returns a single new `Future` whose output is a tuple containing the output of each future you passed once they both complete. We don't even need to spawn a task in the last example. Just give both blocks to the runtime as `async` blocks:

```
// ...
        let fut1 = async {
            for i in 1..10 {
                println!("hi number {i} from the first task!");
                trpl::sleep(Duration::from_millis(500)).await;
            }
        };

        let fut2 = async {
            for i in 1..5 {
                println!("hi number {i} from the second task!");
                trpl::sleep(Duration::from_millis(500)).await;
            }
        };

        trpl::join(fut1, fut2).await;
```

- This differs from spawning the task or running separate threads. Where those two have some randomness, `trpl::join` is fair, meaning it checks each future equally often, alternating between them. Runtimes don't have to guarantee for any given operation, and they often offer different API's so you can choose whether or not you want fairness.
- Sharing data between futures will also feel familiar.

```
let (tx, mut rx) = trpl::channel();

let val = String::from("hi");
tx.send(val).unwrap();

let received = rx.recv().await.unwrap();
println!("received '{received}'");
```

- The synchronous `Receiver::recv` method in `std::mpsc::channel` blocks until it receives a message. The `trpl::Receiver::recv` method does not, because it is async. Instead of blocking, it hands control back to the runtime until either a message is received or the channel closes. By contrast, we don't `await` the `send` call, because it doesn't block. It doesn't need to.

```
let (tx, mut rx) = trpl::channel();

let vals = vec![
    String::from("hi"),
    String::from("from"),
    String::from("the"),
    String::from("future"),
];

for val in vals {
    tx.send(val).unwrap();
    trpl::sleep(Duration::from_millis(500)).await;
}

while let Some(value) = rx.recv().await {
    println!("received '{value}'");
}
```

- Rust can't use a `for` loop with an asynchronously produced series of items, so we used a `while let` conditional loop. This code has a big problem. Code within one async block

executes linearly. When you run this, it sends four messages over a 2000 millisecond window, then prints out all four, then hangs indefinitely since the channel was never closed. To get the behavior we want, we need to put the `tx` and `rx` operations in their own async blocks. We also call `trpl::join` to await both futures. We would still be sequential if we called `await` on one future then again on the next.

```
let tx_fut = async {
    let vals = vec![
        String::from("hi"),
        String::from("from"),
        String::from("the"),
        String::from("future"),
    ];

    for val in vals {
        tx.send(val).unwrap();
        trpl::sleep(Duration::from_millis(500)).await;
    }
};

let rx_fut = async {
    while let Some(value) = rx.recv().await {
        println!("received '{value}'");
    }
};

trpl::join(tx_fut, rx_fut).await;
```

- Now the messages print out at 500 millisecond intervals, but the program never exits still because of the interaction between the `while let` loop and `trpl::join`. To close the channel, we would call `rx.close()` or let the sender side (`tx`) drop. To call `rx.close()` we would have to know how many messages we are expecting. `tx` doesn't drop now because it's borrowed, but if we could move `tx` into the `async` block where it was used, it would be dropped once the block ends. The `move` keyword works with `async` blocks like it does with closures, so this change would let our program complete:

```
let (tx, mut rx) = trpl::channel();

let tx_fut = async move {
    // --snip--
```

- This async channel is also a multiple-producer channel, so we can call `clone` on `tx` if we want to send messages from multiple futures.

```
let (tx, mut rx) = trpl::channel();

let tx1 = tx.clone();
let tx1_fut = async move {
    let vals = vec![
        String::from("hi"),
        String::from("from"),
        String::from("the"),
        String::from("future"),
    ];

    for val in vals {
        tx1.send(val).unwrap();
        trpl::sleep(Duration::from_millis(500)).await;
    }
};

let rx_fut = async {
    while let Some(value) = rx.recv().await {
        println!("received '{value}'");
    }
};

let tx_fut = async move {
    let vals = vec![
        String::from("more"),
        String::from("messages"),
        String::from("for"),
        String::from("you"),
    ];

    for val in vals {
        tx.send(val).unwrap();
        trpl::sleep(Duration::from_millis(1500)).await;
    }
};

trpl::join!(tx1_fut, tx_fut, rx_fut);
```

- We need to switch from `trpl::join` to `trpl::join!`, which awaits an arbitrary number of futures where we know the number of futures at compile time.

17.3 - Working With Any Number of Futures

- As we know, Rust gives a runtime a chance to pause the task and switch to another one if the future being awaited isn't ready. The inverse is also true: Rust *only* pauses async blocks and hands control back to a runtime at an await point. Everything between await points is synchronous. That means your future can *starve* other futures by doing a lot of work in an async block without an await point. If you are doing expensive setup or long-running work, or if you have a future that will keep doing some particular task indefinitely, you'll need to think about when and where to hand control back to the runtime.

```
fn slow(name: &str, ms: u64) {
    thread::sleep(Duration::from_millis(ms));
    println!("'{name}' ran for {ms}ms");
}
```

- This code uses `std::thread::sleep` instead of `trpl::sleep` so that calling `slow` will block the current thread for some number of milliseconds. This is a stand in for a real-world operation that is both long-running and blocking.

```
let a = async {
    println!("'a' started.");
    slow("a", 30);
    slow("a", 10);
    slow("a", 20);
    trpl::sleep(Duration::from_millis(50)).await;
    println!("'a' finished.");
};

let b = async {
    println!("'b' started.");
    slow("b", 75);
    slow("b", 10);
    slow("b", 15);
    slow("b", 350);
    trpl::sleep(Duration::from_millis(50)).await;
    println!("'b' finished.");
};

trpl::select(a, b).await;

/* OUTPUT:
'a' started.
'a' ran for 30ms
'a' ran for 10ms
'a' ran for 20ms
'b' started.
'b' ran for 75ms
'b' ran for 10ms
'b' ran for 15ms
'b' ran for 350ms
'a' finished.  */
```

- So `a` does all its long running operations then sleeps, and `b` does all its long running operations then sleeps, and finally `a` prints `'a' finished.` and terminates first. To allow both futures to make progress between their slow tasks, we need await points so we can hand control back to the runtime. The `trpl::yield_now` function introduces a yield point without sleeping.

```
let a = async {
    println!("'a' started.");
    slow("a", 30);
    trpl::yield_now().await;
    slow("a", 10);
    trpl::yield_now().await;
    slow("a", 20);
    trpl::yield_now().await;
    println!("'a' finished.");
};

let b = async {
    println!("'b' started.");
    slow("b", 75);
    trpl::yield_now().await;
    slow("b", 10);
    trpl::yield_now().await;
    slow("b", 15);
    trpl::yield_now().await;
    slow("b", 350);
    trpl::yield_now().await;
    println!("'b' finished.");
};

/* OUTPUT:
'a' started.
'a' ran for 30ms
'b' started.
'b' ran for 75ms
'a' ran for 10ms
'b' ran for 10ms
'a' ran for 20ms
'b' ran for 15ms
'a' finished.   */
```

- In the real world, you won't be yielding every other line. It is really inexpensive to yield, but it's not free. In many cases, trying to break up a compute-bound task might make it significantly slower, so sometimes its better for overall performance to let an operation block briefly.
- It is easy to build your own async primitives. Consider the `timeout` function.

```rust
use std::time::Duration;
use trpl::Either;

async fn timeout<F: Future>(
    future_to_try: F,
    max_time: Duration,
) -> Result<F::Output, Duration> {
    match trpl::select(future_to_try, trpl::sleep(max_time)).await {
        Either::Left(output) => Ok(output),
        Either::Right(_) => Err(max_time),
    }
}

fn main() {
    trpl::block_on(async {
        let slow = async {
            trpl::sleep(Duration::from_secs(5)).await;
            "Finally finished"
        };

        match timeout(slow, Duration::from_secs(2)).await {
            Ok(message) => println!("Succeeded with '{message}'"),
            Err(duration) => {
                println!("Failed after {} seconds", duration.as_secs())
            }
        }
    });
}
```

17.4 - Streams: Futures in Sequence

- Streams are futures, so we can use them with any other kind of future and combine them in interesting way. A stream is also like an asynchronous form of iteration. We can create a stream from any iterator. `StreamExt` is a trait that gives streams a `next` method. The `Stream` trait defines a low-level interface that effectively combines the `Iterator` and `Future` traits, and the `StreamExt` trait supplies a higher-level set of APIs on top. Neither is part of the standard library, but most ecosystem crates use similar definitions.

```rust
use trpl::StreamExt;

fn main() {
    trpl::block_on(async {
        let values = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
        let iter = values.iter().map(|n| n * 2);
        let mut stream = trpl::stream_from_iter(iter);

        while let Some(value) = stream.next().await {
            println!("The value was: {value}");
        }
    });
}
```

- (Not in the book) Converting an in-memory array into a stream above is just a toy example to demonstrate syntax. In real-world code, you convert an `Iterator` into a `Stream` when you need to perform asynchronous work for each item, which standard iterators cannot do:
  - Standard `Iterator::map` only accepts synchronous closures. You cannot call `.await` inside an iterator's closure (e.g. `iter.map(|url| get(url).await)` will not compile).
  - `StreamExt::then` is stream's async version of `map`: its closure returns a `Future`, and `then` automatically awaits that future sequentially before yielding each value:

```rust
use futures::stream::StreamExt;

// Fetches URLs sequentially, awaiting each before pulling the next:
let stream = trpl::stream_from_iter(urls)
    .then(|url| async move {
        reqwest::get(&url).await
    });
```

- `StreamExt::map` vs. `StreamExt::then` and controlled concurrency:
  - `StreamExt::map` is synchronous. Calling an `async fn` without `.await` (e.g. `reqwest::get(url)`) returns an un-polled, lazy `Future`.
  - By using `map` to create a stream of futures, we can chain `.buffer_unordered(N)` to run up to `N` futures in parallel with automatic backpressure:

```rust
use futures::stream::StreamExt;

// Runs up to 10 HTTP requests concurrently:
let results = trpl::stream_from_iter(urls)
    .map(|url| reqwest::get(url)) // stream of un-polled futures
    .buffer_unordered(10)         // executes up to 10 in parallel
    .collect::<Vec<_>>()
    .await;
```

Method	Closure returns	Stream yields	Execution
`map(f)`	`T` (or raw `Future`)	`T`	Synchronous
`then(f)`	`Future<Output = T>`	`T`	Sequential*
`buffer_unordered`	Stream of futures	`T`	Concurrent (N)

`*` `then(f)` awaits each future one-by-one before yielding its output.

17.5 - A Closer Look at the Traits for Async

- Here is how Rust defines the `Future` trait and associated `Poll` enum:

```rust
use std::pin::Pin;
use std::task::{Context, Poll};

pub trait Future {
    type Output;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) ->
Poll<Self::Output>;
}

// `Poll` is a lot like `Option`
pub enum Poll<T> {
    Ready(T),
    Pending,
}
```

- It is rare to need to call `poll` directly. Should you need to, remember that the caller should not call `poll` again after the future has returned `Ready`. Many futures will panic. `await` calls desugar into something like matching on a `poll` call to where the results are

processed or `Pending` causes it to loop and try again, but really it isn't a hard loop since `Pending` really hands off control for a bit to the async runtime. Put simply, a runtime polls each future it is responsible for, putting the future back to sleep when it is not ready yet.

- Consider where we called `join!` on three `Future`s. It is more common to put your `Future`s in a `Vec` and call `join_all`. So consider this example:

```
// DOES NOT COMPILE! error[E0277]: `dyn Future<Output = ()>` cannot be
//                                 unpinned
        let tx_fut = async move {
            // --snip--
        };

        let futures: Vec<Box<dyn Future<Output = ()>>> =
            vec![Box::new(tx1_fut), Box::new(rx_fut), Box::new(tx_fut)];

        trpl::join_all(futures).await;
```

- The `Pin` type guarantees its value won't be moved in memory. This is our first time seeing a type annotation for `self`. It works like type annotations for other function parameters with two key differences:
    - It tells Rust what type `self` must be for the method to be called.
    - It can't be just any type. It's restricted to the type on which the method is implemented, a reference or smart pointer to that type, or a `Pin` wrapping a reference to that type.
- `Pin` is a wrapper for types that implement the `Deref` or `DerefMut` traits such as `&`, `&mut`, `Box`, and `Rc`. It has no behavior of its own. Behind the scenes, Rust is representing asynchronous code as a big state machine. A `Future` can easily end up with a reference to itself. But if you want to move a `Future` into a `Vec`, it might have to be moved in memory. Any object with a reference to itself is unsafe to move, since the reference points to the actual memory address of what it refers to. Theoretically, Rust could update these internal references, but that would come with a lot of performance overhead. Instead, we would like it if the data structure would not move in memory so the contents would not be moved. When we *pin* a value by wrapping a pointer to that value in `Pin`, it can no longer move. Thus if you have `Pin<Box<SomeType>>`, you actually pin the `SomeType` value, not the `Box` pointer. `Unpin` is a marker trait that tells the compiler that a given type does not need to uphold any guarantees about whether the value in question can be safely moved. `Unpin` is

implemented automatically for all types where it can prove it is safe. `Unpin` not being implemented is a special case notated `impl !Unpin for SomeType`.

- So two things to keep in mind about the relationship between `Pin` and `Unpin`. First, `Unpin` is the "normal" case, and `!Unpin` is the special case. Second, whether a type implements `Unpin` or `!Unpin` only matters when you're using a pinned pointer to that type like `Pin<&mut SomeType>`.
- Consider a concrete example of a `String`. It has no internal references so implements `Unpin`. If we wrap `String` in `Pin`, it is perfectly safe to move it around in memory, even replacing it with another `String`.
- The core problem addressed by pinning is self-reference, or a future which contains a pointer to itself. This happens when an async block contains a local variable that refers to another local variable in the future.
- Now back to our earlier example that does not compile. We tried to stick anything that implements `Future` into a `Vec`, but these `Future`s might have internal references, so they don't automatically implement `Unpin`. We can call the `pin!` macro to pin our `Future`s though, so they won't be moved:

```
use std::pin::{Pin, pin};

// --snip--
        let tx1_fut = pin!(async move {
            // --snip--
        });

        let rx_fut = pin!(async {
            // --snip--
        });

        let tx_fut = pin!(async move {
            // --snip--
        });

        let futures: Vec<Pin<&mut dyn Future<Output = ()>>> =
            vec![tx1_fut, rx_fut, tx_fut];

        trpl::join_all(futures).await;
```

- `Pin` and `Unpin` are mostly important for designing lower-level libraries or building an async runtime rather than day-to-day Rust code, but basic understanding of them can help you understand certain error messages.
- Streams are similar to asynchronous iterators. They are not defined by the standard library, but the `futures` crate used throughout the ecosystem has a common definition. Consider this definition which combines what we know about iterators and futures along with a little of what we just learned about `Pin`.

```
use std::pin::Pin;
use std::task::{Context, Poll};

trait Stream {
    type Item;

    fn poll_next(
        self: Pin<&mut Self>,
        cx: &mut Context<'_>
    ) -> Poll<Option<Self::Item>>;
}

trait StreamExt: Stream {
    async fn next(&mut self) -> Option<Self::Item>
    where
        Self: Unpin;

    // other methods...
}
```

- `StreamExt` is automatically implemented for any type that implements `Stream` so you only need to implement `Stream` for your own data types to inherit a lot of useful methods. Just remember to `use futures::stream::StreamExt;`.

17.6 - Putting It All Together: Futures, Tasks, and Threads

- Thread based concurrency is good, but an OS thread can consume a lot of memory and an embedded system may not even have an OS to manage threads. In the async system a task is similar to a thread but managed by the runtime. Tasks are similar to lightweight, runtime-managed threads with added capabilities. Thread based concurrency can be more straightforward though. Many runtimes combine the two approaches and use an approach

called *work stealing* to transparently move tasks around between threads based upon how the threads are being utilized to improve overall performance.

- When thinking about thread based concurrency versus async, consider these rules of thumb:
  - If the work is very *parallelizable* (that is, CPU-bound), such as processing a bunch of data where each part can be processed separately, threads are a better choice.
  - If the work is very *concurrent* (that is, I/O bound), such as handling messages from a bunch of different sources that may come in at different intervals or different rates, async is a better choice.
- You don't have to choose though. You can combine the two approaches and use each for what its best at.

```
use std::{thread, time::Duration};

fn main() {
    let (tx, mut rx) = trpl::channel();

    thread::spawn(move || {
        for i in 1..11 {
            tx.send(i).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });

    trpl::block_on(async {
        while let Some(message) = rx.recv().await {
            println!("{message}");
        }
    });
}
```

- In practice, you might spawn a dedicated thread for a set of video encoding tasks (because video encoding is compute-bound) but notify the UI that those are done with an async channel.

17.7 - Chapter Summary: Asynchronous Programming

- Rust's asynchronous model separates language primitives (`Future` , `Pin` , `Unpin` , `Poll` , `async` , `await`) from runtime execution. `std` intentionally omits an async runtime, timer,

and network driver to remain suitable for `#![no_std]` bare-metal targets.

- (Not in the book) The book uses a pedagogical facade crate (`trpl`). In real-world production Rust, the Tokio and Futures ecosystems are the de facto standard:

`trpl` helper	Real-world production equivalent	Crate
`block_on(fut)`	`#[tokio::main] async fn main()`	`tokio`
`spawn_task(fut)`	`tokio::spawn(fut)`	`tokio`
`sleep(dur)`	`tokio::time::sleep(dur)`	`tokio`
`yield_now()`	`tokio::task::yield_now()`	`tokio`
`channel()`	`tokio::sync::mpsc::channel(buffer)`	`tokio`
`join(a, b)` / `join!`	`tokio::join!(a, b)`	`tokio`
`join_all(vec)`	`futures::future::join_all(vec)`	`futures`
`select(a, b)`	`tokio::select!` (macro, no `Either`)	`tokio`
`stream_from_iter(it)`	`futures::stream::iter(it)`	`futures`
`StreamExt`	`futures::stream::StreamExt`	`futures`
`get(url)`	`reqwest::get(url)`	`reqwest`

- **Key Takeaways**:
  - **Lazy Futures**: Futures in Rust do no work until polled. Calling an `async fn` constructs a state-machine struct; `.await` drives it to its next suspension point or completion.
  - **Cooperative Multitasking**: Everything between `.await` points executes synchronously. A compute-heavy loop without an await point starves other futures on the thread. Long operations must call `yield_now().await` or be offloaded to dedicated threads via `tokio::task::spawn_blocking` .
  - **Streams vs. Iterators**:
    - An `Iterator` is synchronous: `next()` gives an item immediately or blocks the thread.
    - A `Stream` is asynchronous: `next()` returns a `Future` yielding an item or pausing the task until data arrives.
    - `StreamExt::then` is the async counterpart to `map` (awaits each future sequentially); chaining `map` with `.buffer_unordered(N)` executes up to `N`

futures concurrently with backpressure.

- **Pinning (`Pin` / `Unpin`)**:
  - Futures containing local references across `.await` points become self-referential structs. Moving them would invalidate internal pointers.
  - `Pin<P<T>>` guarantees that `T` will never be moved again in memory before it is dropped.
  - `Unpin` is the default auto-trait for almost all standard Rust types. Pinning only restricts types explicitly marked `!Unpin` (like compiler- generated futures).
- **Threads vs. Async Tasks**:
  - **Threads** (`std::thread::spawn`): Best for *CPU-bound* / compute-heavy parallelism (video encoding, cryptography, numeric simulation).
  - **Async Tasks** (`tokio::spawn`): Best for *I/O-bound* concurrency (thousands of web sockets, network requests, database connections).
  - In real architectures, both are combined: an async application offloads heavy CPU work to worker threads while handling I/O with async tasks.

Chapter 18 - Object Oriented Programming Features

18.1 - Characteristics of Object-Oriented Languages

- Object-oriented programming (OOP) languages share certain common characteristics: objects, encapsulation, and inheritance. Rust is influenced by many programming paradigms, including OOP. Let's look at how Rust supports these various OOP characteristics.
- Structs are our objects. They package both data and the procedures that operate on that data.
- The option to use the `pub` keyword or not for different parts of code enables encapsulation of implementation details in Rust.
- If a language must have inheritance to be object oriented, Rust is not an object-oriented language. Rust does support polymorphism in the form of *bounded parametric polymorphism* through using generics to abstract over different possible types and trait bounds to impose constraints. Rust also supports trait objects instead of inheritance to achieve polymorphism at runtime, and we will look at those next.

18.2 - Using Trait Objects to Abstract over Shared Behavior

- If you need your vector to hold more than one type and have a finite list of types (say integers, floats, and text for a spreadsheet cell), it is fine to define an enum such that each of its variants holds a different type and then make a vector of enums. Sometimes we want our library user to be able to extend the types. For example, we might have a GUI library that calls a `draw` method on each object in our vector. We could include basic types like `Button` or `TextField`, but a user might want to add an `Image`, `SelectBox`, or `CustomTextField`. With inheritance this could easily be implemented with a `Component` class with a `draw` method that the other classes would inherit from. Rust doesn't have inheritance, but it has trait objects, and the purpose of trait objects is to allow abstraction across common behavior.

```rust
// our gui library in the gui project
// src/lib.rs
pub trait Draw {
    fn draw(&self);
}

pub struct Screen {
    pub components: Vec<Box<dyn Draw>>,
}

impl Screen {
    pub fn run(&self) {
        for component in self.components.iter() {
            component.draw();
        }
    }
}

pub struct Button {
    pub width: u32,
    pub height: u32,
    pub label: String,
}

impl Button {
    pub fn click(&self) {
        // Code to click
    }
}

impl Draw for Button {
    fn draw(&self) {
        // code to actually draw a button
    }
}

// in user code which is why `use`
// traits must be brought into scope to implement or even invoke their
methods
// src/lib.rs
use gui::Draw;

struct SelectBox {
    width: u32,
    height: u32,
```

```
    options: Vec<String>,
}

impl Draw for SelectBox {
    fn draw(&self) {
        // code to actually draw a select box
    }
}

// Now in the user's main function where we don't even need `Draw`
// if we wanted to call `draw` directly, we'd need to use the trait
// but we just need `Screen` for a `run` call
// src/main.rs
use gui::{Button, Screen};
use myapp::SelectBox;

fn main() {
    let screen = Screen {
        components: vec![
            Box::new(SelectBox {
                width: 75,
                height: 10,
                options: vec![
                    String::from("Yes"),
                    String::from("Maybe"),
                    String::from("No"),
                ],
            }),
            Box::new(Button {
                width: 50,
                height: 10,
                label: String::from("OK"),
            }),
        ],
    };

    screen.run();
}
```

- This works differently from defining a struct that uses a generic type parameter with trait bounds. A generic type parameter in a container or struct can be substituted with only one concrete type at a time (e.g. a `Screen` could only hold all `Button`s or all `TextField`s), whereas trait objects allow multiple concrete types to fill in for the trait object in the same collection at runtime. If you'll only ever have homogeneous collections, using generics and

trait bounds is preferable because the definitions will be monomorphized at compile time to use the concrete types.

Feature	Trait Bound (`<T: Trait>`)	Trait Object (`dyn Trait`)
Dispatch	Static (compile time)	Dynamic (runtime vtable)
Inlining	Fully inlineable	Mostly prevents inlining
Collections	Homogeneous (single type)	Heterogeneous (mixed type)
Code Size	Monomorphization bloat	Single shared vtable
Memory	Sized, direct stack/inline	Fat ptr (data + vtable)

- If we tried to initialize our `Screen` with `components: vec![Box::new(String::from("Hi"))],` instead, we would get an `error[E0277]: the trait bound `String: Draw` is not satisfied`.
- Trait objects interact poorly with type inference sometimes.

```
// DOES NOT COMPILE! error[E0308]: mismatched types
fn main() {
  let components = vec![
        Box::new(SelectBox { /* .. */ }),
        Box::new(Button { /* .. */ }),
    ];
    let screen = Screen { components };
    screen.run();
}
```

- We can fix this by adding type annotations in one of two different places:

```rust
// annotate the first element
let components = vec![
    Box::new(SelectBox { /* .. */ }) as Box<dyn Draw>,
    Box::new(Button { /* .. */ }),
];

// or annotate the `Vec` declaration
let components: Vec<Box<dyn Draw>> = vec![
    Box::new(SelectBox { /* .. */ }),
    Box::new(Button { /* .. */ }),
];
```

- In general, it is good to be aware that using trait objects can cause a worse developer experience for API clients in the case of type inference.
- Unlike generics which use *static dispatch* and know what method you're calling at compile time, using trait objects relies on dynamic dispatch and carries a runtime cost while also preventing inlining our method's code.

18.3 - Implementing an Object-Oriented Design Pattern

- The *state pattern* is an object-oriented design pattern. The crux of the pattern is that we define a set of states a value can have internally. The states are represented by a set of *state objects*, and the value's behavior changes based on its state. We will work through an example of a blog post struct that has a field to hold its state, which will be a state object that takes on values from the set "draft", "review", or "published."
- Each state object is responsible for its own behavior and for governing when it should change into another state. The value that holds a state object knows nothing about the different behavior of the states or when to transition between states. In Rust, we use structs and traits rather than objects and inheritance. The advantage of the state pattern is that when the business requirements of the program change, we won't need to change the code of the value holding the state or the code that uses the value. We'll only need to update the code inside one of the state objects to change its rules, or perhaps add more state objects.
- We're going to do this two ways. First we will try to implement the state pattern in a more traditional object-oriented way. Then, we'll use an approach that is a bit more natural to Rust.

- The final functionality will look like this:

    1. A blog post starts as an empty draft.
    2. When the draft is done, a review of the post is requested.
    3. When the post is approved, it gets published.
    4. Only published blog posts return content to print so that unapproved posts can't accidentally be published.
- Any other changes attempted on a post should have no effect. For example, if we try to approve a draft blog post before we've requested a review, the post should remain an unpublished draft. It is important to remember users don't interact with the state objects. The states change in response to the methods called by our library's users on the `Post` instance, but they don't have to manage the state changes directly. That way users can't make a mistake with the states.

- Note: I'm presenting each project at its completion, but for a step-by-step of building this out one assert at a time see: https://rust-book.cs.brown.edu/ch18-03-oo-design-patterns.html.

```
// "blog" crate
// src/main.rs
use blog::Post;

// Before a `Post` returns `content` it must be reviewed and approved.
fn main() {
    let mut post = Post::new();

    post.add_text("I ate a salad for lunch today");
    assert_eq!("", post.content());

    post.request_review();
    assert_eq!("", post.content());

    post.approve();
    assert_eq!("I ate a salad for lunch today", post.content());
}

// src/lib.rs
// `Post` is a public struct with private fields and `new` is a public
method.
pub struct Post {
    // this isn't "optional", but we need to use `take` to transfer ownership
    state: Option<Box<dyn State>>,
    content: String,
}

impl Post {
    pub fn new() -> Post {
        Post {
            state: Some(Box::new(Draft {})),
            content: String::new(),
        }
    }

    // `add_text` doesn't interact with `state` and always works the same.
    pub fn add_text(&mut self, text: &str) {
        self.content.push_str(text);
    }

    // the rule lives in the `state` so we delegate.
    // `as_ref` here returns an `Option<&Box<dyn State>>`.
    // `unwrap` gives us a `&Box<dyn State>`.
    // through deref coercion, we can call content on the `&Box` to call
    // content on the reference to the State.
```

```
    pub fn content(&self) -> &str {
        self.state.as_ref().unwrap().content(self)
    }

    // public `request_review` calls private `request_review` on our `state`.
    // `take` steals the value out of an `Option` and leaves `None` in its
    // place. we use `take` so we can transfer ownership here.
    pub fn request_review(&mut self) {
        if let Some(s) = self.state.take() {
            self.state = Some(s.request_review());
        }
    }

    // We have two methods that use a similar pattern. If we had a lot more
    // we could consider implementing a macro for this behavior.
    pub fn approve(&mut self) {
        if let Some(s) = self.state.take() {
            self.state = Some(s.approve());
        }
    }
        }
    }
}

trait State {
    // We can't give default implementations for these two, because they need
    // to return `self` and it is dynamically typed.
    fn request_review(self: Box<Self>) -> Box<dyn State>;
    fn approve(self: Box<Self>) -> Box<dyn State>;
    fn content<'a>(&self, post: &'a Post) -> &'a str {
        ""
    }
}

struct Draft {}

impl State for Draft {
    // this method consumes the current `state` and returns a new `state`.
    // this is called on a `Box` holding the type.
    fn request_review(self: Box<Self>) -> Box<dyn State> {
        Box::new(PendingReview {})
    }

    fn approve(self: Box<Self>) -> Box<dyn State> {
        self
    }
```

```
}

struct PendingReview {}

impl State for PendingReview {
    fn request_review(self: Box<Self>) -> Box<dyn State> {
        self
    }

    fn approve(self: Box<Self>) -> Box<dyn State> {
        Box::new(Published {})
    }
}

struct Published {}

impl State for Published {
    fn request_review(self: Box<Self>) -> Box<dyn State> {
        self
    }

    fn approve(self: Box<Self>) -> Box<dyn State> {
        self
    }

    // Explicit lifetime since we return a borrowed slice from `post`.
    fn content<'a>(&self, post: &'a Post) -> &'a str {
        &post.content
    }
}
```

- Rust method receivers are not limited to `&self`, `&mut self`, or `self`. Rust also supports smart-pointer receivers (**arbitrary self types**):
  - `self: Box<Self>`: consumes the heap box and its allocation. This is required when a method on a trait object needs to take ownership by value: because `dyn State` is unsized (`!Sized`), plain `self: Self` would violate object safety (dyn compatibility). `Box<Self>` has a fixed pointer size, so it preserves object safety while allowing the method to consume the state. Methods with this receiver can only be called on boxed values.
  - `self: Rc<Self>` / `self: Arc<Self>`: takes a reference-counted pointer; common in concurrent/async code when a method spawns a task that needs its own clone of the enclosing `Arc`.

    - `self: Pin<&mut Self>`: used by the `Future` trait (`poll`) to guarantee the underlying struct will never move in memory across `.await` points.
- In contrast, `content(&self, post: &Post)` uses plain `&self`. When called via `self.state.as_ref().unwrap().content(self)`, `.as_ref()` borrows the inner value as `&Box<dyn State>`. Rust's **deref coercion** then automatically dereferences `&Box<dyn State>` to `&dyn State`, so `content` receives a borrowed reference to the inner state without moving it out of the box.

- You normally cannot move a value out of a borrowed reference, even if it is `mut`. Whenever you see `take()` on an `Option` or `std::mem::take()` / `std::mem::replace()`, it almost always means: "I have a `&mut` reference, but I need to temporarily take ownership of one of its fields."

- You could also use an enum for this, but then every place that checks the value of the enum will need a `match` expression or similar to handle every possible variant. In Rust, enum variants are not distinct types, so you cannot implement traits or methods on individual variants (e.g. no `impl Draft`). Instead, all methods live on the parent enum and must match on `self`.

- This highlights the classic **Expression Problem** trade-off between the object-oriented trait-object pattern and the functional enum pattern:

Feature	Trait Objects (`dyn State`)	Enums (`match self`)
New States	Easy: add struct + impl	Hard: update every match
New Ops	Hard: update trait + types	Easy: add one method
Dispatch	Dynamic via vtable pointer	Static direct branching
Memory	Heap (`Box`), `Option`	Stack inline, no `Option`

  With trait objects, state-specific behavior is encapsulated per struct and dispatched via vtables without conditional branching. With enums, you must write repetitive `match` arms, but you eliminate heap allocation (`Box`), `Option`, `.take()`, and dynamic dispatch indirection.

- The struct/trait approach is extensible in the sense that an API client could potentially create a new state (such as `Retracted`) without changing the core API functionality. When adding this state, the methods for other states do not need to be changed. Whereas with

enums, a client cannot add a new branch to the enum. Moreover, all match expressions must be updated when a state is added.

- Now we're going to look at implementing the same things, but doing it in a way that takes more advantage of Rust's strengths: making invalid states and transitions into compile-time errors by encoding states and behavior as types. Rather than encapsulating states so outside code has no knowledge of them, we encode the states as distinct types. Rust's type system then prevents attempts to use draft posts where published posts are required by issuing a compiler error. Instead of `content()` returning `""` for a draft, a `DraftPost` doesn't even have a `content` method to call.

```rust
// "blog" crate
// src/main.rs
fn main() {
    let mut post = Post::new();

    post.add_text("I ate a salad for lunch today");
    // nothing to assert here. our post doesn't even have a `content` method.

    // we are shadowing `post` as it goes through its transformations.
    let post = post.request_review();

    let post = post.approve();

    assert_eq!("I ate a salad for lunch today", post.content());
}

//  src/lib.rs
pub struct Post {
    content: String,
}

pub struct DraftPost {
    content: String,
}

pub struct PendingReviewPost {
    content: String,
}

impl Post {
    // our `new` function returns a draft post. as there's no way to set the
    // private content field, we can't construct a `Post` directly.
    pub fn new() -> DraftPost {
        DraftPost {
            content: String::new(),
        }
    }

    pub fn content(&self) -> &str {
        &self.content
    }
}

impl DraftPost {
    pub fn add_text(&mut self, text: &str) {
```

```
        self.content.push_str(text);
    }

    // `request_review` takes ownership of `self` to consume it.
    pub fn request_review(self) -> PendingReviewPost {
        PendingReviewPost {
            content: self.content,
        }
    }
}

impl PendingReviewPost {
    // We're in the same module, so we can set content directly even though
    // it's private.  `approve` takes ownership of `self` to consume it.
    pub fn approve(self) -> Post {
        Post {
            content: self.content,
        }
    }
}
```

18.4 - Ownership Inventory #4

- *Note: Ownership Inventory #4 has been moved to the Appendix at the end of these notes for high-yield review.*

18.5 - Design Trade-offs

- *Note: The Design Trade-offs challenges have been moved to the Appendix at the end of these notes for high-yield review.*

Chapter 19 - Patterns and Matching

- Patterns are a special syntax in Rust for matching against the structure of types, both complex and simple. A pattern consists of some combination of the following:
  - Literals
  - Destructured arrays, enums, structs, or tuples
  - Variables
  - Wildcards
  - Placeholders

19.1 - All the Places Patterns Can Be Used

- Patterns can be used in the arms of a `match` expression.
- You've been using patterns without realizing, because the variable name in a `let` statement is a simple pattern. You can see the pattern matching aspect of `let` more clearly with destructuring like: `let (x, y, z) = (1, 2, 3)`. If the number of elements in the pattern don't match the number of elements in the tuple, the overall type won't match and we'll get a compiler error `error[E0308]: mismatched types`.
- `if let` statements are like a `match` that only matches one case. Optionally it can have an `else` if the `match` fails. Rust doesn't require that the arms relate to each other in an `if let`, `else if`, and `else if let` series. You can also freely mix in standard `if` conditions. The downside of `if let` is the compiler doesn't check for exhaustiveness like a `match`.

```
fn main() {
    let favorite_color: Option<&str> = None;
    let is_tuesday = false;
    let age: Result<u8, _> = "34".parse();

    if let Some(color) = favorite_color {
        println!("Using your favorite color, {color}, as the background");
    } else if is_tuesday {
        println!("Tuesday is green day!");
    } else if let Ok(age) = age {
        if age > 30 {
            println!("Using purple as the background color");
        } else {
            println!("Using orange as the background color");
        }
    } else {
        println!("Using blue as the background color");
    }
}
```

- A `while let` conditional loop allows a `while` loop to run for as long as a pattern continues to match.

```rust
let (tx, rx) = std::sync::mpsc::channel();
std::thread::spawn(move || {
    for val in [1, 2, 3] {
        tx.send(val).unwrap();
    }
});

while let Ok(value) = rx.recv() {
    println!("{value}");
}
```

- The value directly after a `for` is a pattern. In `for x in y`, `x` is a pattern.

```rust
let v = vec!['a', 'b', 'c'];

for (index, value) in v.iter().enumerate() {
    println!("{value} is at index {index}");
}
```

- Function parameters can also be patterns. In `fn foo(x: i32)`, the `x` is a pattern. Closure parameter lists work the same.

```rust
fn print_coordinates(&(x, y): &(i32, i32)) {
    println!("Current location: ({x}, {y})");
}

fn main() {
    let point = (3, 5);
    print_coordinates(&point);
}
```

19.2 - Refutability: Whether a Pattern Might Fail to Match

- Patterns that will match for any possible value passed are *irrefutable*. Patterns that can fail to match for some possible values are *refutable*.
    - In the expression `if let Some(x) = a_value`, then `Some(x)` is refutable. If the value in the `a_value` variable is `None` rather than `Some`, the `Some(x)` pattern will not match.

    - In the expression `if let &[x, ..] = a_slice`, then `&[x, ..]` is refutable. If the value in the `a_slice` variable has zero elements, the `&[x, ..]` pattern will not match.
- Function parameters, `let` statements, and `for` loops can only accept irrefutable patterns because the program cannot do anything meaningful when values don't match. `if let`, `while let`, and `let...else` accept refutable or irrefutable patterns, but the compiler warns against irrefutable patterns. Refutability is evaluated strictly against the variable's static type, which is why we could, for example, pass a tuple to a function and destructure it.
- Using a refutable pattern where an irrefutable pattern is needed results in `error[E0005]` (specifically `refutable pattern in local binding` for `let`, `in function argument` for functions, or `in `for` loop binding`).
- If you use an irrefutable pattern where a refutable pattern is expected, you will get a `warning: irrefutable `let..else` pattern` (or other pattern). The last arm of a `match` statement can be irrefutable.

19.3 - Pattern Syntax

- You can match patterns directly against literals. This syntax is useful when you want your code to take an action if it gets a particular concrete value.
- Named variables are irrefutable patterns that match any value. If you use a variable in a `match` arm, it will declare a new variable while shadowing any existing variable with the same name and matching any value.
- In `match` expressions, you can match multiple patterns using the `|` syntax.
- The `..=` syntax allows us to match an inclusive range of values. Ranges are only allowed for numeric or `char` values.

```
let x = 5;

match x {
  1..=5 => println!("one through five"),
  _ => println!("something else"),
}
```

- We can also use patterns to destructure structs, enums, and tuples to use different parts of these values.

```rust
// structs
// it's common to match the variable names to the field names of the struct,
// but this is not required.
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 0, y: 7 };

    let Point { x: a, y: b } = p;
    assert_eq!(0, a);
    assert_eq!(7, b);
}

// shorthand with matching variable names
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 0, y: 7 };

    let Point { x, y } = p;
    assert_eq!(0, x);
    assert_eq!(7, y);
}

// you can also match literals
fn main() {
    let p = Point { x: 0, y: 7 };

    match p {
        Point { x, y: 0 } => println!("On the x axis at {x}"),
        Point { x: 0, y } => println!("On the y axis at {y}"),
        Point { x, y } => {
            println!("On neither axis: ({x}, {y})");
        }
    }
}

// enums
enum Message {
```

```rust
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

fn main() {
    let msg = Message::ChangeColor(0, 160, 255);

    match msg {
        Message::Quit => {
            println!("The Quit variant has no data to destructure.");
        }
        Message::Move { x, y } => {
            println!(
                "Move in the x direction {x} and in the y direction {y}"
            );
        }
        Message::Write(text) => {
            println!("Text message: {text}");
        }
        Message::ChangeColor(r, g, b) => {
            println!("Change color to red {r}, green {g}, and blue {b}");
        }
    }
}

// matching nested items work as well
enum Color {
    Rgb(i32, i32, i32),
    Hsv(i32, i32, i32),
}

enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(Color),
}

fn main() {
    let msg = Message::ChangeColor(Color::Hsv(0, 160, 255));

    match msg {
        Message::ChangeColor(Color::Rgb(r, g, b)) => {
            println!("Change color to red {r}, green {g}, and blue {b}");
```

```
        }
        Message::ChangeColor(Color::Hsv(h, s, v)) => {
            println!("Change color to hue {h}, saturation {s}, value {v}");
        }
        _ => (),
    }
}

// or compound items
let ((feet, inches), Point { x, y }) = ((3, 10), Point { x: 3, y: -10 });
```

- You can ignore entire values with the `_` pattern which matches everything and binds nothing or by using any name that starts with `_`. You can also ignore part of a pattern by using `_` within the pattern, or use `..` to ignore remaining parts of a value. Ignoring a function parameter sometimes makes sense when implementing traits and not needing every parameter.

```rust
// using `_` to ignore a whole value
fn foo(_: i32, y: i32) {
    println!("This code only uses the y parameter: {y}");
}

fn main() {
    foo(3, 4);
}

// using `_` as part of a pattern
let mut setting_value = Some(5);
let new_setting_value = Some(10);

match (setting_value, new_setting_value) {
    (Some(_), Some(_)) => {
        println!("Can't overwrite an existing customized value");
    }
    _ => {
        setting_value = new_setting_value;
    }
}

println!("setting is {setting_value:?}");

// using `_` to ignore multiple parts of a pattern
let numbers = (2, 4, 8, 16, 32);

match numbers {
    (first, _, third, _, fifth) => {
        println!("Some numbers: {first}, {third}, {fifth}");
    }
}

// Using `..` to ignore parts of a struct
struct Point {
    x: i32,
    y: i32,
    z: i32,
}

let origin = Point { x: 0, y: 0, z: 0 };

match origin {
    Point { x, .. } => println!("x is {x}"),
}
```

```rust
// using `..` to ignore the middle of a tuple
fn main() {
    let numbers = (2, 4, 8, 16, 32);

    match numbers {
        (first, .., last) => {
            println!("Some numbers: {first}, {last}");
        }
    }
}
```

- If you don't plan to use a variable, begin its name with an `_` and you won't get a compiler warning about not using the variable.
- The `..` pattern ignores any parts of a value that we haven't explicitly matched in the rest of the pattern. Rust will error if we use it ambiguously.
- A *match guard* is an additional `if` condition specified after the pattern in a `match` arm that must also match for that arm to be chosen. They are only available in match expressions. The compiler doesn't try to check for exhaustiveness when match guard expressions are involved. When multiple conditions are combined with `|`, the match guard applies to all the patterns.

```rust
let num = Some(4);

match num {
    Some(x) if x % 2 == 0 => println!("The number {x} is even"),
    Some(x) => println!("The number {x} is odd"),
    None => (),
}
```

- The at operator (`@`) lets us create a variable that holds a value at the same time we're testing that value for a pattern match.

```rust
enum Message {
    Hello { id: i32 },
}

let msg = Message::Hello { id: 5 };

match msg {
    Message::Hello { id: id @ 3..=7 } => {
        println!("Found an id in range: {id}")
    }
    Message::Hello { id: 10..=12 } => {
        println!("Found an id in another range")
    }
    Message::Hello { id } => println!("Found some other id: {id}"),
}
```

Chapter 20 - Advanced Features

20.1 - Unsafe Rust

Performing Unsafe Superpowers

- To switch to unsafe Rust, use the `unsafe` keyword and start a new block to hold the unsafe code. Unsafe Rust lets you:
  1. Dereference a raw pointer.
  2. Call an unsafe function or method.
  3. Access or modify a mutable static variable.
  4. Implement an unsafe trait.
  5. Access fields of `union`s.
- You should isolate unsafe code by enclosing it within a safe abstraction and providing a safe API.

Dereferencing a Raw Pointer

- You can create a *raw pointer* anywhere but can only dereference it in an unsafe block. `&raw const num` creates a `*const i32` immutable raw pointer, and `&raw mut num` creates a `*mut i32` mutable raw pointer. The `*` isn't the dereference operator; it's part of the type name.

```
let mut num = 5;

let r1 = &raw const num;
let r2 = &raw mut num;

unsafe {
    println!("r1 is: {}", *r1);
    println!("r2 is: {}", *r2);
}
```

- Raw pointers:
  - Are allowed to ignore the borrowing rules by having both immutable and mutable pointers or multiple mutable pointers to the same location
  - Aren't guaranteed to point to valid memory
  - Are allowed to be null
  - Don't implement any automatic cleanup
- By opting out of having Rust enforce these guarantees, you can give up guaranteed safety in exchange for greater performance or the ability to interface with another language or hardware where Rust's guarantees don't apply.

```
// This is legal, but not very safe.
let address =  0x012345usize;
let r = address as *const i32;
```

Calling an Unsafe Function or Method

- Unsafe functions or methods look like regular methods, but they have `unsafe` before the rest of the definition. The `unsafe` keyword in this context means the function has requirements we need to uphold when we call this function, because Rust can't guarantee we've met these requirements. By calling an unsafe function within an `unsafe` block, we're saying that we've read this function's documentation and we take responsibility for upholding the function's contracts.

```
unsafe fn dangerous() {}

unsafe {
    dangerous();
}
```

- Despite the whole function being marked as unsafe, to actually perform any unsafe operations within the body of an unsafe function, you still need to use an `unsafe` block, just as within a regular function, and the compiler will warn you if you forget. Marking the function as `unsafe` is meant to tell the caller that they need to read and uphold the contract, not that the function necessarily performs unsafe operations inside, but it isn't uncommon for unsafe functions to do unsafe things, preferably in small, marked unsafe blocks.

- Much as an unsafe function may not have any unsafe code, a safe function may use an `unsafe` block. It is common to wrap unsafe code in a safe API. We will look at `split_at_mut`.

```rust
// DOES NOT COMPILER! error[E0499]: cannot borrow `*value` as mutable more
//                                  than once at a time
// an attempt to write `split_at_mut` using safe Rust, but it won't compile
fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();

    assert!(mid <= len);

    (&mut values[..mid], &mut values[mid..])
}

// COMPILES!
// this is the actual implementation of `split_at_mut`
use std::slice;

fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();
    // here `as_mut_ptr` returns a raw pointer with the type `*mut i32`.
    let ptr = values.as_mut_ptr();

    assert!(mid <= len);

    unsafe {
        (
            // both `from_raw_parts_mut` (because it must use a raw pointer)
            // and calling `ptr.add` are unsafe operations.
            slice::from_raw_parts_mut(ptr, mid),
            slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}


fn main() {
    let mut v = vec![1, 2, 3, 4, 5, 6];

    let r = &mut v[..];

    let (a, b) = r.split_at_mut(3);

    assert_eq!(a, &mut [1, 2, 3]);
    assert_eq!(b, &mut [4, 5, 6]);
}
```

- The use of the `extern` keyword facilitates creation and use of a *Foreign Function Interface (FFI)*, which is a way for a programming language to define functions and enable a different (foreign) programming language to call those functions. Functions declared within `extern` blocks are generally unsafe to call from Rust, so `extern` blocks must also be marked unsafe. However, `abs` doesn't have any memory safety considerations, so below we specifically mark it with the `safe` keyword, so it can be called without an `unsafe` block. Marking a function `safe` doesn't make it safe! Instead, it is a promise you are making to Rust that it is safe. It is still your responsibility to make sure that promise is kept!

```
// without `safe`
unsafe extern "C" {
    fn abs(input: i32) -> i32;
}

fn main() {
    unsafe {
        println!("Absolute value of -3 according to C: {}", abs(-3));
    }
}

// with `safe`
unsafe extern "C" {
    safe fn abs(input: i32) -> i32;
}

fn main() {
    println!("Absolute value of -3 according to C: {}", abs(-3));
}
```

- Note that the above works with no special linking. Rust always links the C standard library into compilations behind the scenes, so you can define an `extern` block for any C function in the C standard library. You can also include other C libraries with `#[link(name = "libname")]` attributes on your `extern` block, but in practice you seldom see manual C `extern` blocks or link attributes. Most C libraries are packaged as `-sys` cargo crates.

- We can call Rust functions from C as well. Here `extern` is being used to apply to a specific function rather than an entire block. Also, we have to turn off compiler name mangling to keep the name consistent for calling. This only requires `unsafe` in the attribute, as the `extern` block is safe.

```rust
#[unsafe(no_mangle)]
pub extern "C" fn call_from_c() {
    println!("Just called a Rust function from C!");
}
```

Accessing or Modifying a Mutable Static Variable

- Rust global variables are called static variables. Rust supports global variables, but they can be problematic with Rust's ownership rules. For example, two threads accessing the same mutable global variable can cause a data race. Accessing an immutable static variable is sae.

```rust
static HELLO_WORLD: &str = "Hello, world!";

fn main() {
    println!("value is: {HELLO_WORLD}");
}
```

- Unlike constants, immutable static variables have a fixed address in memory. Static variables can also be mutable, and then accessing and modifying them is unsafe.

```rust
static mut COUNTER: u32 = 0;

/// SAFETY: Calling this from more than a single thread at a time is
undefined
/// behavior, so you *must* guarantee you only call it from a single thread
at
/// a time.
unsafe fn add_to_count(inc: u32) {
    unsafe {
        COUNTER += inc;
    }
}

fn main() {
    unsafe {
        // SAFETY: This is only called from a single thread in `main`.
        add_to_count(3);
        println!("COUNTER: {}", *(&raw const COUNTER));
    }
}
```

- Whenever we write an unsafe function, it is idiomatic to write a comment starting with SAFETY and explaining what the caller needs to do to call the function safely. Likewise, whenever we perform an unsafe operation, it is idiomatic to write a comment starting with SAFETY to explain how the safety rules are upheld. Clippy will warn you if you fail to put a SAFETY doc comment on an unsafe function. Clippy also enforces the private SAFETY comment on the unsafe block.

- Additionally, the compiler will deny by default any attempt to create references to a mutable static variable through a compiler lint. You must either explicitly opt out of that lint's protections by adding an `#[allow(static_mut_refs)]` annotation or access the mutable static variable via a raw pointer created with one of the raw borrow operators. That includes cases where the reference is created invisibly, as when it is used in the `println!` in this code listing. Requiring references to static mutable variables to be created via raw pointers helps make the safety requirements for using them more obvious.

- Modern Rust strongly discourages `static mut`. The canonical, idiomatic alternative for global counters and flags is an immutable `static` holding an atomic type (which implements `Sync`), avoiding `unsafe` entirely:

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

// Idiomatic: immutable static with safe interior mutability (Sync).
// AtomicUsize::new is a const fn, allowing compile-time initialization.
static REQUEST_COUNT: AtomicUsize = AtomicUsize::new(0);

fn handle_request() {
    REQUEST_COUNT.fetch_add(1, Ordering::Relaxed);
}

fn main() {
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| handle_request());
        }
    });

    println!("Total requests: {}", REQUEST_COUNT.load(Ordering::Relaxed));
}
```

- Rule of thumb for `const` vs. immutable `static`:

    - Default to `const` for ordinary constants (integers, floats, string slices, fixed configuration). Constants are inlined directly at each call site at compile time and have no fixed memory address, allowing the compiler to perform aggressive constant-folding and dead-code elimination.
    - Use an immutable `static` when you need a dedicated, fixed memory address:
        - Shared interior mutability and global synchronization across threads (e.g., `AtomicUsize`, `Mutex`, `RwLock`). Declaring a lock or atomic as `const` would instantiate a separate copy at each call site, breaking synchronization.
        - Large data structures (lookup tables, precomputed buffers) to prevent binary bloat from multiple inlined copies across compilation units.
        - Stable address identity when comparing pointers via `std::ptr::eq`.
        - C FFI symbols that require an exact, exported symbol in the binary's data segment.

Implementing an Unsafe Trait

- We can use `unsafe` to implement an unsafe trait. A trait is unsafe when at least one of its methods has some invariant that the compiler can't verify.

```
unsafe trait Foo {
    // methods go here
}

unsafe impl Foo for i32 {
    // method implementations go here
}
```

- If we implement a type that contains a type that does not implement `Send` or `Sync`, such as raw pointers, and we want to mark that type as `Send` or `Sync`, we must use unsafe. Rust can't verify that our type upholds the guarantees that it can be safely sent across threads or accessed from multiple threads; therefore, we need to do those checks manually and indicate as such with `unsafe`.

Accessing Fields of a Union

- Accessing fields of a union is the final action that works only with `unsafe`. A *union* is similar to a `struct`, but only one declared field is used in a particular instance at one time. Unions are primarily used to interface with unions in C code. Accessing union fields is unsafe because Rust can't guarantee the type of the data currently being stored in the union instance. For more detail see: https://doc.rust-lang.org/reference/items/unions.html.

Using Miri to Check Unsafe Code

- Miri is a dynamic tool that works at runtime to ensure code is safe and correct. It requires a nightly build of Rust. You can install both by running: `rustup +nightly component add miri`. Then to run Miri on your project, you simply type `cargo +nightly miri run` or `cargo +nightly miri test`. There is no need to change your project to build with nightly by default. Running `cargo +nightly miri test` in CI is standard practice for Rust code using the `unsafe` keyword, but most production binaries are built with stable.

Using Unsafe Code Correctly

- Using `unsafe` to use one of the superpowers is not wrong or even frowned upon, but it is trickier to get `unsafe` code correct because the compiler can't help uphold memory safety. The Rustonomicon is the official guide for `unsafe`.
- Among other things, using `unsafe` code or a safe wrapper around `unsafe` code is the idiomatic way to get two mutable references to disjoint indices in an array, allow values to be uninitialized when they are not being read, and having a reference to one field of a struct sit in another field of the same struct. But returning a pointer to a stack-allocated variable is never valid to do with unsafe code. The only appropriate workaround is to return an `Rc<T>` instead of an `&T`.

20.2 - Advanced Traits

Defining Traits with Associated Types

- *Associated types* connect a type placeholder with a trait such that the trait method definitions can use these placeholder types in their signatures. The implementor of a trait will specify the concrete type. You may recall seeing them in the `Iterator` chapter.

```
pub trait Iterator {
    type Item;

    fn next(&mut self) -> Option<Self::Item>;
}
```

- This is similar to generics, but when a trait has a generic parameter, it can be implemented for a type multiple times, changing the concrete type of the generic type parameters each time. So when we used the `next` method, it would require type annotations to indicate which implementation of `Iterator` we want to use. With associated traits, we don't need

to annotate types, because we can't implement a trait on a type multiple times. There is only one `impl Iterator for Counter`, so there is only one type for `Item.`

Using Default Generic Parameters and Operator Overloading

- When we use generic type parameters, we can specify a default concrete type for the generic type. This eliminates the need for implementors of the trait to specify a concrete type if the default type works. You do this using the `<PlaceholderType=ConcreteType>` syntax. This can be especially useful for *operator overloading*. Rust doesn't allow you to create your own operators or overload arbitrary operators, but you can overload the operations and corresponding traits listed in `std::ops` by implementing the traits associated with the operator.

```rust
use std::ops::Add;

#[derive(Debug, Copy, Clone, PartialEq)]
struct Point {
    x: i32,
    y: i32,
}

impl Add for Point {
    type Output = Point;

    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}

fn main() {
    assert_eq!(
        Point { x: 1, y: 0 } + Point { x: 2, y: 3 },
        Point { x: 3, y: 3 }
    );
}

// This relies on a default generic type in the `Add` trait definition.
trait Add<Rhs=Self> {
    type Output;

    fn add(self, rhs: Rhs) -> Self::Output;
}
```

- If we don't specify a type when implementing the `Add` trait, it defaults to `Self`, the type of what we are implementing the method for. Instead of adding two `Point` values, we could implement addition for a `Point` and something else. Here is an example of adding `Millimeters` to `Meters`:

```rust
use std::ops::Add;

struct Millimeters(u32);
struct Meters(u32);

impl Add<Meters> for Millimeters {
    type Output = Millimeters;

    fn add(self, other: Meters) -> Millimeters {
        Millimeters(self.0 + (other.0 * 1000))
    }
}
```

- Default type parameters are used in two main ways:
    1. To extend a type without breaking existing code
    2. To allow customization in specific cases most users won't need

Disambiguating Between Identically Named Methods

- For multiple implementations of a method, we use the qualified name and pass it a reference to `self`.

```rust
trait Pilot {
    fn fly(&self);
}

trait Wizard {
    fn fly(&self);
}

struct Human;

impl Pilot for Human {
    fn fly(&self) {
        println!("This is your captain speaking.");
    }
}

impl Wizard for Human {
    fn fly(&self) {
        println!("Up!");
    }
}

impl Human {
    fn fly(&self) {
        println!("*waving arms furiously*");
    }
}

fn main() {
    let person = Human;
    Pilot::fly(&person);
    Wizard::fly(&person);
    person.fly();
}

/* OUTPUT:

This is your captain speaking.
Up!
*waving arms furiously*  */
```

- However, associated functions that are not methods don't have a `self` parameter. When there are multiple types or traits that define non-method functions with the same function name, Rust doesn't always know which type you mean unless you use fully qualified syntax.

```
trait Animal {
    fn baby_name() -> String;
}

struct Dog;

impl Dog {
    fn baby_name() -> String {
        String::from("Spot")
    }
}

impl Animal for Dog {
    fn baby_name() -> String {
        String::from("puppy")
    }
}

fn main() {
    // This prints "A baby dog is called Spot", the `Dog` associated
function.
    println!("A baby dog is called a {}", Dog::baby_name());
    // This prints "A baby dog is called puppy"
    println!("A baby dog is called a {}", <Dog as Animal>::baby_name());
}
```

- In general, fully qualified syntax is defined as follows:

```
<Type as Trait>::function(receiver_if_method, next_arg, ...);
```

Using Supertraits

- Sometimes you might write a trait definition that depends on another trait: For a type to implement the first trait, you want to require that type to also implement the second trait. That way your trait definition can make use of the associated items of the second trait. The trait of your trait definition is relying on is called the *supertrait* of your trait. Here is an example where we define `OutlinePrint` to require an implementation of `fmt::Display`. It is very similar to adding a trait bound to the trait.

```rust
use std::fmt;

trait OutlinePrint: fmt::Display {
    fn outline_print(&self) {
        let output = self.to_string();
        let len = output.len();
        println!("{}", "*".repeat(len + 4));
        println!("*{}*", " ".repeat(len + 2));
        println!("* {output} *");
        println!("*{}*", " ".repeat(len + 2));
        println!("{}", "*".repeat(len + 4));
    }
}

struct Point {
    x: i32,
    y: i32,
}

impl OutlinePrint for Point {}

impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}
```

Implementing External Traits with the Newtype Pattern

- The newtype pattern involves creating a new type in a tuple struct. This is a good way to get around the restriction that we're only allowed to implement a trait on a type if either the trait or the type (or both) are local to our crate. The tuple struct has one field and is a thin wrapper around the type for which we want to implement a trait. Then the wrapper is local to our crate, so we can implement the trait for the wrapper. The wrapper type is elided at compiler time, so there is no runtime performance penalty.

```rust
use std::fmt;

struct Wrapper(Vec<String>);

// Neither `Display` nor `Vec<T>` are implemented in this crate, so we need
// a `Wrapper`.
impl fmt::Display for Wrapper {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}

fn main() {
    let w = Wrapper(vec![String::from("hello"), String::from("world")]);
    println!("w = {w}");
}
```

- The downside to this method is that `Wrapper` is a new type, so it doesn't have the methods of the value it's holding. We would have to implement any `Vec<T>` methods we needed directly on `Wrapper` such that the methods delegate to `self.0`. If we wanted the new type to have every method the inner type has, implementing the `Deref` trait on `Wrapper` to return the inner type solves this via deref coercion:

```rust
use std::fmt;
use std::ops::Deref;

struct Wrapper(Vec<String>);

impl fmt::Display for Wrapper {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}

impl Deref for Wrapper {
    type Target = Vec<String>;

    fn deref(&self) -> &Self::Target {
        &self.0
    }
}

fn main() {
    let w = Wrapper(vec![String::from("hello"), String::from("world")]);

    // Display implementation on Wrapper:
    println!("w = {w}");

    // Vec methods accessed transparently via deref coercion:
    println!("len: {}", w.len());
    println!("first element: {}", w[0]);
}
```

If we do *not* want `Wrapper` to expose all the underlying methods (e.g. to restrict the public API), we avoid `Deref` and manually implement only the specific methods we want.

20.3 - Advanced Types

Type Safety and Abstraction with the Newtype Pattern

- The newtype pattern has other uses, including statically enforcing that values are never confused and indicating the units of a value. A good example of the latter is last chapter when we defined `Millimeters` and `Meters` as wrappers around `u32`. If you have a function defined for `Millimeters`, it would be a compiler error to pass it a `Meters` value or even a plain `u32`. We can also use the newtype pattern to abstract away some

implementation details of a type: The new type can expose a public API that is different from the API of the private inner type. Newtypes can also hide internal implementation. If we wrap a `HashMap<i32, String>` in a `People` type with a public API, no one needs to know that we assign an `i32` ID to names internally. The newtype pattern is a lightweight way to achieve encapsulation to hide implementation details.

- Another major idiomatic use of newtypes (popularized by "Parse, don't validate" and *100 Exercises to Learn Rust*) is enforcing field invariants at the newtype boundary. Instead of a monolithic struct keeping all fields private and running a giant validation suite in its constructor, each field is wrapped in its own validated newtype. Because an invalid newtype can never be constructed, the enclosing struct's fields can safely remain public:

```rust
// Inner field is private; can only be constructed via TryFrom:
pub struct TicketTitle(String);

impl TryFrom<String> for TicketTitle {
    type Error = &'static str;

    fn try_from(value: String) -> Result<Self, Self::Error> {
        if value.is_empty() {
            Err("Title cannot be empty")
        } else if value.len() > 50 {
            Err("Title cannot exceed 50 characters")
        } else {
            Ok(TicketTitle(value))
        }
    }
}

// Because TicketTitle is guaranteed valid by construction, Ticket's
// fields can be public with no risk of invalid state:
pub struct Ticket {
    pub title: TicketTitle,
    pub id: u64,
}
```

Type Synonyms and Type Aliases

- Rust provides the ability to declare a *type alias* to give an existing type another name with the `type` keyword. We don't get the type-checking benefits of the newtype pattern, but it makes our code more descriptive of intended purpose. It also helps to decrease repetition of long types.

```rust
type Kilometers = i32;

let x: i32 = 5;
let y: Kilometers = 5;

println!("x + y = {}", x + y);

// Consider a long type: `Box<dyn Fn() + Send + 'static>`
let f: Box<dyn Fn() + Send + 'static> = Box::new(|| println!("hi"));

fn takes_long_type(f: Box<dyn Fn() + Send + 'static>) {
    // --snip--
}

fn returns_long_type() -> Box<dyn Fn() + Send + 'static> {
    // --snip--
}

// A type alias makes this code more manageable by reducing repetition.
type Thunk = Box<dyn Fn() + Send + 'static>;

let f: Thunk = Box::new(|| println!("hi"));

fn takes_long_type(f: Thunk) {
    // --snip--
}

fn returns_long_type() -> Thunk {
    // --snip--
}
```

- Type aliases are commonly used with `Result<T, E>` for reducing repetition. Because it's just an alias, we can still use all the standard `Result` methods and the `?` operator.

```rust
// Consider how often `Result<..., Error>` is used in the following code.
use std::fmt;
use std::io::Error;

pub trait Write {
    fn write(&mut self, buf: &[u8]) -> Result<usize, Error>;
    fn flush(&mut self) -> Result<(), Error>;

    fn write_all(&mut self, buf: &[u8]) -> Result<(), Error>;
    fn write_fmt(&mut self, fmt: fmt::Arguments) -> Result<(), Error>;
}

// Instead we can alias `Result<T>` with the `Error` filled in.
type Result<T> = std::result::Result<T, std::io::Error>;

pub trait Write {
    fn write(&mut self, buf: &[u8]) -> Result<usize>;
    fn flush(&mut self) -> Result<()>;

    fn write_all(&mut self, buf: &[u8]) -> Result<()>;
    fn write_fmt(&mut self, fmt: fmt::Arguments) -> Result<()>;
}
```

The Never Type That Never Returns

- Rust has a special type named `!` known as the *empty type* because it has no values. We normally call it the *never type* because it stands in the place of the return type when a function will never return. Functions that never return are called *diverging functions*. It is impossible to create types of the type `!`, so a function that returns `!` can never return.

```rust
fn bar() -> ! {
    print!("forever ");

    loop {
        print!("and ever ");
    }
}
```

- Remembering that `match` arms return the same type, consider the following code:

```rust
let guess: u32 = match guess.trim().parse() {
    Ok(num) => num,
    Err(_) => continue,
};
```

- But `continue` isn't the same type as `num`. `continue` has a `!` value. When Rust computes the type of `guess`, it looks at both match arms. Because `!` can never have a value, it concludes `u32` is the type of guess. The formal way to say this is that expressions of type `!` can be coerced into any other type. The `!` type also applies to calls to the `panic!` macro. Neither `continue` nor `panic!` return a value. One jumps back to the start of a `while` and the other ends the program.

Dynamically Sized Types and the `Sized` Trait

- Rust needs to know certain details of its types, such as how much space to allocate for a value of a particular type. This leaves *dynamically sized types* in an odd position. They are often referred to as *DSTs* or *unsized types* and let us write code using values whose size we can know only at runtime. `str` (but not `&str`) is a DST. In many cases, such as storing a string entered by a user, we don't know how long the string is until runtime. We can neither create a variable of type `str` nor take an argument of type `str`. The size of `&str` is known at compile time; it is twice the length of `usize` to store the memory address of where the string is located and its length. In general, this is how dynamically sized types are used in Rust. They contain metadata that stores the size of the dynamic information. The golden rule of dynamically sized types is that we must always put values of dynamically sized types behind a pointer of some kind.

- This works with all sorts of pointers such as `Box<str>` or `Rc<str>`. We've already been using a different dynamically sized type with smart pointers: traits. Every trait is a dynamically sized type we can refer to by using the name of the trait. To use traits as trait objects, we must put them behind a pointer such as `&dyn Trait` or `Box<dyn Trait>` or `Rc<dyn Trait>`.

- To work with DSTs, Rust provides the `Sized` trait to determine whether or not a type's size is known at compile time. This trait is automatically implemented for everything whose size is known at compile time. In addition, Rust implicitly adds a bound on `Sized` to every generic function. Because of this, passing `&str` is not suitable for a generic `&T`.

```rust
// This:
fn generic<T>(t: T) {
  // --snip--
}

// Is the same as this:
fn generic<T: Sized>(t: T) {
    // --snip--
}

// You can manually relax this restriction like this:
fn generic<T: ?Sized>(t: &T) {
    // --snip--
}
```

- Note above that we changed the argument in the `?Sized` version to take an `&T`. Since the parameter might not be `Sized`, we need to use it behind some kind of pointer.

Beyond the Book: Struct Defaults and the Default Trait

- Because Rust lacks default function parameters (e.g., `fn foo(port = 80)`), the idiomatic pattern for functions with many optional parameters is combining the `Default` trait with struct update syntax (`..`).
- Define a configuration struct, implement `Default` (or `#[derive(Default)]`), and override only the specific fields needed at the call site:

```rust
#[derive(Debug)]
pub struct ServerConfig {
    pub host: String,
    pub port: u16,
    pub max_connections: usize,
    pub timeout_seconds: u64,
    pub tls: bool,
}

impl Default for ServerConfig {
    fn default() -> Self {
        ServerConfig {
            host: String::from("127.0.0.1"),
            port: 8080,
            max_connections: 1000,
            timeout_seconds: 30,
            tls: false,
        }
    }
}

fn start_server(config: ServerConfig) {
    println!("Server running on {}:{}", config.host, config.port);
}

fn main() {
    // Override only port and tls; fill remaining fields from default:
    let config = ServerConfig {
        port: 443,
        tls: true,
        ..Default::default()
    };

    start_server(config);
}
```

- This provides named arguments at the call site and preserves backward compatibility: adding new fields with defaults to `ServerConfig` later never breaks existing callers.

Beyond the Book: The Builder Pattern

- The builder pattern provides a fluent, chained API for constructing complex structs with optional parameters, step-by-step configuration, and deferred validation.

- The consuming builder pattern (`mut self -> Self`) is the most common in Rust: each setter consumes and returns the builder by value, and `.build()` performs final validation:

```rust
pub struct Server {
    host: String,
    port: u16,
    tls: bool,
}

pub struct ServerBuilder {
    host: String,
    port: u16,
    tls: bool,
}

impl Server {
    pub fn builder() -> ServerBuilder {
        ServerBuilder {
            host: String::from("127.0.0.1"),
            port: 8080,
            tls: false,
        }
    }
}

impl ServerBuilder {
    pub fn host(mut self, host: impl Into<String>) -> Self {
        self.host = host.into();
        self
    }

    pub fn port(mut self, port: u16) -> Self {
        self.port = port;
        self
    }

    pub fn tls(mut self, tls: bool) -> Self {
        self.tls = tls;
        self
    }

    pub fn build(self) -> Result<Server, &'static str> {
        if self.port == 0 {
            return Err("Port must be non-zero");
        }
        Ok(Server {
            host: self.host,
            port: self.port,
```

```
            tls: self.tls,
        })
    }
}

fn main() {
    let server = Server::builder()
        .port(443)
        .tls(true)
        .build()
        .expect("Failed to build server");
}
```

- Taking `self` by value in `.build(self)` ensures the builder cannot be reused or left in an inconsistent state after construction.

20.4 - Advanced Functions and Closures

Function Pointers

- Much like you can pass a closure to a function, you can pass a function to a function. Functions coerce to the type `fn`, not to be confused with the `Fn` closure trait. `fn` is called a *function pointer*.

```
fn add_one(x: i32) -> i32 {
    x + 1
}

fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {
    f(arg) + f(arg)
}

fn main() {
    let answer = do_twice(add_one, 5);

    println!("The answer is: {answer}");
}
```

- Function pointers implement all three of the closure traits (`Fn`, `FnMut`, and `FnOnce`), meaning you can always pass a function pointer as an argument for a function that expects a closure. It is best to write functions using a generic type and one of the closure traits so your functions can accept either functions or closures. You would mainly want to only

accept functions with the `fn` syntax when interfacing with external code that doesn't have closures such as C. `map` is an example of a function that accepts both.

```
let list_of_numbers = vec![1, 2, 3];

// 1. Using an inline closure:
let list_of_strings: Vec<String> =
    list_of_numbers.iter().map(|i| i.to_string()).collect();

// 2. Using the ToString trait function pointer:
let list_of_strings: Vec<String> =
    list_of_numbers.iter().map(ToString::to_string).collect();

// 3. Or using the concrete type's method directly:
let list_of_strings: Vec<String> =
    list_of_numbers.iter().map(i32::to_string).collect();
```

- We cannot simply write `map(to_string)` because `to_string` is not a free-standing function in the global namespace; it is a method defined on the `ToString` trait. To reference a method as a function pointer, we must qualify its namespace--either via the trait (`ToString::to_string`) or directly via the concrete type (`i32::to_string`).
- You might recall that enum variants that take arguments become initializer functions. We can also pass them as arguments for methods that take closures.

```
enum Status {
    Value(u32),
    Stop,
}

let list_of_statuses: Vec<Status> = (0u32..20).map(Status::Value).collect();
```

- This method is interchangeable with passing the value to a closure, so use whichever feels more natural.

Returning Closures

- Closures are represented by traits, which means you can't return closures directly. You can only use the function pointer `fn` as return type if the closure doesn't capture any values from its scope.

```rust
// DOES NOT COMPILE! error[E0308]: mismatched types
fn main() {
    let handlers = vec![returns_closure(), returns_initialized_closure(123)];
    for handler in handlers {
        let output = handler(5);
        println!("{output}");
    }
}

fn returns_closure() -> impl Fn(i32) -> i32 {
    |x| x + 1
}

fn returns_initialized_closure(init: i32) -> impl Fn(i32) -> i32 {
    move |x| x + init
}
```

- The solution is to use a trait object.

```rust
fn returns_closure() -> Box<dyn Fn(i32) -> i32> {
    Box::new(|x| x + 1)
}

fn returns_initialized_closure(init: i32) -> Box<dyn Fn(i32) -> i32> {
    Box::new(move |x| x + init)
}
```

20.5 - Macros

- The term *macro* refers to a family of features in Rust - declarative macros with `macro_rules!` and three kinds of procedural macros:
  - Custom `#[derive]` macros that specify code added with the `derive` attribute used on structs and enums
  - Attribute-like macros that define custom attributes usable on any item
  - Function-like macros that look like function calls but operate on tokens specified as their argument

The Difference Between Macros and Functions

- Macros are a way of writing code that writes other code, which is known as *metaprogramming*. Macros *expand* to produce more code than the code you've written manually.
- A function signature must declare the number and type of parameter the function has. But macros can take a variable number of parameters. We can call `println!("hello")` and `println!("hello {}", name)`. Macros are also expanded before the compiler interprets the meaning of the code, so a macro can, for example, implement a trait on a given type. A function cannot because it gets called at runtime and a trait needs to be implemented at compile time. Macro definitions are generally more difficult to read, understand, and maintain than function definitions. You must also define macros or bring them into scope before you call them in a file, as opposed to functions you can define anywhere and call anywhere (i.e. sequentially before in the file where the macro is used; macros are processed top-to-bottom).

Declarative Macros for General Metaprogramming

- The most widely used form of macros in Rust is the *declarative macro*. These can also be referred to as "macros by example," "`macro_rules!` macros," or just plain "macros." They are like a `match` expression but can compare a value to patterns that are associated with particular code: In this situation, the value is the literal Rust source code passed to the macro; the patterns are compared with the structure of that source code; and the code associated with each pattern, when matched, replaces the code passed to the macro. This all happens during compilation.

- To define a macro, you use the `macro_rules!` construct.

```rust
// Slightly simplified
#[macro_export]
macro_rules! vec {
    ( $( $x:expr ),* ) => {
        {
            let mut temp_vec = Vec::new();
            $(
                temp_vec.push($x);
            )*
            temp_vec
        }
    };
}
let v: Vec<i32> = vec![1, 2, 3];
```

- This could be summarized as the following in between a static header and footer for declaring and returning `temp_vec`:

```
Match: ( $( $x:expr ),* ) => Repeat: $( temp_vec.push($x); )*
```

- This macro only has one arm, but a more complex macro could have multiple.
- Let's focus on the pattern to start. First, we use a set of parentheses to encompass the whole pattern. We use a dollar sign ($) to declare a variable in the macro system that will contain the Rust code matching the pattern. The dollar sign makes it clear this is a macro variable as opposed to a regular Rust variable. Next comes a set of parentheses that captures values that match the pattern within the parentheses for use in the replacement code. Within $() is $x:expr, which matches any Rust expression and gives the expression the name $x. The comma following `$()` indicates a literal comma. And the `*` specifies that the pattern matches zero or more of whatever precedes the `*`.
- Now let's look at the pattern in the body of code associated with this arm: `temp_vec.push()` within `$()*` is generated for each part that matches `$()` in the pattern zero or more times depending on how many times the pattern matches. The `$x` is replaced with each expression match.
- To learn more see The Little Book of Rust Macros.

Procedural Macros for Generating Code from Attributes

- The second form of macros is the procedural macro, which acts more like a function (and is a type of procedure). *Procedure macros* accept some code as an input, operate on that code, and produce some code as an output rather than matching against patterns and replacing the code with other code as declarative macros do. The three kinds of procedural macros are custom `derive`, attribute-like, and function-like, and all work in a similar fashion.
- When creating procedural macros, the definitions must reside in their own crate with a special crate type. In the following example `#[some_attribute]` is a placeholder for using a specific macro variety.

```
use proc_macro::TokenStream;

#[some_attribute]
pub fn some_name(input: TokenStream) -> TokenStream {
}
```

- The source code that the macro is operating on makes up the input `Tokenstream`, and the code the macro produces is the output `Tokenstream`. We could have multiple kinds of procedural macros in the same crate.

Custom `derive` Macros

- Instead of making users implement the `HelloMacro` trait for each of their types, we'll provide a procedure macro so users can annotate their type with `#[derive(HelloMacro)]` to get a default implementation of the `hello_macro` function.

```rust
// The code a user of our crate will be able to write.
use hello_macro::HelloMacro;
use hello_macro_derive::HelloMacro;

#[derive(HelloMacro)]
struct Pancakes;

fn main() {
    Pancakes::hello_macro();
}

// OUTPUT:
// Hello, Macro! My name is Pancakes!

// "hello_macro" crate from `cargo new hello_macro --lib`
// src/lib.rs
pub trait HelloMacro {
    fn hello_macro();
}

// "hello_macro_derive" crate also created with --lib
// hello_macro_derive/Cargo.toml
[package]
name = "hello_macro_derive"
version = "0.1.0"
edition = "2024"

[lib]
proc-macro = true

[dependencies]
syn = "2.0"
quote = "1.0"

// hello_macro_derive/src/lib.rs
use proc_macro::TokenStream;
use quote::quote;

#[proc_macro_derive(HelloMacro)]
pub fn hello_macro_derive(input: TokenStream) -> TokenStream {
    // Construct a representation of Rust code as a syntax tree
    // that we can manipulate.
    let ast = syn::parse(input).unwrap();

    // Build the trait implementation.
```

```
    impl_hello_macro(&ast)
}

fn impl_hello_macro(ast: &syn::DeriveInput) -> TokenStream {
    let name = &ast.ident;
    let generated = quote! {
        impl HelloMacro for #name {
            fn hello_macro() {
                println!("Hello, Macro! My name is {}!", stringify!(#name));
            }
        }
    };
    generated.into()
}

// "pancakes" crate created with `cargo new pancakes`
// pancakes/Cargo.toml
[package]
name = "pancakes"
version = "0.1.0"
edition = "2024"

[dependencies]
hello_macro = { path = "../hello_macro" }
hello_macro_derive = { path = "../hello_macro_derive" }

// pancakes/src/main.rs
use hello_macro::HelloMacro;
use hello_macro_derive::HelloMacro;

#[derive(HelloMacro)]
struct Pancakes;

fn main() {
    Pancakes::hello_macro();
}
```

- It's interesting that Rust doesn't allow a macro crate to export anything but macros, so we define our trait in `hello_macro`. It will be published separately, and users will have to depend on both.
- Note that `hello_macro_derive` will be the same function with a different name for most procedural macros. `impl_hello_macro` will be different depending on your procedural macro's purpose.

- In this example, we used `unwrap` because a macro should `panic!` on failure, but in Production code, you should display more information about what went wrong by manually calling `panic!` or at least using `expect`.

Attribute-Like Macros

- Attribute-like macros are similar to custom `derive` macros, but instead of generating code for the `derive` attribute, they allow you to create new attributes. They're also more flexible: `derive` only works for structs and enums; attributes can be applied to other items as well, such as functions.

```
// using an attribute macro
#[route(GET, "/")]
fn index() {}

// defined in the framework as a procedural macro like
#[proc_macro_attribute]
pub fn route(attr: TokenStream, item: TokenStream) -> TokenStream {}
```

- The first `TokenStream` is for the contents of the attribute: the `GET, "/"` part. The second is the body of the item the attribute is attached to: in this case, `fn index() {}` and the rest of the function's body.

- Other than that, attribute-like macros work the same way as custom derive macros: You create a crate with the proc-macro crate type and implement a function that generates the code you want!

Function-Like Macros

- Function-like macros define macros that look like function calls. Similarly to `macro_rules!` macros, they're more flexible than functions; for example, they can take an unknown number of arguments. However, `macro_rules!` macros can only be defined using the match-like syntax we discussed in the "Declarative Macros for General Metaprogramming" section earlier. Function-like macros take a `TokenStream` parameter, and their definition manipulates that `TokenStream` using Rust code as the other two types of procedural macros do.

```
let sql = sql!(SELECT * FROM posts WHERE id=1);
```

- This macro would parse the SQL statement inside it and check that it's syntactically correct, which is much more complex processing than a `macro_rules!` macro can do. The `sql!` macro would be defined like this:

```
#[proc_macro]
pub fn sql(input: TokenStream) -> TokenStream {}
```

- This definition is similar to the custom derive macro's signature: We receive the tokens that are inside the parentheses and return the code we wanted to generate.

Chapter 21 - Final Project: Building a Multithreaded Web Server

- The end-of-book project is a multi-threaded web server. It will not be a fine example of production code (use a crate for that), but it will serve the purpose of helping us learn and review. We're not using async and await.
- Version 1 simply accepts a connection from a browser and prints out "Connection established!" on the command-line (often many times for modern browsers).

```
use std::net::TcpListener;

fn main() {
    // `bind` returns a `Result` since it might fail.
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();

    // `incoming` returns an iterator over streams. Each stream is an open
    // connection. `unwrap` will terminate our program if the stream has any
    // errors.
    for stream in listener.incoming() {
        let stream = stream.unwrap();

        // output is on our command-line, not back to the client (yet)
        println!("Connection established!");
    }
}
```

- Version 2 reads from the connection and outputs the request to our terminal.

```rust
use std::{
    io::{BufReader, prelude::*},
    net::{TcpListener, TcpStream},
};

fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();

    for stream in listener.incoming() {
        let stream = stream.unwrap();

        handle_connection(stream);
    }
}

fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&stream);
    // `BufReader` implements `std::io::BufRead` to provide `lines`.
    let http_request: Vec<_> = buf_reader
        // lines returns a `Result<String, std::io::Error>`.
        .lines()
        .map(|result| result.unwrap())
        // A request ends with two newline characters in a row.
        .take_while(|line| !line.is_empty())
        .collect();

    // `{http_request:#?}` is pretty debug format.
    println!("Request: {http_request:#?}");
}
```

- And here's what our request looks like from Firefox:

```
Request: [
    "GET / HTTP/1.1",
     "Host: 127.0.0.1:7878",
     "User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; ...)
Firefox",
    "Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
     "Accept-Language: en-US,en;q=0.9",
     "Accept-Encoding: gzip, deflate, br, zstd",
    "Connection: keep-alive",
     "Upgrade-Insecure-Requests: 1",
     "Sec-Fetch-Dest: document",
    "Sec-Fetch-Mode: navigate",
     "Sec-Fetch-Site: none",
     "Sec-Fetch-User: ?1",
    "DNT: 1",
     "Sec-GPC: 1",
     "Priority: u=0, i",
]
```

- A request has the format:

```
Method Request-URI HTTP-Version CRLF
headers CRLF
message-body
```

- But GET requests have no body. We will write a response that looks like:

```
HTTP-Version Status-Code Reason-Phrase CRLF
headers CRLF
message-body
```

- We can write a response by replacing our final `println!` with:

```
    let response = "HTTP/1.1 200 OK\r\n\r\n";
    // `write_all` takes a `&[u8]` and can fail so returns a `Result`
    stream.write_all(response.as_bytes()).unwrap();
```

- This is the point where we get a blank page in our browser instead of an error. Let's actually return an HTML file:

```rust
use std::{
    fs,
    io::{BufReader, prelude::*},
    net::{TcpListener, TcpStream},
};
// --snip--
fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&stream);
    let http_request: Vec<_> = buf_reader
        .lines()
        .map(|result| result.unwrap())
        .take_while(|line| !line.is_empty())
        .collect();

    let status_line = "HTTP/1.1 200 OK";
    let contents = fs::read_to_string("hello.html").unwrap();
    let length = contents.len();

    let response = format!(
        "{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}"
    );

    stream.write_all(response.as_bytes()).unwrap();
}
```

- And we can display a page! Be sure to put a "hello.html" in your project root. But currently we display the same page no matter what the browser requests. Let's validate the request and selectively respond.

```rust
fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&stream);
    // the first `unwrap` is the `Option` from the iterator, the second
    // `unwrap` is the `Result` from `line`. We only want the first line now.
    let request_line = buf_reader.lines().next().unwrap().unwrap();

    if request_line == "GET / HTTP/1.1" {
        let status_line = "HTTP/1.1 200 OK";
        let contents = fs::read_to_string("hello.html").unwrap();
        let length = contents.len();

        let response = format!(
            "{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}"
        );

        stream.write_all(response.as_bytes()).unwrap();
    } else {
        let status_line = "HTTP/1.1 404 NOT FOUND";
        let contents = fs::read_to_string("404.html").unwrap();
        let length = contents.len();

        let response = format!(
            "{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}"
        );

        stream.write_all(response.as_bytes()).unwrap();
    }
}
```

- Now, we don't have any tests, so we shouldn't be refactoring. But it's easy enough to test manually, and it would be nice to have less repetition.

```rust
fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&stream);
    let request_line = buf_reader.lines().next().unwrap().unwrap();

    let (status_line, filename) = if request_line == "GET / HTTP/1.1" {
        ("HTTP/1.1 200 OK", "hello.html")
    } else {
        ("HTTP/1.1 404 NOT FOUND", "404.html")
    };

    let contents = fs::read_to_string(filename).unwrap();
    let length = contents.len();

    let response = format!(
        "{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}"
    );

    stream.write_all(response.as_bytes()).unwrap();
}
```

- Right now, our server can only handle one request at a time. This doesn't seem like a problem when all our requests come quickly from one browser, but let's simulate some slow requests with `sleep`.

```
use std::{
    fs,
    io::{BufReader, prelude::*},
    net::{TcpListener, TcpStream},
    thread,
    time::Duration,
};
// --snip--

fn handle_connection(mut stream: TcpStream) {
    // --snip--

    let (status_line, filename) = match &request_line[..] {
        "GET / HTTP/1.1" => ("HTTP/1.1 200 OK", "hello.html"),
        "GET /sleep HTTP/1.1" => {
            thread::sleep(Duration::from_secs(5));
            ("HTTP/1.1 200 OK", "hello.html")
        }
        _ => ("HTTP/1.1 404 NOT FOUND", "404.html"),
    };

    // --snip--
}
```

- When you're trying to design code, writing the client interface first can help guide your design. Write the API of the code so that it's structured in the way you want to call it; then, implement the functionality within that structure rather than implementing the functionality and then designing the public API.
- It would be simple to modify `main` to spawn a thread for every connection. The problem with that is we would easily succumb to a denial of service attack that exhausted our server's resources. Instead we will work with a pool of threads. But let's look at the naive implementation:

```
fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();

    for stream in listener.incoming() {
        let stream = stream.unwrap();

        thread::spawn(|| {
            handle_connection(stream);
        });
    }
}
```

- So if we want a similar API, we could imagine our `ThreadPool` being used like this:

```
fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();
    let pool = ThreadPool::new(4);

    for stream in listener.incoming() {
        let stream = stream.unwrap();

        pool.execute(|| {
            handle_connection(stream);
        });;
    }
}
```

- Now the book develops this step-by-step, driven by compiler messages. We add a `src/lib.rs`. Define a `ThreadPool` struct. Define a `new` method that takes a `usize` (because we'll use it as an index, and it doesn't make sense to be negative), and define an `execute` method. For the `execute` method since we weren't sure what type of closure to accept, we looked at the `thread::spawn` implementation in the standard library and saw that it took a `FnOnce` that had `Send` and `'static` restrictions, so we modeled our `execute` off that. We also saw that `thread::spawn` returns a `JoinHandle<T>`. Since we have no return value, we decided we would need to spawn threads and hold on to a `JoinHandle<()>` in our pool. Here is what we have after those steps:

```rust
// src/lib.rs
use std::thread;

pub struct ThreadPool {
    threads: Vec<thread::JoinHandle<()>>,
}

impl ThreadPool {
    /// Create a new ThreadPool.
    ///
    /// The size is the number of threads in the pool.
    ///
    /// # Panics
    ///
    /// The `new` function will panic if the size is zero.
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let mut threads = Vec::with_capacity(size);

        for _ in 0..size {
            // create some threads and store them in the vector
        }

        ThreadPool { threads }
    }

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
    }
}
```

- So normally spawned threads take a closure and run it. But we need to manage a way to send work to the thread after it's spawned. We are going to create a `Worker` data structure to perform this task. We are going to store these workers instead of `JoinHandle<()>`, and each `Worker` will store its own `JoinHandle<()>` instance. We'll give the worker a function that lets us pass it an `id` so we can distinguish between the different instances of `Worker`.

```rust
use std::thread;

pub struct ThreadPool {
    workers: Vec<Worker>,
}

impl ThreadPool {
    /// Create a new ThreadPool.
    ///
    /// The size is the number of threads in the pool.
    ///
    /// # Panics
    ///
    /// The `new` function will panic if the size is zero.
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id));
        }

        ThreadPool { workers }
    }

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
    }
}

struct Worker {
    id: usize,
    thread: thread::JoinHandle<()>,
}

impl Worker {
    fn new(id: usize) -> Worker {
        let thread = thread::spawn(|| {});

        Worker { id, thread }
    }
}
```

- Note that if `thread::spawn` fails, it panics and our server goes down with it, so in a production-grade thread pool, you would want to use `thread::Builder::new().spawn(...)` which returns a `Result`.
- We need a way to send jobs to our `Worker`s. Channels from `mpsc` seem like a good solution. We'll send our workers `Job`s that hold the closures.

```
use std::{sync::mpsc, thread};

pub struct ThreadPool {
    workers: Vec<Worker>,
    sender: mpsc::Sender<Job>,
}

struct Job;

impl ThreadPool {
    // --snip--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id));
        }

        ThreadPool { workers, sender }
    }
    // --snip--
}
```

- So we want to pass the `receiver` to our `Worker`s.

```
impl ThreadPool {
    // --snip--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id, receiver));
        }

        ThreadPool { workers, sender }
    }
    // --snip--
}

// --snip--

impl Worker {
    fn new(id: usize, receiver: mpsc::Receiver<Job>) -> Worker {
        let thread = thread::spawn(|| {
            receiver;
        });

        Worker { id, thread }
    }
}
```

- If we try to compile this though, we get an error. `error[E0382]: use of moved value: `receiver`` . We are trying to pass the `receiver` to multiple instances, but our channel implementation is multiple *producer*, single *consumer*. Additionally, taking a job off the channel requires mutating the `receiver` , so the threads need a safe way to share and modify the `receiver` . To share ownership across multiple threads and allow the threads to mutate the value, we need to use `Arc<Mutex<T>>` . The `Arc` allows multiple `Worker` instances to own the receiver, and `Mutex` will ensure that only one `Worker` gets a job from the receiver at a time.

```
use std::{
    sync::{Arc, Mutex, mpsc},
    thread,
};
// --snip--

impl ThreadPool {
    // --snip--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let receiver = Arc::new(Mutex::new(receiver));

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id, Arc::clone(&receiver)));
        }

        ThreadPool { workers, sender }
    }

    // --snip--
}

// --snip--

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        // --snip--
    }
}
```

- So now we need to implement `execute` so this actually works. We'll also change `Job` from a struct to a type alias for a trait object that holds the type of closure that `execute` receives, since we don't really need a struct.

```
// --snip--

type Job = Box<dyn FnOnce() + Send + 'static>;

impl ThreadPool {
    // --snip--
    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);

        self.sender.send(job).unwrap();
    }
}

// --snip--
```

- Now we need our `Worker` threads to actually poll the receiving end of the channel and execute these `Job`s.

```
impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        let thread = thread::spawn(move || {
            loop {
                let job = receiver.lock().unwrap().recv().unwrap();

                println!("Worker {id} got a job; executing.");

                job();
            }
        });

        Worker { id, thread }
    }
}
```

- If we had used `while let Ok(job) = receiver.lock().unwrap().recv()` instead of assigning to `job` in a `loop`, the `MutexGuard` would have lasted through the entire `while` block until the `job()` call finished, so a slow request would still cause other requests to wait to be processed. When we use `let job = receiver.lock().unwrap().recv().unwrap()`, any temporary values used in the

expression on the right-hand side of the equal sign are immediately dropped when the `let` statement ends. Other workers cannot receive jobs until the lock is dropped.

- Next we'll implement the `Drop` trait to call `join` on each of the threads in the pool so that they can finish the requests they're working on before closing. Then, we'll implement a way to tell the threads they should stop accepting new requests and shut down. None of this affects the parts of the code that handle executing closures.

- When the `ThreadPool` is dropped, our threads should all join to make sure they finish their work. To call `join` on our threads, we need ownership, so we can't do this through a reference, even a mutable one. We could use `take` and store the `JoinHandle` as an `Option<thread::JoinHandle<()>>`, but it's generally not a good idea to make something that will never be empty an `Option`. Instead, we will use `Vec::drain` that accepts a range parameter to specify which items to remove from a vector and returns an iterator of those items. Passing the `..` range syntax will remove *every* value from the vector.

```
impl Drop for ThreadPool {
    fn drop(&mut self) {
        for worker in &mut self.workers.drain(..) {
            println!("Shutting down worker {}", worker.id);

            worker.thread.join().unwrap();
        }
    }
}
```

- The code compiles, but if we try to run it the main thread will block forever when we try to exit, since the threads are all looping forever waiting on jobs and the `join` will never complete. To fix this we need a change to the `ThreadPool` `Drop` implementation as well as the `Worker` loop. I know I just said don't use `Option` for non-optional values, but we don't fiddle with our `sender` much, and we need to `take` it to `drop` it.

```rust
pub struct ThreadPool {
    workers: Vec<Worker>,
    sender: Option<mpsc::Sender<Job>>,
}
// --snip--
impl ThreadPool {
    pub fn new(size: usize) -> ThreadPool {
        // --snip--

        ThreadPool {
            workers,
            sender: Some(sender),
        }
    }

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);

        self.sender.as_ref().unwrap().send(job).unwrap();
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.sender.take());

        for worker in &mut self.workers.drain(..) {
            println!("Shutting down worker {}", worker.id);

            worker.thread.join().unwrap();
        }
    }
}
```

- In `execute`, we must call `self.sender.as_ref()` before `unwrap()`:
    - `execute` takes `&self` (a shared reference). Calling `self.sender.unwrap()` directly would consume the `Option` by value, which Rust forbids behind a shared reference (`cannot move out of self.sender`).

    - `.as_ref()` converts `&Option<Sender<Job>>` into `Option<&Sender<Job>>`, borrowing the inner sender rather than moving it out.
    - The first `.unwrap()` gets `&Sender<Job>` out of the `Some`.
    - Calling `.send(job)` only requires `&self` on the sender and returns a `Result`, which the second `.unwrap()` checks in case the channel closed.
- Dropping `sender` closes the channel, so the `recv` calls in our threads return an error. And with a small change, we can make them exit their infinite loops.

```
impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        let thread = thread::spawn(move || {
            loop {
                let message = receiver.lock().unwrap().recv();

                match message {
                    Ok(job) => {
                        println!("Worker {id} got a job; executing.");
                        job();
                    }
                    Err(_) => {
                        println!("Worker {id} disconnected; shutting down.");
                        break;
                    }
                }
            }
        });

        Worker { id, thread }
    }
}
```

- And now we'll modify `main` to only accept 2 requests before shutting down.

```rust
fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();
    let pool = ThreadPool::new(4);

    for stream in listener.incoming().take(2) {
        let stream = stream.unwrap();

        pool.execute(|| {
            handle_connection(stream);
        });
    }

    println!("Shutting down.");
}
```

- Output:

```
$ cargo run
   Compiling hello v0.1.0 (ch21_web_server)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.17s
     Running `target/debug/hello`
Worker 0 got a job; executing.
Shutting down.
Shutting down worker 0
Worker 2 got a job; executing.
Worker 1 disconnected; shutting down.
Worker 3 disconnected; shutting down.
Worker 0 disconnected; shutting down.
Shutting down worker 1
Shutting down worker 2
Worker 2 disconnected; shutting down.
Shutting down worker 3
```

- Is it a little overkill setting up a 4 thread pool for a server that can only take 2 requests? Maybe. But we didn't want it to take all day to demonstrate the graceful shutdown.

- Here's the full code for reference:

```rust
// src/main.rs
use hello::ThreadPool;
use std::{
    fs,
    io::{BufReader, prelude::*},
    net::{TcpListener, TcpStream},
    thread,
    time::Duration,
};

fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();
    let pool = ThreadPool::new(4);

    for stream in listener.incoming().take(2) {
        let stream = stream.unwrap();

        pool.execute(|| {
            handle_connection(stream);
        });
    }

    println!("Shutting down.");
}

fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&stream);
    let request_line = buf_reader.lines().next().unwrap().unwrap();

    let (status_line, filename) = match &request_line[..] {
        "GET / HTTP/1.1" => ("HTTP/1.1 200 OK", "hello.html"),
        "GET /sleep HTTP/1.1" => {
            thread::sleep(Duration::from_secs(5));
            ("HTTP/1.1 200 OK", "hello.html")
        }
        _ => ("HTTP/1.1 404 NOT FOUND", "404.html"),
    };

    let contents = fs::read_to_string(filename).unwrap();
    let length = contents.len();

    let response = format!(
        "{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}"
    );
```

```
    stream.write_all(response.as_bytes()).unwrap();
}
```

```rust
// src/lib.rs
use std::{
    sync::{Arc, Mutex, mpsc},
    thread,
};

pub struct ThreadPool {
    workers: Vec<Worker>,
    sender: Option<mpsc::Sender<Job>>,
}

type Job = Box<dyn FnOnce() + Send + 'static>;

impl ThreadPool {
    /// Create a new ThreadPool.
    ///
    /// The size is the number of threads in the pool.
    ///
    /// # Panics
    ///
    /// The `new` function will panic if the size is zero.
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let receiver = Arc::new(Mutex::new(receiver));

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id, Arc::clone(&receiver)));
        }

        ThreadPool {
            workers,
            sender: Some(sender),
        }
    }

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);
```

```
        self.sender.as_ref().unwrap().send(job).unwrap();
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.sender.take());

        for worker in &mut self.workers.drain(..) {
            println!("Shutting down worker {}", worker.id);

            worker.thread.join().unwrap();
        }
    }
}

struct Worker {
    id: usize,
    thread: thread::JoinHandle<()>,
}

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        let thread = thread::spawn(move || {
            loop {
                let message = receiver.lock().unwrap().recv();

                match message {
                    Ok(job) => {
                        println!("Worker {id} got a job; executing.");
                        job();
                    }
                    Err(_) => {
                        println!("Worker {id} disconnected; shutting down.");
                        break;
                    }
                }
            }
        });

        Worker { id, thread }
    }
}
```

Appendix: Ownership Inventories

The Ownership Inventories test understanding of ownership in real-world scenarios inspired by common compiler errors.

Ownership Inventory #1 (Ch 6)

Program 1: Returning Local Stack References

```
/// Makes a string to separate lines of text,
/// returning a default if the provided string is blank
fn make_separator(user_str: &str) -> &str {
    if user_str == "" {
        let default = "=".repeat(10);
        &default
    } else {
        user_str
    }
}
```

- **The Compiler Error:**
  - `cannot return reference to local variable default`.
  - **Why:** `default` is allocated on the stack within `make_separator`. When the function returns, its stack frame is deallocated, leaving `&default` pointing to dead stack memory.
- **The Undefined Behavior Trap:**
  - Calling `let s = make_separator(""); println!("{s}");` triggers the branch and returns a dangling pointer. Printing `s` reads deallocated stack memory.
- **The Fix:**
  - Return an owned `String` instead of a borrowed `&str`:

```
fn make_separator(user_str: &str) -> String {
    if user_str == "" {
        "=".repeat(10)
    } else {
        user_str.to_string()
    }
}
```

Program 2: Moving out of an Option Reference

```rust
/// Gets the string out of an option if it exists,
/// returning a default otherwise
fn get_or_default(arg: &Option<String>) -> String {
    if arg.is_none() {
        return String::new();
    }
    let s = arg.unwrap();
    s
}
```

- **The Compiler Error:**
  - `cannot move out of arg in arg.unwrap()`.
  - **Why:** `Option::unwrap(self)` takes `self` by value (consuming it). Because `arg` is a borrowed reference `&Option<String>`, you cannot move the inner `String` out through the reference.
- **The Undefined Behavior Trap:**
  - If allowed, passing `&opt` and moving the inner string into `s` causes a **double-free**: `s` frees the string when `get_or_default` returns, and `opt` frees the same heap memory again when its enclosing scope exits.
- **The Fix:**
  - Pattern match on `arg`, cloning only when `Some` is present:

```rust
fn get_or_default(arg: &Option<String>) -> String {
    match arg {
        None => String::new(),
        Some(s) => s.clone(),
    }
}
```

  - Or idiomatically: `arg.as_deref().unwrap_or("").to_string()`.

Ownership Inventory #2 (Ch 8)

Program 1: Mutating a Collection While Iterating

```rust
/// Removes all the zeros in-place from a vector of integers.
fn remove_zeros(v: &mut Vec<i32>) {
    for (i, t) in v.iter().enumerate().rev() {
        if *t == 0 {
            v.remove(i);
        }
    }
}
```

- **The Compiler Error:**
  - `v.remove(i) cannot borrow v as mutable`.
  - **Why:** Calling `v.iter()` creates an iterator that immutably borrows `v` for the entire loop. Calling `v.remove(i)` requires a mutable borrow of `v`, conflicting with the active shared borrow.
- **The Undefined Behavior Trap:**
  - This is **iterator invalidation**. In memory, `v.remove(i)` shifts all elements to the left and can reallocate/shrink the buffer. If allowed, subsequent iterations read out-of-bounds or deallocated heap memory.
- **The Fix:**
  - Use the standard library in-place retain method:

```rust
fn remove_zeros(v: &mut Vec<i32>) {
    v.retain(|t| *t != 0);
}
```

Program 2: Simultaneous Mutable Borrows of Disjoint Elements

```rust
/// Reverses the elements of a vector in-place
fn reverse(v: &mut Vec<String>) {
    let n = v.len();
    for i in 0..n / 2 {
        std::mem::swap(&mut v[i], &mut v[n - i - 1]);
    }
}
```

- **The Compiler Error:**
  - Cannot borrow `*v` as mutable more than once at a time.
  - **Why:** In Rust, `&mut v[i]` desugars to `IndexMut::index_mut(v, i)`, which borrows the entire vector `v` mutably. Trying to take `&mut v[n - i - 1]` in the same statement requires a second simultaneous mutable borrow of `v`.
- **Safety Analysis:**
  - This function cannot cause memory safety violations at runtime because `i != n - i - 1` for all loop iterations (disjoint elements). However, the borrow checker cannot inspect runtime index values.
- **The Fix:**
  - Use the standard library method `slice::swap`, which encapsulates the disjoint index borrow:

```
fn reverse(v: &mut Vec<String>) {
    let n = v.len();
    for i in 0..n / 2 {
        v.swap(i, n - i - 1);
    }
}
```

Ownership Inventory #3 (Ch 10)

Program 1: Mutating Through an Immutable Slice Reference

```
/// Returns the n-th largest element in a slice
fn find_nth<T: Ord + Clone>(elems: &[T], n: usize) -> T {
    elems.sort();
    let t = &elems[n];
    return t.clone();
}
```

- **The Compiler Error:**
  - `cannot borrow elems as mutable for sort`.
  - **Why:** `slice::sort` requires `&mut [T]`, but `elems` is an immutable shared slice `&[T]`.
- **Safety Analysis:**
  - Sorting an immutable slice wouldn't cause a memory leak or crash (sorting only moves elements around), but it violates the semantic contract of a read-only query.

- **The Fix:**
  - Collect borrowed references into a local vector, sort the references, and clone only the target element:

```
fn find_nth<T: Ord + Clone>(elems: &[T], n: usize) -> T {
    let mut elem_refs: Vec<&T> = elems.iter().collect();
    elem_refs.sort();
    elem_refs[n].clone()
}
```

Program 2: Inter-procedural Borrow Conflicts Across Struct Fields

```
struct TestResult {
    /// Student's scores on a test
    scores: Vec<usize>,

    /// A possible value to curve all scores
    curve: Option<usize>,
}

impl TestResult {
    pub fn get_curve(&self) -> &Option<usize> {
        &self.curve
    }

    pub fn apply_curve(&mut self) {
        if let Some(curve) = self.get_curve() {
            for score in self.scores.iter_mut() {
                *score += curve;
            }
        }
    }
}
```

- **The Compiler Error:**
  - In `apply_curve`, cannot borrow `self.scores` as mutable for `iter_mut`.
  - **Why:** By lifetime elision, `get_curve(&'a self) -> &'a Option<usize>` borrows the entire `self` struct for lifetime `'a`. That shared borrow remains active inside the `if let Some` block, preventing the simultaneous mutable borrow of `self.scores`.
- **Safety Analysis:**

- This code is completely memory-safe because `self.curve` and `self.scores` occupy disjoint memory locations. The compiler rejects it because method signatures treat `&self` as an atomic unit rather than tracking individual struct fields across function boundaries.

- **The Fix:**
  - Inline the field access or copy the `Copy` field directly:

```rust
pub fn apply_curve(&mut self) {
    if let Some(curve) = self.curve {
        for score in self.scores.iter_mut() {
            *score += curve;
        }
    }
}
```

Ownership Inventory #4 (Ch 18)

Program 1: Trait Object Lifetimes

```rust
/// Adds a Display-able object into a vector of Display trait objects
use std::fmt::Display;

fn add_displayable<T: Display>(
    v: &mut Vec<Box<dyn Display>>,
    t: T,
) {
    v.push(Box::new(t));
}
```

- **The Compiler Error:**
  - `error[E0310]: the parameter type `T` may not live long enough`.
  - **Why:** A trait object `Box<dyn Trait>` without an explicit lifetime bound defaults to `'static` (`Box<dyn Display + 'static>`). Casting `Box<T>` to `Box<dyn Display>` requires that `dyn Display` outlives the vector. Because the lifetime of `T` is unspecified, `T` could contain references that do not live for `'static`.
- **The Undefined Behavior Trap:**
  - If the compiler allowed this without a lifetime check, the following program would compile and cause undefined behavior (use-after-free):

```
let mut v: Vec<Box<dyn Display>> = Vec::new();
{
    let n = 0;
    add_displayable(&mut v, &n);
}
println!("{}", v[0]); // READS DEALLOCATED STACK MEMORY!
```

  - Passing `&n` binds `T` to a short-lived local reference `&'local i32`. When `n` drops at the end of the inner scope, `v[0]` is left with a dangling pointer. Calling `Display::fmt` on `v[0]` reads dead memory.

- **The Idiomatic Fix:**
  - Relate the lifetime of `T` to the lifetime of the trait object by adding an explicit lifetime parameter `'a`:

```
fn add_displayable<'a, T: Display + 'a>(
    v: &mut Vec<Box<dyn Display + 'a>>,
    t: T,
) {
    v.push(Box::new(t));
}
```

  - **Why not `T: Display + 'static`?** Adding `'static` compiles, but it is overly restrictive: it prevents the vector from ever storing trait objects that borrow data.

  - **Why not `&'a mut Vec<..>`?** Placing `'a` on the `Vec` reference only bounds how long the vector is borrowed; it does not communicate how long the data inside the `Box` is allowed to live.

Appendix: Design Trade-offs (Ch 18)

To write effective Rust, knowing language mechanics is insufficient; you must select the right design patterns for architectural requirements. This section covers four real-world design challenges from Brown's edition:

Challenge 1: References (Asset Manager)

Inspiration: Bevy assets, Petgraph node indices, Cargo units.

```rust
pub struct AssetManager { /* .. */ }

// Option 1: return an immutable reference
impl AssetManager {
    pub fn load(&mut self, path: PathBuf) -> &Asset;
}

// Option 2: return a mutable reference
impl AssetManager {
    pub fn load(&mut self, path: PathBuf) -> &mut Asset;
}

// Option 3: return a reference-counted pointer
impl AssetManager {
    pub fn load(&mut self, path: PathBuf) -> Rc<Asset>;
}

// Option 4: return a copyable handle turned into a reference on demand
#[derive(Copy, Clone)]
pub struct AssetHandle(usize);

impl AssetManager {
    pub fn load(&mut self, path: PathBuf) -> AssetHandle;
    pub fn get(&self, handle: AssetHandle) -> &Asset;
}
```

- **Requirement 1: Once loaded, an asset should be permanently immutable.**
  - **Answers:** Options 1, 3, 4.
  - **Why:** Immutability is enforced by types disallowing mutation. `&Asset`, `Rc<Asset>`, and `AssetHandle` only grant read-only access. Option 2 returns `&mut Asset`, which allows clients to mutate the asset in-place.
- **Requirement 2: Clients need to retain access to assets across several short-term borrows of the asset manager.**
  - **Answers:** Options 3, 4.
  - **Why:** In code like `let a = mgr.load("p1"); process(a); mgr.load("p2");`, Options 1 and 2 tie the returned reference lifetime to `&mut self`. Holding `a: &Asset` keeps the manager borrowed, so calling `mgr.load` again triggers an aliasing error. Options 3 (`Rc`) and 4 (`AssetHandle`) return independent handles with lifetimes completely decoupled from the manager borrow.
- **Requirement 3: All assets must be deallocated at a single, predictable time.**
  - **Answers:** Options 1, 2, 4.

- **Why:** The manager retains exclusive ownership in 1, 2, and 4. Dropping the manager guarantees immediate, deterministic deallocation of all assets (Option 4 panics or returns `None` if an old handle is queried later). In Option 3 (`Rc`), ownership is shared; external holders of `Rc<Asset>` keep assets alive indefinitely even after the manager is dropped.

Challenge 2: Trait Trees (Widget Tree)

Inspiration: Yew components, Druid widgets.

```rust
// Option 1: children must be Self
pub trait Widget: Sized {
    fn render(&self) -> Vec<Self>;
}

// Option 2: children are a trait parameter
pub trait Widget<Children> {
    fn render(&self) -> Vec<Children>;
}

// Option 3: children are an associated type
pub trait Widget {
    type Children: Widget;
    fn render(&self) -> Vec<Self::Children>;
}

// Option 4: children are a reference trait object
pub trait Widget {
    fn render(&self) -> Vec<&dyn Widget>;
}

// Option 5: children are a boxed trait object
pub trait Widget {
    fn render(&self) -> Vec<Box<dyn Widget>>;
}
```

- **Requirement 1: Client provides a single `WidgetImpl` enum for all widgets, and implements `Widget` for `WidgetImpl`.**
  - **Answers:** Option 1.
  - **Why:** With an enum representing all widgets, the system is "closed-world" and non-extensible. `Self` is `WidgetImpl`, so `render` returns other variants directly. There is zero dynamic dispatch, zero heap allocation, and contiguous storage in `Vec`. Options

2-5 introduce unneeded generics, associated types, or force boxing onto an enum architecture.

- **Requirement 2: Client provides a unique struct per widget type, and each widget can return a vector of any possible widget type.**
  - **Answers:** Options 4, 5.
  - **Why:** Distinct structs require a *heterogeneous* vector. Options 1, 2, and 3 constrain each `render` implementation to a single concrete child type at compile time. Only trait objects (`dyn Widget`) in Options 4 and 5 allow mixing disparate struct types within the same `Vec` .
- **Requirement 3: Only types implementing `Widget` should be returned from `render` .**
  - **Answers:** Options 1, 3, 4, 5.
  - **Why:** Option 2 declares `Widget<Children>` without a `Children: Widget` trait bound, allowing `render` to return any arbitrary type. Options 1, 3, 4, and 5 all constrain the return type to implement `Widget` .

Challenge 3: Dispatch (Event Callback Registration)

Inspiration: Bevy systems, Diesel queries, Axum handlers.

```rust
pub struct Events { /* .. */ }

// Option 1: separate methods
impl Events {
    pub fn register<E, F: Fn(E)>(&mut self, f: F) { /* .. */ }
    pub fn register_sequential<E, F: Fn(E)>(&mut self, f: F) { /* .. */ }
}

// Option 2: enum wrapper
pub enum Callback<F> {
    Parallel(F),
    Sequential(F),
}

impl Events {
    pub fn register<E, F: Fn(E)>(&mut self, f: Callback<F>) { /* .. */ }
}

// Option 3: marker types with trait overloading
pub trait Register<Marker, F, E> {
    fn register(&mut self, f: F);
}

pub struct Parallel;
pub struct Sequential;

impl<F, E> Register<Parallel, F, E> for Events
where
    F: Fn(E),
{
    fn register(&mut self, f: F) { /* .. */ }
}

impl<F, E> Register<Sequential, F, E> for Events
where
    F: Fn(Sequential, E),
{
    fn register(&mut self, f: F) { /* .. */ }
}
```

- **Requirement 1: A callback should be considered parallelizable by default, and the API should reflect that default.**
    - **Answers:** Options 1, 3.

    - **Why:** In Options 1 and 3, passing a bare closure `events.register(|e| ..)` defaults to parallel. Opting into sequential requires extra syntax: calling `register_sequential` (Option 1) or adding a marker parameter `|_: Sequential, e|` (Option 3). In Option 2, callers must explicitly wrap every callback in either `Callback::Parallel` or `Callback::Sequential`, giving neither behavior default status.
- **Requirement 2: The API should export as few methods as possible.**
    - **Answers:** Options 2, 3.
    - **Why:** Options 2 and 3 export a single method (`register`). Option 1 exports two methods (`register` and `register_sequential`).
- **Requirement 3: The API should lean on the compiler's type inference as little as possible.**
    - **Answers:** Options 1, 2.
    - **Why:** Options 1 and 2 only infer closure types. Option 3 uses an advanced trait overloading trick: `Register` is parameterized on a hidden `Marker` type. The compiler must infer `Marker` from the closure's signature to pick the right trait implementation. If inference fails, users get cryptic errors.

Challenge 4: Intermediates (Data Serialization)

Inspiration: Serde vs. miniserde.

```rust
// Option 1: types serialize to an intermediate `Value` AST enum
pub enum Value {
    String(String),
    Number(isize),
}

pub trait Serialize {
    fn serialize(&self) -> Value;
}

fn value_to_json(value: Value) -> String { /* .. */ }

pub fn to_json(data: impl Serialize) -> String {
    let value = data.serialize();
    value_to_json(value)
}

// Option 2: types serialize via calls to a `Serializer` streaming interface
pub trait Serialize {
    fn serialize<S: Serializer>(&self, serializer: &mut S);
}

pub trait Serializer {
    fn serialize_string(&mut self, s: &str);
    fn serialize_number(&mut self, n: isize);
}

struct JsonSerializer {
    buffer: String,
}

impl Serializer for JsonSerializer { /* .. */ }

pub fn to_json(data: impl Serialize) -> String {
    let mut serializer = JsonSerializer { buffer: String::new() };
    data.serialize(&mut serializer);
    serializer.buffer
}
```

- **Requirement 1: Add a new data format without modifying existing `Serialize` implementations.**
    - **Answers:** Options 1, 2.

    - **Why:** Both decouple data types from output formats. In Option 1, `Value` serves as a standardized intermediate boundary; adding YAML just requires a `value_to_yaml(Value)` function. In Option 2, `Serialize` is generic over any `S: Serializer`; adding YAML just requires implementing `Serializer` for `YamlSerializer`. Existing `Serialize` implementations never change.
- **Requirement 2: Serialization should consume as little memory as possible.**
    - **Answers:** Option 2.
    - **Why:** Option 1 must allocate an intermediate heap AST (`Value`) for the entire data structure before converting to text. Option 2 directly streams tokens into the destination buffer with zero intermediate heap allocations.
- **Requirement 3: Serializing a type into multiple formats should increase compiled binary size as little as possible.**
    - **Answers:** Option 1.
    - **Why:** In Option 1, each type implements `Serialize` once, compiling a single machine-code function that produces a `Value`. In Option 2, `T::serialize<S>` is generic over `S: Serializer`. If a type is serialized to JSON, YAML, and CBOR, the compiler monomorphizes three distinct copies of `serialize` for that type, multiplying binary code size.

Appendix C - Derivable Traits

- Remember the standard library allows you to derive `Debug`, `PartialEq`, `Eq`, `PartialOrd`, `Ord`, `Clone`, `Copy`, `Hash`, and `Default`. Remember that `Copy` is seldom necessary and only derivable if all fields implement `Copy`. Likewise `Default` requires fields to implement `Default`, but most standard types have sane defaults. See the "Beyond the Book" section on `Default` to implement a custom `Default` of your own. For bonus points, make your default name "Lewis". A good snippet for this: `#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord, Hash)]` with `Default` implemented manually or occasionally derived, and an occasional `Copy` when all fields are primitives.

Appendix G - How Rust is Made and "Nightly Rust"

- Rust is on a six-week release cycle. The new release replaces the old as the supported release. Standard releases are backward-compatible and painless; potentially breaking syntax changes (such as reserving new keywords) are gated behind opt-in Editions every three years.

- It's easy to try nightly. `rustup toolchain install nightly` installs it, but stable is still your default. Use `rustup override set nightly` within a project to make that project default to nightly. Most features in nightly are still opt-in with feature flags (`#![feature(...)]`). Many features languish on nightly for a long time since the team is very conservative about stable.